Sage's Doctesting Framework

Release 10.10

The Sage Development Team

Oct 28, 2026

CONTENTS

1 Classes involved in doctesting 1

2 Classes for sources of doctests 23

3 Processes for running doctests 43

4 Parsing docstrings 75

5 Helper for attaching tolerance information to strings 103

6 Reporting doctest results 105

7 Detecting external software 119

8 Test the doctesting framework 129

9 Utility functions 155

10 Fixtures to help testing functionality 165

11 Indices and Tables 173

Python Module Index 175

Index 177

CHAPTER

ONE

CLASSES INVOLVED IN DOCTESTING

This module controls the various classes involved in doctesting.

AUTHORS:

- David Roe (2012-03-27) – initial version, based on Robert Bradshaw's code.

class sage.doctest.control.**DocTestController**(*options*, *args*)

Bases: SageObject

This class controls doctesting of files.

After creating it with appropriate options, call the *run()* method to run the doctests.

add_files()

Check for the flags '–all' and '–new'.

For each one present, this function adds the appropriate directories and files to the todo list.

EXAMPLES:

```
sage: from sage.doctest.control import (DocTestDefaults,
....:                                   DocTestController)
sage: from sage.env import SAGE_SRC
sage: import tempfile
sage: with tempfile.NamedTemporaryFile() as f:
....:     DD = DocTestDefaults(all=True, logfile=f.name)
....:     DC = DocTestController(DD, [])
....:     DC.add_files()
Doctesting ...
sage: os.path.join(SAGE_SRC, 'sage') in DC.files
True
```

```
>>> from sage.all import *
>>> from sage.doctest.control import (DocTestDefaults,
...                                   DocTestController)
>>> from sage.env import SAGE_SRC
>>> import tempfile
>>> with tempfile.NamedTemporaryFile() as f:
...     DD = DocTestDefaults(all=True, logfile=f.name)
...     DC = DocTestController(DD, [])
...     DC.add_files()
Doctesting ...
>>> os.path.join(SAGE_SRC, 'sage') in DC.files
True
```

```
sage: DD = DocTestDefaults(new = True)
sage: DC = DocTestController(DD, [])
sage: DC.add_files()
Doctesting ...
```

```
>>> from sage.all import *
>>> DD = DocTestDefaults(new = True)
>>> DC = DocTestController(DD, [])
>>> DC.add_files()
Doctesting ...
```

cleanup(*final=True*)

Run cleanup activities after actually running doctests.

In particular, saves the stats to disk and closes the logfile.

INPUT:

- `final` – whether to close the logfile

EXAMPLES:

```
sage: from sage.doctest.control import DocTestDefaults, DocTestController
sage: from sage.env import SAGE_SRC
sage: import os
sage: dirname = os.path.join(SAGE_SRC, 'sage', 'rings', 'all.py')
sage: DD = DocTestDefaults()

sage: DC = DocTestController(DD, [dirname])
sage: DC.expand_files_into_sources()
sage: DC.sources.sort(key=lambda s:s.basename)

sage: for i, source in enumerate(DC.sources):
....:     DC.stats[source.basename] = {'walltime': 0.1r * (i+1)}
....:

sage: DC.run()
Running doctests with ID ...
Doctesting 1 file.
.../rings/all.py
    [... tests, ...s wall]
----------------------------------------------------------------------
All tests passed!
----------------------------------------------------------------------
Total time for all tests: ... seconds
    cpu time: ... seconds
    cumulative wall time: ... seconds
Features detected...
0
sage: DC.cleanup()
```

```
>>> from sage.all import *
>>> from sage.doctest.control import DocTestDefaults, DocTestController
>>> from sage.env import SAGE_SRC
```

(continues on next page)

(continued from previous page)

```
>>> import os
>>> dirname = os.path.join(SAGE_SRC, 'sage', 'rings', 'all.py')
>>> DD = DocTestDefaults()

>>> DC = DocTestController(DD, [dirname])
>>> DC.expand_files_into_sources()
>>> DC.sources.sort(key=lambda s:s.basename)

>>> for i, source in enumerate(DC.sources):
...     DC.stats[source.basename] = {'walltime': 0.1 * (i+Integer(1))}
....:

>>> DC.run()
Running doctests with ID ...
Doctesting 1 file.
.../rings/all.py
    [... tests, ...s wall]
----------------------------------------------------------------------
All tests passed!
----------------------------------------------------------------------
Total time for all tests: ... seconds
    cpu time: ... seconds
    cumulative wall time: ... seconds
Features detected...
0
>>> DC.cleanup()
```

create_run_id()

Create the run id.

EXAMPLES:

```
sage: from sage.doctest.control import DocTestDefaults, DocTestController
sage: DC = DocTestController(DocTestDefaults(), [])
sage: DC.create_run_id()
Running doctests with ID ...
```

```
>>> from sage.all import *
>>> from sage.doctest.control import DocTestDefaults, DocTestController
>>> DC = DocTestController(DocTestDefaults(), [])
>>> DC.create_run_id()
Running doctests with ID ...
```

expand_files_into_sources()

Expand `self.files`, which may include directories, into a list of *sage.doctest.sources.FileDocTestSource*

This function also handles the optional command line option.

EXAMPLES:

```
sage: from sage.doctest.control import DocTestDefaults, DocTestController
sage: from sage.env import SAGE_SRC
```

(continues on next page)

(continued from previous page)

```
sage: import os
sage: dirname = os.path.join(SAGE_SRC, 'sage', 'doctest')
sage: DD = DocTestDefaults(optional='all')
sage: DC = DocTestController(DD, [dirname])
sage: DC.expand_files_into_sources()
sage: len(DC.sources)
15
sage: DC.sources[0].options.optional
True
```

```
>>> from sage.all import *
>>> from sage.doctest.control import DocTestDefaults, DocTestController
>>> from sage.env import SAGE_SRC
>>> import os
>>> dirname = os.path.join(SAGE_SRC, 'sage', 'doctest')
>>> DD = DocTestDefaults(optional='all')
>>> DC = DocTestController(DD, [dirname])
>>> DC.expand_files_into_sources()
>>> len(DC.sources)
15
>>> DC.sources[Integer(0)].options.optional
True
```

```
sage: DD = DocTestDefaults(optional='magma,guava')
sage: DC = DocTestController(DD, [dirname])
sage: DC.expand_files_into_sources()
sage: all(t in DC.sources[0].options.optional for t in ['magma','guava'])
True
```

```
>>> from sage.all import *
>>> DD = DocTestDefaults(optional='magma,guava')
>>> DC = DocTestController(DD, [dirname])
>>> DC.expand_files_into_sources()
>>> all(t in DC.sources[Integer(0)].options.optional for t in ['magma','guava
↪'])
True
```

We check that files are skipped appropriately:

```
sage: dirname = tmp_dir()
sage: filename = os.path.join(dirname, 'not_tested.py')
sage: with open(filename, 'w') as f:
....:     _ = f.write("#"*80 + "\n\n\n\n## nodoctest\n    sage: 1+1\n    4")
sage: DC = DocTestController(DD, [dirname])
sage: DC.expand_files_into_sources()
sage: DC.sources
[]
```

```
>>> from sage.all import *
>>> dirname = tmp_dir()
>>> filename = os.path.join(dirname, 'not_tested.py')
```

(continues on next page)

(continued from previous page)

```
>>> with open(filename, 'w') as f:
...     _ = f.write("#"*Integer(80) + "\n\n\n\n## nodoctest\n    sage: 1+1\n␣
↪  4")
>>> DC = DocTestController(DD, [dirname])
>>> DC.expand_files_into_sources()
>>> DC.sources
[]
```

The directory `sage/doctest/tests` contains `nodoctest.py` but the files should still be tested when that directory is explicitly given (as opposed to being recursed into):

```
sage: DC = DocTestController(DD, [os.path.join(SAGE_SRC, 'sage', 'doctest',
↪'tests')])
sage: DC.expand_files_into_sources()
sage: len(DC.sources) >= 10
True
```

```
>>> from sage.all import *
>>> DC = DocTestController(DD, [os.path.join(SAGE_SRC, 'sage', 'doctest',
↪'tests')])
>>> DC.expand_files_into_sources()
>>> len(DC.sources) >= Integer(10)
True
```

filter_sources()

EXAMPLES:

```
sage: from sage.doctest.control import DocTestDefaults, DocTestController
sage: from sage.env import SAGE_SRC
sage: import os
sage: dirname = os.path.join(SAGE_SRC, 'sage', 'doctest')
sage: DD = DocTestDefaults(failed=True)
sage: DC = DocTestController(DD, [dirname])
sage: DC.expand_files_into_sources()
sage: for i, source in enumerate(DC.sources):
....:     DC.stats[source.basename] = {'walltime': 0.1r * (i+1)}
sage: DC.stats['sage.doctest.control'] = {'failed': True, 'walltime': 1.0r}
sage: DC.filter_sources()
Only doctesting files that failed last test.
sage: len(DC.sources)
1
```

```
>>> from sage.all import *
>>> from sage.doctest.control import DocTestDefaults, DocTestController
>>> from sage.env import SAGE_SRC
>>> import os
>>> dirname = os.path.join(SAGE_SRC, 'sage', 'doctest')
>>> DD = DocTestDefaults(failed=True)
>>> DC = DocTestController(DD, [dirname])
>>> DC.expand_files_into_sources()
>>> for i, source in enumerate(DC.sources):
...     DC.stats[source.basename] = {'walltime': 0.1 * (i+Integer(1))}
```

(continues on next page)

(continued from previous page)

```
>>> DC.stats['sage.doctest.control'] = {'failed': True, 'walltime': 1.0}
>>> DC.filter_sources()
Only doctesting files that failed last test.
>>> len(DC.sources)
1
```

load_baseline_stats(*filename*)

Load baseline stats.

This must be a JSON file in the same format that *load_stats()* expects.

EXAMPLES:

```
sage: from sage.doctest.control import DocTestDefaults, DocTestController
sage: DC = DocTestController(DocTestDefaults(), [])
sage: import json
sage: filename = tmp_filename()
sage: with open(filename, 'w') as stats_file:
....:     json.dump({'sage.doctest.control':{'failed':True}}, stats_file)
sage: DC.load_baseline_stats(filename)
sage: DC.baseline_stats['sage.doctest.control']
{'failed': True}
```

```
>>> from sage.all import *
>>> from sage.doctest.control import DocTestDefaults, DocTestController
>>> DC = DocTestController(DocTestDefaults(), [])
>>> import json
>>> filename = tmp_filename()
>>> with open(filename, 'w') as stats_file:
...     json.dump({'sage.doctest.control':{'failed':True}}, stats_file)
>>> DC.load_baseline_stats(filename)
>>> DC.baseline_stats['sage.doctest.control']
{'failed': True}
```

If the file doesn't exist, nothing happens. If there is an error, print a message. In any case, leave the stats alone:

```
sage: d = tmp_dir()
sage: DC.load_baseline_stats(os.path.join(d))  # Cannot read a directory
Error loading baseline stats from ...
sage: DC.load_baseline_stats(os.path.join(d, "no_such_file"))
sage: DC.baseline_stats['sage.doctest.control']
{'failed': True}
```

```
>>> from sage.all import *
>>> d = tmp_dir()
>>> DC.load_baseline_stats(os.path.join(d))  # Cannot read a directory
Error loading baseline stats from ...
>>> DC.load_baseline_stats(os.path.join(d, "no_such_file"))
>>> DC.baseline_stats['sage.doctest.control']
{'failed': True}
```

load_environment()

Return the module that provides the global environment.

EXAMPLES:

```
sage: from sage.doctest.control import DocTestDefaults, DocTestController
sage: DC = DocTestController(DocTestDefaults(), [])
sage: 'BipartiteGraph' in DC.load_environment().__dict__
True
sage: DC = DocTestController(DocTestDefaults(environment='sage.doctest.all'),␣
↪[])
sage: 'BipartiteGraph' in  DC.load_environment().__dict__
False
sage: 'run_doctests' in DC.load_environment().__dict__
True
```

```
>>> from sage.all import *
>>> from sage.doctest.control import DocTestDefaults, DocTestController
>>> DC = DocTestController(DocTestDefaults(), [])
>>> 'BipartiteGraph' in DC.load_environment().__dict__
True
>>> DC = DocTestController(DocTestDefaults(environment='sage.doctest.all'),␣
↪[])
>>> 'BipartiteGraph' in  DC.load_environment().__dict__
False
>>> 'run_doctests' in DC.load_environment().__dict__
True
```

load_stats (*filename*)

Load stats from the most recent run(s).

Stats are stored as a JSON file, and include information on which files failed tests and the walltime used for execution of the doctests.

EXAMPLES:

```
sage: from sage.doctest.control import DocTestDefaults, DocTestController
sage: DC = DocTestController(DocTestDefaults(), [])
sage: import json
sage: filename = tmp_filename()
sage: with open(filename, 'w') as stats_file:
....:     json.dump({'sage.doctest.control': {'walltime': 1.0r}}, stats_file)
sage: DC.load_stats(filename)
sage: DC.stats['sage.doctest.control']
{'walltime': 1.0}
```

```
>>> from sage.all import *
>>> from sage.doctest.control import DocTestDefaults, DocTestController
>>> DC = DocTestController(DocTestDefaults(), [])
>>> import json
>>> filename = tmp_filename()
>>> with open(filename, 'w') as stats_file:
...     json.dump({'sage.doctest.control': {'walltime': 1.0}}, stats_file)
>>> DC.load_stats(filename)
```

(continues on next page)

(continued from previous page)

```
>>> DC.stats['sage.doctest.control']
{'walltime': 1.0}
```

If the file doesn't exist, nothing happens. If there is an error, print a message. In any case, leave the stats alone:

```
sage: d = tmp_dir()
sage: DC.load_stats(os.path.join(d))  # Cannot read a directory
Error loading stats from ...
sage: DC.load_stats(os.path.join(d, "no_such_file"))
sage: DC.stats['sage.doctest.control']
{'walltime': 1.0}
```

```
>>> from sage.all import *
>>> d = tmp_dir()
>>> DC.load_stats(os.path.join(d))  # Cannot read a directory
Error loading stats from ...
>>> DC.load_stats(os.path.join(d, "no_such_file"))
>>> DC.stats['sage.doctest.control']
{'walltime': 1.0}
```

log(*s*, *end='\n'*)

Log the string `s + end` (where `end` is a newline by default) to the logfile and print it to the standard output.

EXAMPLES:

```
sage: from sage.doctest.control import DocTestDefaults, DocTestController
sage: DD = DocTestDefaults(logfile=tmp_filename())
sage: DC = DocTestController(DD, [])
sage: DC.log("hello world")
hello world
sage: DC.logfile.close()
sage: with open(DD.logfile) as f:
....:     print(f.read())
hello world
```

```
>>> from sage.all import *
>>> from sage.doctest.control import DocTestDefaults, DocTestController
>>> DD = DocTestDefaults(logfile=tmp_filename())
>>> DC = DocTestController(DD, [])
>>> DC.log("hello world")
hello world
>>> DC.logfile.close()
>>> with open(DD.logfile) as f:
...     print(f.read())
hello world
```

In serial mode, check that logging works even if `stdout` is redirected:

```
sage: DD = DocTestDefaults(logfile=tmp_filename(), serial=True)
sage: DC = DocTestController(DD, [])
sage: from sage.doctest.forker import SageSpoofInOut
```

(continues on next page)

(continued from previous page)

```
sage: with open(os.devnull, 'w') as devnull:
....:     S = SageSpoofInOut(devnull)
....:     S.start_spoofing()
....:     DC.log("hello world")
....:     S.stop_spoofing()
hello world
sage: DC.logfile.close()
sage: with open(DD.logfile) as f:
....:     print(f.read())
hello world
```

```
>>> from sage.all import *
>>> DD = DocTestDefaults(logfile=tmp_filename(), serial=True)
>>> DC = DocTestController(DD, [])
>>> from sage.doctest.forker import SageSpoofInOut
>>> with open(os.devnull, 'w') as devnull:
...     S = SageSpoofInOut(devnull)
...     S.start_spoofing()
...     DC.log("hello world")
...     S.stop_spoofing()
hello world
>>> DC.logfile.close()
>>> with open(DD.logfile) as f:
...     print(f.read())
hello world
```

Check that no duplicate logs appear, even when forking (Issue #15244):

```
sage: DD = DocTestDefaults(logfile=tmp_filename())
sage: DC = DocTestController(DD, [])
sage: DC.log("hello world")
hello world
sage: if os.fork() == 0:
....:     DC.logfile.close()
....:     os._exit(0)
sage: DC.logfile.close()
sage: with open(DD.logfile) as f:
....:     print(f.read())
hello world
```

```
>>> from sage.all import *
>>> DD = DocTestDefaults(logfile=tmp_filename())
>>> DC = DocTestController(DD, [])
>>> DC.log("hello world")
hello world
>>> if os.fork() == Integer(0):
...     DC.logfile.close()
...     os._exit(Integer(0))
>>> DC.logfile.close()
>>> with open(DD.logfile) as f:
...     print(f.read())
hello world
```

run()

This function is called after initialization to set up and run all doctests.

EXAMPLES:

```
sage: from sage.doctest.control import DocTestDefaults, DocTestController
sage: from sage.env import SAGE_SRC
sage: import os
sage: DD = DocTestDefaults()
sage: filename = os.path.join(SAGE_SRC, "sage", "sets", "non_negative_
↪integers.py")
sage: DC = DocTestController(DD, [filename])
sage: DC.run()
Running doctests with ID ...
Doctesting 1 file.
.../sage/sets/non_negative_integers.py
    [... tests, ...s wall]
----------------------------------------------------------------------
All tests passed!
----------------------------------------------------------------------
Total time for all tests: ... seconds
    cpu time: ... seconds
    cumulative wall time: ... seconds
Features detected...
0
```

```
>>> from sage.all import *
>>> from sage.doctest.control import DocTestDefaults, DocTestController
>>> from sage.env import SAGE_SRC
>>> import os
>>> DD = DocTestDefaults()
>>> filename = os.path.join(SAGE_SRC, "sage", "sets", "non_negative_integers.
↪py")
>>> DC = DocTestController(DD, [filename])
>>> DC.run()
Running doctests with ID ...
Doctesting 1 file.
.../sage/sets/non_negative_integers.py
    [... tests, ...s wall]
----------------------------------------------------------------------
All tests passed!
----------------------------------------------------------------------
Total time for all tests: ... seconds
    cpu time: ... seconds
    cumulative wall time: ... seconds
Features detected...
0
```

We check that Issue #25378 is fixed (testing external packages while providing a logfile does not raise a ValueError: I/O operation on closed file):

```
sage: logfile = tmp_filename(ext='.log')
sage: DD = DocTestDefaults(optional=set(['sage', 'external']),␣
↪logfile=logfile)
```

(continues on next page)

(continued from previous page)

```
sage: filename = tmp_filename(ext='.py')
sage: DC = DocTestController(DD, [filename])
sage: DC.run()
Running doctests with ID ...
Using --optional=external,sage
Features to be detected: ...
Doctesting 1 file.
....py
    [0 tests, ...s wall]
----------------------------------------------------------------------
All tests passed!
----------------------------------------------------------------------
Total time for all tests: ... seconds
    cpu time: ... seconds
    cumulative wall time: ... seconds
Features detected...
0
```

```
>>> from sage.all import *
>>> logfile = tmp_filename(ext='.log')
>>> DD = DocTestDefaults(optional=set(['sage', 'external']), logfile=logfile)
>>> filename = tmp_filename(ext='.py')
>>> DC = DocTestController(DD, [filename])
>>> DC.run()
Running doctests with ID ...
Using --optional=external,sage
Features to be detected: ...
Doctesting 1 file.
....py
    [0 tests, ...s wall]
----------------------------------------------------------------------
All tests passed!
----------------------------------------------------------------------
Total time for all tests: ... seconds
    cpu time: ... seconds
    cumulative wall time: ... seconds
Features detected...
0
```

We test the `--hide` option (Issue #34185):

```
sage: from sage.doctest.control import test_hide
sage: filename = tmp_filename(ext='.py')
sage: with open(filename, 'w') as f:
....:     f.write(test_hide)
....:     f.close()
714
sage: DF = DocTestDefaults(hide='buckygen,all')
sage: DC = DocTestController(DF, [filename])
sage: DC.run()
Running doctests with ID ...
Using --optional=sage...
```

(continues on next page)

(continued from previous page)

```
Features to be detected: ...
Doctesting 1 file.
....py
    [4 tests, ...s wall]
----------------------------------------------------------------------
All tests passed!
----------------------------------------------------------------------
Total time for all tests: ... seconds
    cpu time: ... seconds
    cumulative wall time: ... seconds
Features detected...
0

sage: DF = DocTestDefaults(hide='benzene,optional')
sage: DC = DocTestController(DF, [filename])
sage: DC.run()
Running doctests with ID ...
Using --optional=sage
Features to be detected: ...
Doctesting 1 file.
....py
    [4 tests, ...s wall]
----------------------------------------------------------------------
All tests passed!
----------------------------------------------------------------------
Total time for all tests: ... seconds
    cpu time: ... seconds
    cumulative wall time: ... seconds
Features detected...
0
```

```
>>> from sage.all import *
>>> from sage.doctest.control import test_hide
>>> filename = tmp_filename(ext='.py')
>>> with open(filename, 'w') as f:
...     f.write(test_hide)
...     f.close()
714
>>> DF = DocTestDefaults(hide='buckygen,all')
>>> DC = DocTestController(DF, [filename])
>>> DC.run()
Running doctests with ID ...
Using --optional=sage...
Features to be detected: ...
Doctesting 1 file.
....py
    [4 tests, ...s wall]
----------------------------------------------------------------------
All tests passed!
----------------------------------------------------------------------
Total time for all tests: ... seconds
    cpu time: ... seconds
```

(continues on next page)

(continued from previous page)

```
    cumulative wall time: ... seconds
Features detected...
0

>>> DF = DocTestDefaults(hide='benzene,optional')
>>> DC = DocTestController(DF, [filename])
>>> DC.run()
Running doctests with ID ...
Using --optional=sage
Features to be detected: ...
Doctesting 1 file.
....py
    [4 tests, ...s wall]
----------------------------------------------------------------------
All tests passed!
----------------------------------------------------------------------
Total time for all tests: ... seconds
    cpu time: ... seconds
    cumulative wall time: ... seconds
Features detected...
0
```

Test *Features that have been hidden* message:

```
sage: DC.run()                                  # optional - meataxe
Running doctests with ID ...
Using --optional=sage
Features to be detected: ...
Doctesting 1 file.
....py
    [4 tests, ...s wall]
----------------------------------------------------------------------
All tests passed!
----------------------------------------------------------------------
Total time for all tests: ... seconds
    cpu time: ... seconds
    cumulative wall time: ... seconds
Features detected...
Features that have been hidden: ...meataxe...
0
```

```
>>> from sage.all import *
>>> DC.run()                                  # optional - meataxe
Running doctests with ID ...
Using --optional=sage
Features to be detected: ...
Doctesting 1 file.
....py
    [4 tests, ...s wall]
----------------------------------------------------------------------
All tests passed!
----------------------------------------------------------------------
```

(continues on next page)

(continued from previous page)

```
Total time for all tests: ... seconds
    cpu time: ... seconds
    cumulative wall time: ... seconds
Features detected...
Features that have been hidden: ...meataxe...
0
```

run_doctests()

Actually run the doctests.

This function is called by *run()*.

EXAMPLES:

```
sage: from sage.doctest.control import DocTestDefaults, DocTestController
sage: from sage.env import SAGE_SRC
sage: import os
sage: dirname = os.path.join(SAGE_SRC, 'sage', 'rings', 'homset.py')
sage: DD = DocTestDefaults()
sage: DC = DocTestController(DD, [dirname])
sage: DC.expand_files_into_sources()
sage: DC.run_doctests()
Doctesting 1 file.
.../sage/rings/homset.py
    [... tests, ...s wall]
----------------------------------------------------------------------
All tests passed!
----------------------------------------------------------------------
Total time for all tests: ... seconds
    cpu time: ... seconds
    cumulative wall time: ... seconds...
```

```
>>> from sage.all import *
>>> from sage.doctest.control import DocTestDefaults, DocTestController
>>> from sage.env import SAGE_SRC
>>> import os
>>> dirname = os.path.join(SAGE_SRC, 'sage', 'rings', 'homset.py')
>>> DD = DocTestDefaults()
>>> DC = DocTestController(DD, [dirname])
>>> DC.expand_files_into_sources()
>>> DC.run_doctests()
Doctesting 1 file.
.../sage/rings/homset.py
    [... tests, ...s wall]
----------------------------------------------------------------------
All tests passed!
----------------------------------------------------------------------
Total time for all tests: ... seconds
    cpu time: ... seconds
    cumulative wall time: ... seconds...
```

run_val_gdb(*testing=False*)

Spawns a subprocess to run tests under the control of gdb, lldb, or valgrind.

INPUT:

- `testing` – boolean (default: `False`); if `True` then the command to be run will be printed rather than a subprocess started

EXAMPLES:

Note that the command lines include unexpanded environment variables. It is safer to let the shell expand them than to expand them here and risk insufficient quoting.

```
sage: from sage.doctest.control import DocTestDefaults, DocTestController
sage: DD = DocTestDefaults(gdb=True)
sage: DC = DocTestController(DD, ["hello_world.py"])
sage: DC.run_val_gdb(testing=True)
exec gdb --eval-command="run" --args ...python... -m sage.doctest --serial...␣
↪--timeout=0... hello_world.py
```

```
>>> from sage.all import *
>>> from sage.doctest.control import DocTestDefaults, DocTestController
>>> DD = DocTestDefaults(gdb=True)
>>> DC = DocTestController(DD, ["hello_world.py"])
>>> DC.run_val_gdb(testing=True)
exec gdb --eval-command="run" --args ...python... -m sage.doctest --serial...␣
↪--timeout=0... hello_world.py
```

```
sage: DD = DocTestDefaults(valgrind=True, optional='all', timeout=172800)
sage: DC = DocTestController(DD, ["hello_world.py"])
sage: DC.run_val_gdb(testing=True)
exec valgrind --tool=memcheck --leak-resolution=high --leak-check=full --num-
↪callers=25 --suppressions=.../valgrind/pyalloc.supp --suppressions=.../
↪valgrind/sage.supp --suppressions=.../valgrind/sage-additional.supp --
↪suppressions=.../valgrind/valgrind-python.supp  --log-file=.../valgrind/
↪sage-memcheck.%p ...python... -m sage.doctest --serial... --timeout=172800..
↪. --optional=all hello_world.py
```

```
>>> from sage.all import *
>>> DD = DocTestDefaults(valgrind=True, optional='all',␣
↪timeout=Integer(172800))
>>> DC = DocTestController(DD, ["hello_world.py"])
>>> DC.run_val_gdb(testing=True)
exec valgrind --tool=memcheck --leak-resolution=high --leak-check=full --num-
↪callers=25 --suppressions=.../valgrind/pyalloc.supp --suppressions=.../
↪valgrind/sage.supp --suppressions=.../valgrind/sage-additional.supp --
↪suppressions=.../valgrind/valgrind-python.supp  --log-file=.../valgrind/
↪sage-memcheck.%p ...python... -m sage.doctest --serial... --timeout=172800..
↪. --optional=all hello_world.py
```

save_stats(*filename*)

Save stats from the most recent run as a JSON file.

WARNING: This function overwrites the file.

EXAMPLES:

```
sage: from sage.doctest.control import DocTestDefaults, DocTestController
sage: DC = DocTestController(DocTestDefaults(), [])
sage: DC.stats['sage.doctest.control'] = {'walltime': 1.0r}
sage: filename = tmp_filename()
sage: DC.save_stats(filename)
sage: import json
sage: with open(filename) as f:
....:     D = json.load(f)
sage: D['sage.doctest.control']
{'walltime': 1.0}
```

```
>>> from sage.all import *
>>> from sage.doctest.control import DocTestDefaults, DocTestController
>>> DC = DocTestController(DocTestDefaults(), [])
>>> DC.stats['sage.doctest.control'] = {'walltime': 1.0}
>>> filename = tmp_filename()
>>> DC.save_stats(filename)
>>> import json
>>> with open(filename) as f:
...     D = json.load(f)
>>> D['sage.doctest.control']
{'walltime': 1.0}
```

sort_sources()

This function sorts the sources so that slower doctests are run first.

EXAMPLES:

```
sage: from sage.doctest.control import DocTestDefaults, DocTestController
sage: from sage.env import SAGE_SRC
sage: import os
sage: dirname = os.path.join(SAGE_SRC, 'sage', 'doctest')
sage: DD = DocTestDefaults(nthreads=2)
sage: DC = DocTestController(DD, [dirname])
sage: DC.expand_files_into_sources()
sage: DC.sources.sort(key=lambda s:s.basename)
sage: for i, source in enumerate(DC.sources):
....:     DC.stats[source.basename] = {'walltime': 0.1r * (i+1)}
sage: DC.sort_sources()
Sorting sources by runtime so that slower doctests are run first....
sage: print("\n".join(source.basename for source in DC.sources))
sage.doctest.util
sage.doctest.test
sage.doctest.sources
sage.doctest.rif_tol
sage.doctest.reporting
sage.doctest.parsing_test
sage.doctest.parsing
sage.doctest.marked_output
sage.doctest.forker
sage.doctest.fixtures
sage.doctest.external
sage.doctest.control
```

(continues on next page)

(continued from previous page)

```
sage.doctest.check_tolerance
sage.doctest.all
sage.doctest
```

```
>>> from sage.all import *
>>> from sage.doctest.control import DocTestDefaults, DocTestController
>>> from sage.env import SAGE_SRC
>>> import os
>>> dirname = os.path.join(SAGE_SRC, 'sage', 'doctest')
>>> DD = DocTestDefaults(nthreads=Integer(2))
>>> DC = DocTestController(DD, [dirname])
>>> DC.expand_files_into_sources()
>>> DC.sources.sort(key=lambda s:s.basename)
>>> for i, source in enumerate(DC.sources):
...     DC.stats[source.basename] = {'walltime': 0.1 * (i+Integer(1))}
>>> DC.sort_sources()
Sorting sources by runtime so that slower doctests are run first....
>>> print("\n".join(source.basename for source in DC.sources))
sage.doctest.util
sage.doctest.test
sage.doctest.sources
sage.doctest.rif_tol
sage.doctest.reporting
sage.doctest.parsing_test
sage.doctest.parsing
sage.doctest.marked_output
sage.doctest.forker
sage.doctest.fixtures
sage.doctest.external
sage.doctest.control
sage.doctest.check_tolerance
sage.doctest.all
sage.doctest
```

source_baseline(*source*)

Return the `baseline_stats` value of `source`.

INPUT:

- `source` – a *DocTestSource* instance

OUTPUT: a dictionary

EXAMPLES:

```
sage: from sage.doctest.control import DocTestDefaults, DocTestController
sage: filename = sage.doctest.util.__file__
sage: DD = DocTestDefaults()
sage: DC = DocTestController(DD, [filename])
sage: DC.expand_files_into_sources()
sage: DC.source_baseline(DC.sources[0])
{}
```

```
>>> from sage.all import *
>>> from sage.doctest.control import DocTestDefaults, DocTestController
>>> filename = sage.doctest.util.__file__
>>> DD = DocTestDefaults()
>>> DC = DocTestController(DD, [filename])
>>> DC.expand_files_into_sources()
>>> DC.source_baseline(DC.sources[Integer(0)])
{}
```

class sage.doctest.control.**DocTestDefaults**(*runtest_default=False*, ***kwds*)

Bases: SageObject

This class is used for doctesting the Sage doctest module.

The interface of this object should be compatible with the `options` input to *DocTestController*, that is, the same interface as the argument object parsed by the `argparse.ArgumentParser` in `sage.doctest.__main__._make_parser`.

INPUT:

- `runtest_default` – boolean (default: `False`); if `True`, fills in attribute to be the same as the defaults defined in `sage-runtests`. If `False`, change defaults in a few places for use in doctests of the doctester, which is mostly to make doctesting more predictable.
- `**kwds` – attributes to override defaults

EXAMPLES:

```
sage: from sage.doctest.control import DocTestDefaults
sage: D = DocTestDefaults(); D
DocTestDefaults()
sage: D.timeout
-1
```

```
>>> from sage.all import *
>>> from sage.doctest.control import DocTestDefaults
>>> D = DocTestDefaults(); D
DocTestDefaults()
>>> D.timeout
-1
```

Keyword arguments become attributes:

```
sage: D = DocTestDefaults(timeout=100); D
DocTestDefaults(timeout=100)
sage: D.timeout
100
```

```
>>> from sage.all import *
>>> D = DocTestDefaults(timeout=Integer(100)); D
DocTestDefaults(timeout=100)
>>> D.timeout
100
```

The defaults for `sage-runtests`:

```
sage: D = DocTestDefaults(runtest_default=True); D
DocTestDefaults(abspath=False, file_iterations=0, global_iterations=0,
                optional='sage,optional', random_seed=None,
                stats_path='.../timings2.json')
```

```
>>> from sage.all import *
>>> D = DocTestDefaults(runtest_default=True); D
DocTestDefaults(abspath=False, file_iterations=0, global_iterations=0,
                optional='sage,optional', random_seed=None,
                stats_path='.../timings2.json')
```

class sage.doctest.control.**Logger**(**files*)

Bases: object

File-like object which implements writing to multiple files at once.

EXAMPLES:

```
sage: from sage.doctest.control import Logger
sage: with open(tmp_filename(), "w+") as t:
....:     L = Logger(sys.stdout, t)
....:     _ = L.write("hello world\n")
....:     _ = t.seek(0)
....:     t.read()
hello world
'hello world\n'
```

```
>>> from sage.all import *
>>> from sage.doctest.control import Logger
>>> with open(tmp_filename(), "w+") as t:
...     L = Logger(sys.stdout, t)
...     _ = L.write("hello world\n")
...     _ = t.seek(Integer(0))
...     t.read()
hello world
'hello world\n'
```

flush()

Flush all files.

write(*x*)

Write x to all files.

sage.doctest.control.**run_doctests**(*module*, *options=None*)

Run the doctests in a given file.

INPUT:

- module – a Sage module, a string, or a list of such
- options – a DocTestDefaults object or None

EXAMPLES:

```
sage: run_doctests(sage.rings.all)
Running doctests with ID ...
Doctesting 1 file.
.../sage/rings/all.py
    [... tests, ...s wall]
----------------------------------------------------------------------
All tests passed!
----------------------------------------------------------------------
Total time for all tests: ... seconds
    cpu time: ... seconds
    cumulative wall time: ... seconds
Features detected...
```

```
>>> from sage.all import *
>>> run_doctests(sage.rings.all)
Running doctests with ID ...
Doctesting 1 file.
.../sage/rings/all.py
    [... tests, ...s wall]
----------------------------------------------------------------------
All tests passed!
----------------------------------------------------------------------
Total time for all tests: ... seconds
    cpu time: ... seconds
    cumulative wall time: ... seconds
Features detected...
```

sage.doctest.control.**skipdir**(*dirname*)

Return `True` if and only if the directory `dirname` should not be doctested.

EXAMPLES:

```
sage: from sage.doctest.control import skipdir
sage: skipdir(sage.env.SAGE_SRC)
False
sage: skipdir(os.path.join(sage.env.SAGE_SRC, "sage", "doctest", "tests"))
True
```

```
>>> from sage.all import *
>>> from sage.doctest.control import skipdir
>>> skipdir(sage.env.SAGE_SRC)
False
>>> skipdir(os.path.join(sage.env.SAGE_SRC, "sage", "doctest", "tests"))
True
```

sage.doctest.control.**skipfile**(*filename*, *tested_optional_tags*, *if_installed*, *log=False*)

Return `True` if and only if the file `filename` should not be doctested.

INPUT:

- `filename` – name of a file
- `tested_optional_tags` – list or tuple or set of optional tags to test, or `False` (no optional test) or `True` (all optional tests)

- `if_installed` – boolean (default: `False`); whether to skip Python/Cython files that are not installed as modules
- `log` – function to call with log messages, or `None`

If `filename` contains a line of the form `"# sage.doctest: optional - xyz")`, then this will return `False` if "xyz" is in `tested_optional_tags`. Otherwise, it returns the matching tag ("optional - xyz").

EXAMPLES:

```
sage: from sage.doctest.control import skipfile
sage: skipfile("skipme.c")
True
sage: filename = tmp_filename(ext='.pyx')
sage: skipfile(filename)
False
sage: with open(filename, "w") as f:
....:     _ = f.write("# nodoctest")
sage: skipfile(filename)
True
sage: with open(filename, "w") as f:
....:     _ = f.write("# sage.doctest: "    # broken in two source lines to avoid␣
↪the pattern
....:                 "optional - xyz")     # of relint (multiline_doctest_
↪comment)
sage: skipfile(filename, False)
'optional - xyz'
sage: bool(skipfile(filename, False))
True
sage: skipfile(filename, ['abc'])
'optional - xyz'
sage: skipfile(filename, ['abc', 'xyz'])
False
sage: skipfile(filename, True)
False
```

```
>>> from sage.all import *
>>> from sage.doctest.control import skipfile
>>> skipfile("skipme.c")
True
>>> filename = tmp_filename(ext='.pyx')
>>> skipfile(filename)
False
>>> with open(filename, "w") as f:
...     _ = f.write("# nodoctest")
>>> skipfile(filename)
True
>>> with open(filename, "w") as f:
...     _ = f.write("# sage.doctest: "    # broken in two source lines to avoid␣
↪the pattern
...                 "optional - xyz")     # of relint (multiline_doctest_comment)
>>> skipfile(filename, False)
'optional - xyz'
>>> bool(skipfile(filename, False))
True
```

(continues on next page)

(continued from previous page)

```
>>> skipfile(filename, ['abc'])
'optional - xyz'
>>> skipfile(filename, ['abc', 'xyz'])
False
>>> skipfile(filename, True)
False
```

CHAPTER

TWO

CLASSES FOR SOURCES OF DOCTESTS

This module defines various classes for sources from which doctests originate, such as files, functions or database entries.

AUTHORS:

- David Roe (2012-03-27) – initial version, based on Robert Bradshaw's code.

class sage.doctest.sources.**DictAsObject**(*attrs*)

Bases: dict

A simple subclass of dict that inserts the items from the initializing dictionary into attributes.

EXAMPLES:

```
sage: from sage.doctest.sources import DictAsObject
sage: D = DictAsObject({'a':2})
sage: D.a
2
```

```
>>> from sage.all import *
>>> from sage.doctest.sources import DictAsObject
>>> D = DictAsObject({'a':Integer(2)})
>>> D.a
2
```

class sage.doctest.sources.**DocTestSource**(*options*)

Bases: object

This class provides a common base class for different sources of doctests.

INPUT:

- options – a *sage.doctest.control.DocTestDefaults* instance or equivalent

_process_doc(*doctests*, *doc*, *namespace*, *start*)

Appends doctests defined in doc to the list doctests.

This function is called when a docstring block is completed (either by ending a triple quoted string in a Python file, unindenting from a comment block in a ReST file, or ending a verbatim or lstlisting environment in a LaTeX file).

INPUT:

- doctests – a running list of doctests to which the new test(s) will be appended
- doc – list of lines of a docstring, each including the trailing newline

- `namespace` – dictionary or *sage.doctest.util.RecordingDict*, used in the creation of new `doctest.DocTest` s
- `start` – integer giving the line number of the start of this docstring in the larger file

EXAMPLES:

```
sage: # long time
sage: from sage.doctest.control import DocTestDefaults
sage: from sage.doctest.sources import FileDocTestSource
sage: from sage.doctest.parsing import SageDocTestParser
sage: filename = sage.doctest.util.__file__
sage: FDS = FileDocTestSource(filename, DocTestDefaults())
sage: doctests, _ = FDS.create_doctests({})
sage: manual_doctests = []
sage: for dt in doctests:
....:     FDS.qualified_name = dt.name
....:     FDS._process_doc(manual_doctests, dt.docstring, {}, dt.lineno-1)
sage: doctests == manual_doctests
True
```

```
>>> from sage.all import *
>>> # long time
>>> from sage.doctest.control import DocTestDefaults
>>> from sage.doctest.sources import FileDocTestSource
>>> from sage.doctest.parsing import SageDocTestParser
>>> filename = sage.doctest.util.__file__
>>> FDS = FileDocTestSource(filename, DocTestDefaults())
>>> doctests, _ = FDS.create_doctests({})
>>> manual_doctests = []
>>> for dt in doctests:
...     FDS.qualified_name = dt.name
...     FDS._process_doc(manual_doctests, dt.docstring, {}, dt.lineno-
↪Integer(1))
>>> doctests == manual_doctests
True
```

file_optional_tags()

Return the set of tags that should apply to all doctests in this source.

This default implementation just returns the empty set.

EXAMPLES:

```
sage: from sage.doctest.control import DocTestDefaults
sage: from sage.doctest.sources import StringDocTestSource, PythonSource
sage: from sage.structure.dynamic_class import dynamic_class
sage: s = "'''\n    sage: 2 + 2\n    4\n'''"
sage: PythonStringSource = dynamic_class('PythonStringSource',␣
↪(StringDocTestSource, PythonSource))
sage: PSS = PythonStringSource('<runtime>', s, DocTestDefaults(), 'runtime')
sage: PSS.file_optional_tags
set()
```

```
>>> from sage.all import *
>>> from sage.doctest.control import DocTestDefaults
>>> from sage.doctest.sources import StringDocTestSource, PythonSource
>>> from sage.structure.dynamic_class import dynamic_class
>>> s = "'''\n    sage: 2 + 2\n    4\n'''"
>>> PythonStringSource = dynamic_class('PythonStringSource',␣
↪(StringDocTestSource, PythonSource))
>>> PSS = PythonStringSource('<runtime>', s, DocTestDefaults(), 'runtime')
>>> PSS.file_optional_tags
set()
```

class sage.doctest.sources.**FileDocTestSource**(*path*, *options*)

Bases: *DocTestSource*

This class creates doctests from a file.

INPUT:

- `path` – string; the filename
- `options` – a *sage.doctest.control.DocTestDefaults* instance or equivalent

EXAMPLES:

```
sage: from sage.doctest.control import DocTestDefaults
sage: from sage.doctest.sources import FileDocTestSource
sage: filename = sage.doctest.sources.__file__
sage: FDS = FileDocTestSource(filename, DocTestDefaults())
sage: FDS.basename
'sage.doctest.sources'
```

```
>>> from sage.all import *
>>> from sage.doctest.control import DocTestDefaults
>>> from sage.doctest.sources import FileDocTestSource
>>> filename = sage.doctest.sources.__file__
>>> FDS = FileDocTestSource(filename, DocTestDefaults())
>>> FDS.basename
'sage.doctest.sources'
```

basename()

The basename of this file source, e.g. `sage.doctest.sources`.

EXAMPLES:

```
sage: from sage.doctest.control import DocTestDefaults
sage: from sage.doctest.sources import FileDocTestSource
sage: filename = os.path.join(SAGE_SRC,'sage','rings','integer.pyx')
sage: FDS = FileDocTestSource(filename, DocTestDefaults())
sage: FDS.basename
'sage.rings.integer'
```

```
>>> from sage.all import *
>>> from sage.doctest.control import DocTestDefaults
>>> from sage.doctest.sources import FileDocTestSource
>>> filename = os.path.join(SAGE_SRC,'sage','rings','integer.pyx')
```

(continues on next page)

(continued from previous page)

```
>>> FDS = FileDocTestSource(filename, DocTestDefaults())
>>> FDS.basename
'sage.rings.integer'
```

create_doctests(*namespace*)

Return a list of doctests for this file.

INPUT:

- `namespace` – dictionary or *sage.doctest.util.RecordingDict*

OUTPUT:

- `doctests` – list of doctests defined in this file
- `extras` – dictionary

EXAMPLES:

```
sage: from sage.doctest.control import DocTestDefaults
sage: from sage.doctest.sources import FileDocTestSource
sage: filename = sage.doctest.sources.__file__
sage: FDS = FileDocTestSource(filename, DocTestDefaults())
sage: doctests, extras = FDS.create_doctests(globals())
sage: len(doctests)
43
sage: extras['tab']
False
```

```
>>> from sage.all import *
>>> from sage.doctest.control import DocTestDefaults
>>> from sage.doctest.sources import FileDocTestSource
>>> filename = sage.doctest.sources.__file__
>>> FDS = FileDocTestSource(filename, DocTestDefaults())
>>> doctests, extras = FDS.create_doctests(globals())
>>> len(doctests)
43
>>> extras['tab']
False
```

We give a self referential example:

```
sage: doctests[20].name
'sage.doctest.sources.FileDocTestSource.create_doctests'
sage: doctests[20].examples[8].source
'doctests[Integer(20)].examples[Integer(8)].source\n'
```

```
>>> from sage.all import *
>>> doctests[Integer(20)].name
'sage.doctest.sources.FileDocTestSource.create_doctests'
>>> doctests[Integer(20)].examples[Integer(8)].source
'doctests[Integer(20)].examples[Integer(8)].source\n'
```

file_optional_tags()

Return the set of tags that should apply to all doctests in this source.

EXAMPLES:

```
sage: from sage.doctest.control import DocTestDefaults
sage: from sage.doctest.sources import FileDocTestSource
sage: import sage.tests.numpy
sage: filename = sage.tests.numpy.__file__
sage: FDS = FileDocTestSource(filename, DocTestDefaults())
sage: FDS.file_optional_tags
{'numpy': None}
```

```
>>> from sage.all import *
>>> from sage.doctest.control import DocTestDefaults
>>> from sage.doctest.sources import FileDocTestSource
>>> import sage.tests.numpy
>>> filename = sage.tests.numpy.__file__
>>> FDS = FileDocTestSource(filename, DocTestDefaults())
>>> FDS.file_optional_tags
{'numpy': None}
```

in_lib()

Whether this file is to be treated as a module in a Python package.

Such files aren't loaded before running tests.

This uses `sage.misc.package_dir.is_package_or_sage_namespace_package_dir` but can be overridden via *DocTestDefaults*.

EXAMPLES:

```
sage: from sage.doctest.control import DocTestDefaults
sage: from sage.doctest.sources import FileDocTestSource
sage: import os
sage: filename = os.path.join(SAGE_SRC, 'sage', 'rings', 'integer.pyx')
sage: FDS = FileDocTestSource(filename, DocTestDefaults())
sage: FDS.in_lib
True
sage: filename = os.path.join(SAGE_SRC, 'sage', 'doctest', 'tests', 'abort.rst
↪')
sage: FDS = FileDocTestSource(filename, DocTestDefaults())
sage: FDS.in_lib
False
```

```
>>> from sage.all import *
>>> from sage.doctest.control import DocTestDefaults
>>> from sage.doctest.sources import FileDocTestSource
>>> import os
>>> filename = os.path.join(SAGE_SRC, 'sage', 'rings', 'integer.pyx')
>>> FDS = FileDocTestSource(filename, DocTestDefaults())
>>> FDS.in_lib
True
>>> filename = os.path.join(SAGE_SRC, 'sage', 'doctest', 'tests', 'abort.rst')
>>> FDS = FileDocTestSource(filename, DocTestDefaults())
>>> FDS.in_lib
False
```

You can override the default:

```
sage: FDS = FileDocTestSource("hello_world.py", DocTestDefaults())
sage: FDS.in_lib
False
sage: FDS = FileDocTestSource("hello_world.py", DocTestDefaults(force_
↪lib=True))
sage: FDS.in_lib
True
```

```
>>> from sage.all import *
>>> FDS = FileDocTestSource("hello_world.py", DocTestDefaults())
>>> FDS.in_lib
False
>>> FDS = FileDocTestSource("hello_world.py", DocTestDefaults(force_lib=True))
>>> FDS.in_lib
True
```

printpath()

Whether the path is printed absolutely or relatively depends on an option.

EXAMPLES:

```
sage: from sage.doctest.control import DocTestDefaults
sage: from sage.doctest.sources import FileDocTestSource
sage: import os
sage: filename = os.path.realpath(sage.doctest.sources.__file__)
sage: root = os.path.join(os.path.dirname(filename), '..')
sage: cwd = os.getcwd()
sage: os.chdir(root)
sage: FDS = FileDocTestSource(filename, DocTestDefaults(randorder=0,
....:                                                   abspath=False))
sage: FDS.printpath
'doctest/sources.py'
sage: FDS = FileDocTestSource(filename, DocTestDefaults(randorder=0,
....:                                                   abspath=True))
sage: FDS.printpath
'.../sage/doctest/sources.py'
sage: os.chdir(cwd)
```

```
>>> from sage.all import *
>>> from sage.doctest.control import DocTestDefaults
>>> from sage.doctest.sources import FileDocTestSource
>>> import os
>>> filename = os.path.realpath(sage.doctest.sources.__file__)
>>> root = os.path.join(os.path.dirname(filename), '..')
>>> cwd = os.getcwd()
>>> os.chdir(root)
>>> FDS = FileDocTestSource(filename, DocTestDefaults(randorder=Integer(0),
...                                                   abspath=False))
>>> FDS.printpath
'doctest/sources.py'
>>> FDS = FileDocTestSource(filename, DocTestDefaults(randorder=Integer(0),
```

(continues on next page)

(continued from previous page)

```
...                                                    abspath=True))
>>> FDS.printpath
'.../sage/doctest/sources.py'
>>> os.chdir(cwd)
```

class sage.doctest.sources.**PythonSource**

Bases: *SourceLanguage*

This class defines the functions needed for the extraction of doctests from python sources.

EXAMPLES:

```
sage: from sage.doctest.control import DocTestDefaults
sage: from sage.doctest.sources import FileDocTestSource
sage: filename = sage.doctest.sources.__file__
sage: FDS = FileDocTestSource(filename, DocTestDefaults())
sage: type(FDS)
<class 'sage.doctest.sources.PythonFileSource'>
```

```
>>> from sage.all import *
>>> from sage.doctest.control import DocTestDefaults
>>> from sage.doctest.sources import FileDocTestSource
>>> filename = sage.doctest.sources.__file__
>>> FDS = FileDocTestSource(filename, DocTestDefaults())
>>> type(FDS)
<class 'sage.doctest.sources.PythonFileSource'>
```

ending_docstring(*line*)

Determines whether the input line ends a docstring.

INPUT:

- `line` – string, one line of an input file

OUTPUT: an object that, when evaluated in a boolean context, gives `True` or `False` depending on whether the input line marks the end of a docstring

EXAMPLES:

```
sage: from sage.doctest.control import DocTestDefaults
sage: from sage.doctest.sources import FileDocTestSource
sage: from sage.doctest.util import NestedName
sage: filename = sage.doctest.sources.__file__
sage: FDS = FileDocTestSource(filename, DocTestDefaults())
sage: FDS._init()
sage: FDS.quotetype = "'''"
sage: FDS.ending_docstring("'''")
<...Match object...>
sage: FDS.ending_docstring('\"\"\"')
```

```
>>> from sage.all import *
>>> from sage.doctest.control import DocTestDefaults
>>> from sage.doctest.sources import FileDocTestSource
>>> from sage.doctest.util import NestedName
```

(continues on next page)

(continued from previous page)

```
>>> filename = sage.doctest.sources.__file__
>>> FDS = FileDocTestSource(filename, DocTestDefaults())
>>> FDS._init()
>>> FDS.quotetype = "'''"
>>> FDS.ending_docstring("'''")
<...Match object...>
>>> FDS.ending_docstring('\"\"\"')
```

start_finish_can_overlap = False

starting_docstring(*line*)

Determines whether the input line starts a docstring.

If the input line does start a docstring (a triple quote), then this function updates `self.qualified_name`.

INPUT:

- `line` – string; one line of an input file

OUTPUT: either `None` or a Match object

EXAMPLES:

```
sage: from sage.doctest.control import DocTestDefaults
sage: from sage.doctest.sources import FileDocTestSource
sage: from sage.doctest.util import NestedName
sage: filename = sage.doctest.sources.__file__
sage: FDS = FileDocTestSource(filename, DocTestDefaults())
sage: FDS._init()
sage: FDS.starting_docstring("r'''")
<...Match object...>
sage: FDS.ending_docstring("'''")
<...Match object...>
sage: FDS.qualified_name = NestedName(FDS.basename)
sage: FDS.starting_docstring("class MyClass():")
sage: FDS.starting_docstring("    def hello_world(self):")
sage: FDS.starting_docstring("        '''")
<...Match object...>
sage: FDS.qualified_name
sage.doctest.sources.MyClass.hello_world
sage: FDS.ending_docstring("    '''")
<...Match object...>
sage: FDS.starting_docstring("class NewClass():")
sage: FDS.starting_docstring("    '''")
<...Match object...>
sage: FDS.ending_docstring("    '''")
<...Match object...>
sage: FDS.qualified_name
sage.doctest.sources.NewClass
sage: FDS.starting_docstring("print(")
sage: FDS.starting_docstring("    '''Not a docstring")
sage: FDS.starting_docstring("    ''')")
sage: FDS.starting_docstring("def foo():")
sage: FDS.starting_docstring("    '''This is a docstring'''")
<...Match object...>
```

```
>>> from sage.all import *
>>> from sage.doctest.control import DocTestDefaults
>>> from sage.doctest.sources import FileDocTestSource
>>> from sage.doctest.util import NestedName
>>> filename = sage.doctest.sources.__file__
>>> FDS = FileDocTestSource(filename, DocTestDefaults())
>>> FDS._init()
>>> FDS.starting_docstring("r'''")
<...Match object...>
>>> FDS.ending_docstring("'''")
<...Match object...>
>>> FDS.qualified_name = NestedName(FDS.basename)
>>> FDS.starting_docstring("class MyClass():")
>>> FDS.starting_docstring("    def hello_world(self):")
>>> FDS.starting_docstring("        '''")
<...Match object...>
>>> FDS.qualified_name
sage.doctest.sources.MyClass.hello_world
>>> FDS.ending_docstring("    '''")
<...Match object...>
>>> FDS.starting_docstring("class NewClass():")
>>> FDS.starting_docstring("    '''")
<...Match object...>
>>> FDS.ending_docstring("    '''")
<...Match object...>
>>> FDS.qualified_name
sage.doctest.sources.NewClass
>>> FDS.starting_docstring("print(")
>>> FDS.starting_docstring("    '''Not a docstring")
>>> FDS.starting_docstring("    ''')")
>>> FDS.starting_docstring("def foo():")
>>> FDS.starting_docstring("    '''This is a docstring'''")
<...Match object...>
```

class sage.doctest.sources.**RestSource**

Bases: *SourceLanguage*

This class defines the functions needed for the extraction of doctests from ReST sources.

EXAMPLES:

```
sage: from sage.doctest.control import DocTestDefaults
sage: from sage.doctest.sources import FileDocTestSource
sage: filename = "sage_doc.rst"
sage: FDS = FileDocTestSource(filename, DocTestDefaults())
sage: type(FDS)
<class 'sage.doctest.sources.RestFileSource'>
```

```
>>> from sage.all import *
>>> from sage.doctest.control import DocTestDefaults
>>> from sage.doctest.sources import FileDocTestSource
>>> filename = "sage_doc.rst"
>>> FDS = FileDocTestSource(filename, DocTestDefaults())
```

(continues on next page)

(continued from previous page)

```
>>> type(FDS)
<class 'sage.doctest.sources.RestFileSource'>
```

ending_docstring(*line*)

When the indentation level drops below the initial level the block ends.

INPUT:

- `line` – string; one line of an input file

OUTPUT: boolean; whether the verbatim block is ending

EXAMPLES:

```
sage: from sage.doctest.control import DocTestDefaults
sage: from sage.doctest.sources import FileDocTestSource
sage: filename = "sage_doc.rst"
sage: FDS = FileDocTestSource(filename, DocTestDefaults())
sage: FDS._init()
sage: FDS.starting_docstring("Hello world::")
True
sage: FDS.ending_docstring("    sage: 2 + 2")
False
sage: FDS.ending_docstring("    4")
False
sage: FDS.ending_docstring("We are now done")
True
```

```
>>> from sage.all import *
>>> from sage.doctest.control import DocTestDefaults
>>> from sage.doctest.sources import FileDocTestSource
>>> filename = "sage_doc.rst"
>>> FDS = FileDocTestSource(filename, DocTestDefaults())
>>> FDS._init()
>>> FDS.starting_docstring("Hello world::")
True
>>> FDS.ending_docstring("    sage: 2 + 2")
False
>>> FDS.ending_docstring("    4")
False
>>> FDS.ending_docstring("We are now done")
True
```

parse_docstring(*docstring*, *namespace*, *start*)

Return a list of doctest defined in this docstring.

Code blocks in a REST file can contain python functions with their own docstrings in addition to in-line doctests. We want to include the tests from these inner docstrings, but Python's doctesting module has a problem if we just pass on the whole block, since it expects to get just a docstring, not the Python code as well.

Our solution is to create a new doctest source from this code block and append the doctests created from that source. We then replace the occurrences of "sage:" and ">>>" occurring inside a triple quote with "safe:" so that the doctest module doesn't treat them as tests.

EXAMPLES:

```
sage: from sage.doctest.control import DocTestDefaults
sage: from sage.doctest.sources import FileDocTestSource
sage: from sage.doctest.parsing import SageDocTestParser
sage: from sage.doctest.util import NestedName
sage: filename = "sage_doc.rst"
sage: FDS = FileDocTestSource(filename, DocTestDefaults())
sage: FDS.parser = SageDocTestParser(set(['sage']))
sage: FDS.qualified_name = NestedName('sage_doc')
sage: s = "Some text::\n\n    def example_python_function(a, \
....:      b):\n        '''\n        Brief description \
....:      of function.\n\n        EXAMPLES::\n\n            \
....:      sage: test1()\n            sage: test2()\n        \
....:      '''\n        return a + b\n\n    sage: test3()\n\nMore \
....:      ReST documentation."
sage: tests = FDS.parse_docstring(s, {}, 100)
sage: len(tests)
2
sage: for ex in tests[0].examples:
....:     print(ex.sage_source)
test3()
sage: for ex in tests[1].examples:
....:     print(ex.sage_source)
test1()
test2()
sig_on_count() # check sig_on/off pairings (virtual doctest)
```

```
>>> from sage.all import *
>>> from sage.doctest.control import DocTestDefaults
>>> from sage.doctest.sources import FileDocTestSource
>>> from sage.doctest.parsing import SageDocTestParser
>>> from sage.doctest.util import NestedName
>>> filename = "sage_doc.rst"
>>> FDS = FileDocTestSource(filename, DocTestDefaults())
>>> FDS.parser = SageDocTestParser(set(['sage']))
>>> FDS.qualified_name = NestedName('sage_doc')
>>> s = "Some text::\n\n    def example_python_function(a, \
...      b):\n        '''\n        Brief description \
...      of function.\n\n        EXAMPLES::\n\n            \
...      sage: test1()\n            sage: test2()\n        \
...      '''\n        return a + b\n\n    sage: test3()\n\nMore \
...      ReST documentation."
>>> tests = FDS.parse_docstring(s, {}, Integer(100))
>>> len(tests)
2
>>> for ex in tests[Integer(0)].examples:
...     print(ex.sage_source)
test3()
>>> for ex in tests[Integer(1)].examples:
...     print(ex.sage_source)
test1()
test2()
sig_on_count() # check sig_on/off pairings (virtual doctest)
```

start_finish_can_overlap = True

starting_docstring(*line*)

A line ending with a double colon starts a verbatim block in a ReST file, as does a line containing `.. CODE-BLOCK:: language`.

This function also determines whether the docstring block should be joined with the previous one, or should be skipped.

INPUT:

- `line` – string; one line of an input file

OUTPUT: either `None` or a Match object

EXAMPLES:

```
sage: from sage.doctest.control import DocTestDefaults
sage: from sage.doctest.sources import FileDocTestSource
sage: filename = "sage_doc.rst"
sage: FDS = FileDocTestSource(filename, DocTestDefaults())
sage: FDS._init()
sage: FDS.starting_docstring("Hello world::")
True
sage: FDS.ending_docstring("    sage: 2 + 2")
False
sage: FDS.ending_docstring("    4")
False
sage: FDS.ending_docstring("We are now done")
True
sage: FDS.starting_docstring(".. link")
sage: FDS.starting_docstring("::")
True
sage: FDS.linking
True
```

```
>>> from sage.all import *
>>> from sage.doctest.control import DocTestDefaults
>>> from sage.doctest.sources import FileDocTestSource
>>> filename = "sage_doc.rst"
>>> FDS = FileDocTestSource(filename, DocTestDefaults())
>>> FDS._init()
>>> FDS.starting_docstring("Hello world::")
True
>>> FDS.ending_docstring("    sage: 2 + 2")
False
>>> FDS.ending_docstring("    4")
False
>>> FDS.ending_docstring("We are now done")
True
>>> FDS.starting_docstring(".. link")
>>> FDS.starting_docstring("::")
True
>>> FDS.linking
True
```

class sage.doctest.sources.**SourceLanguage**

Bases: object

An abstract class for functions that depend on the programming language of a doctest source.

Currently supported languages include Python, ReST and LaTeX.

parse_docstring(*docstring*, *namespace*, *start*)

Return a list of doctest defined in this docstring.

This function is called by *_process_doc()*. The default implementation, defined here, is to use the *sage.doctest.parsing.SageDocTestParser* attached to this source to get doctests from the docstring.

INPUT:

- docstring – string containing documentation and tests
- namespace – dictionary or *sage.doctest.util.RecordingDict*
- start – integer; one less than the starting line number

EXAMPLES:

```
sage: # long time
sage: from sage.doctest.control import DocTestDefaults
sage: from sage.doctest.sources import FileDocTestSource
sage: from sage.doctest.parsing import SageDocTestParser
sage: from sage.doctest.util import NestedName
sage: filename = sage.doctest.util.__file__
sage: FDS = FileDocTestSource(filename, DocTestDefaults())
sage: doctests, _ = FDS.create_doctests({})
sage: for dt in doctests:
....:     FDS.qualified_name = dt.name
....:     dt.examples = dt.examples[:-1] # strip off the sig_on() test
....:     assert(FDS.parse_docstring(dt.docstring,{},dt.lineno-1)[0] == dt)
```

```
>>> from sage.all import *
>>> # long time
>>> from sage.doctest.control import DocTestDefaults
>>> from sage.doctest.sources import FileDocTestSource
>>> from sage.doctest.parsing import SageDocTestParser
>>> from sage.doctest.util import NestedName
>>> filename = sage.doctest.util.__file__
>>> FDS = FileDocTestSource(filename, DocTestDefaults())
>>> doctests, _ = FDS.create_doctests({})
>>> for dt in doctests:
...     FDS.qualified_name = dt.name
...     dt.examples = dt.examples[:-Integer(1)] # strip off the sig_on() test
...     assert(FDS.parse_docstring(dt.docstring,{},dt.lineno-
↪Integer(1))[Integer(0)] == dt)
```

class sage.doctest.sources.**StringDocTestSource**(*basename*, *source*, *options*, *printpath*, *lineno_shift=0*)

Bases: *DocTestSource*

This class creates doctests from a string.

INPUT:

- basename – string such as 'sage.doctests.sources', going into the names of created doctests and examples

- `source` – string, giving the source code to be parsed for doctests
- `options` – a *sage.doctest.control.DocTestDefaults* or equivalent
- `printpath` – string, to be used in place of a filename when doctest failures are displayed
- `lineno_shift` – integer (default: 0) by which to shift the line numbers of all doctests defined in this string

EXAMPLES:

```
sage: from sage.doctest.control import DocTestDefaults
sage: from sage.doctest.sources import StringDocTestSource, PythonSource
sage: from sage.structure.dynamic_class import dynamic_class
sage: s = "'''\n    sage: 2 + 2\n    4\n'''"
sage: PythonStringSource = dynamic_class('PythonStringSource',
↪(StringDocTestSource, PythonSource))
sage: PSS = PythonStringSource('<runtime>', s, DocTestDefaults(), 'runtime')
sage: dt, extras = PSS.create_doctests({})
sage: len(dt)
1
sage: extras['tab']
[]
sage: extras['line_number']
False

sage: s = "'''\n\tsage: 2 + 2\n\t4\n'''"
sage: PSS = PythonStringSource('<runtime>', s, DocTestDefaults(), 'runtime')
sage: dt, extras = PSS.create_doctests({})
sage: extras['tab']
['2', '3']

sage: s = "'''\n    sage: import warnings; warnings.warn('foo')\n    doctest:1:␣
↪UserWarning: foo \n'''"
sage: PSS = PythonStringSource('<runtime>', s, DocTestDefaults(), 'runtime')
sage: dt, extras = PSS.create_doctests({})
sage: extras['line_number']
True
```

```
>>> from sage.all import *
>>> from sage.doctest.control import DocTestDefaults
>>> from sage.doctest.sources import StringDocTestSource, PythonSource
>>> from sage.structure.dynamic_class import dynamic_class
>>> s = "'''\n    sage: 2 + 2\n    4\n'''"
>>> PythonStringSource = dynamic_class('PythonStringSource',(StringDocTestSource,␣
↪PythonSource))
>>> PSS = PythonStringSource('<runtime>', s, DocTestDefaults(), 'runtime')
>>> dt, extras = PSS.create_doctests({})
>>> len(dt)
1
>>> extras['tab']
[]
>>> extras['line_number']
False

>>> s = "'''\n\tsage: 2 + 2\n\t4\n'''"
```

(continues on next page)

(continued from previous page)

```
>>> PSS = PythonStringSource('<runtime>', s, DocTestDefaults(), 'runtime')
>>> dt, extras = PSS.create_doctests({})
>>> extras['tab']
['2', '3']

>>> s = "'''\n    sage: import warnings; warnings.warn('foo')\n    doctest:1:␣
↪UserWarning: foo \n'''"
>>> PSS = PythonStringSource('<runtime>', s, DocTestDefaults(), 'runtime')
>>> dt, extras = PSS.create_doctests({})
>>> extras['line_number']
True
```

create_doctests(*namespace*)

Create doctests from this string.

INPUT:

- `namespace` – dictionary or *sage.doctest.util.RecordingDict*

OUTPUT:

- `doctests` – list of doctests defined by this string
- `extras` – dictionary with `extras['tab']` either `False` or a list of linenumbers on which tabs appear

EXAMPLES:

```
sage: from sage.doctest.control import DocTestDefaults
sage: from sage.doctest.sources import StringDocTestSource, PythonSource
sage: from sage.structure.dynamic_class import dynamic_class
sage: s = "'''\n    sage: 2 + 2\n    4\n'''"
sage: PythonStringSource = dynamic_class('PythonStringSource',
↪(StringDocTestSource, PythonSource))
sage: PSS = PythonStringSource('<runtime>', s, DocTestDefaults(), 'runtime')
sage: dt, extras = PSS.create_doctests({})
sage: for t in dt:
....:     print("{} {}".format(t.name, t.examples[0].sage_source))
<runtime> 2 + 2
sage: extras
{...'tab': []...}
```

```
>>> from sage.all import *
>>> from sage.doctest.control import DocTestDefaults
>>> from sage.doctest.sources import StringDocTestSource, PythonSource
>>> from sage.structure.dynamic_class import dynamic_class
>>> s = "'''\n    sage: 2 + 2\n    4\n'''"
>>> PythonStringSource = dynamic_class('PythonStringSource',
↪(StringDocTestSource, PythonSource))
>>> PSS = PythonStringSource('<runtime>', s, DocTestDefaults(), 'runtime')
>>> dt, extras = PSS.create_doctests({})
>>> for t in dt:
...     print("{} {}".format(t.name, t.examples[Integer(0)].sage_source))
<runtime> 2 + 2
>>> extras
{...'tab': []...}
```

class sage.doctest.sources.**TexSource**

Bases: *SourceLanguage*

This class defines the functions needed for the extraction of doctests from a LaTeX source.

EXAMPLES:

```
sage: from sage.doctest.control import DocTestDefaults
sage: from sage.doctest.sources import FileDocTestSource
sage: filename = "sage_paper.tex"
sage: FDS = FileDocTestSource(filename, DocTestDefaults())
sage: type(FDS)
<class 'sage.doctest.sources.TexFileSource'>
```

```
>>> from sage.all import *
>>> from sage.doctest.control import DocTestDefaults
>>> from sage.doctest.sources import FileDocTestSource
>>> filename = "sage_paper.tex"
>>> FDS = FileDocTestSource(filename, DocTestDefaults())
>>> type(FDS)
<class 'sage.doctest.sources.TexFileSource'>
```

ending_docstring(*line*, *check_skip=True*)

Determines whether the input line ends a docstring.

Docstring blocks in tex files are defined by verbatim or lstlisting environments, and can be linked together by adding %link immediately after the end{verbatim} or end{lstlisting}.

Within a verbatim (or lstlisting) block, you can tell Sage not to process the rest of the block by including a %skip line.

INPUT:

- `line` – string, one line of an input file
- `check_skip` – boolean (default: `True`); used internally in `starting_docstring`

OUTPUT: boolean; whether the input line marks the end of a docstring (verbatim block)

EXAMPLES:

```
sage: from sage.doctest.control import DocTestDefaults
sage: from sage.doctest.sources import FileDocTestSource
sage: filename = "sage_paper.tex"
sage: FDS = FileDocTestSource(filename, DocTestDefaults())
sage: FDS._init()
sage: FDS.ending_docstring(r"\end{verbatim}")
True
sage: FDS.ending_docstring(r"\end{lstlisting}")
True
sage: FDS.linking
False
```

```
>>> from sage.all import *
>>> from sage.doctest.control import DocTestDefaults
>>> from sage.doctest.sources import FileDocTestSource
>>> filename = "sage_paper.tex"
```

(continues on next page)

(continued from previous page)

```
>>> FDS = FileDocTestSource(filename, DocTestDefaults())
>>> FDS._init()
>>> FDS.ending_docstring(r"\end{verbatim}")
True
>>> FDS.ending_docstring(r"\end{lstlisting}")
True
>>> FDS.linking
False
```

Use %link to link with the next verbatim block:

```
sage: FDS.ending_docstring(r"\end{verbatim}%link")
True
sage: FDS.linking
True
```

```
>>> from sage.all import *
>>> FDS.ending_docstring(r"\end{verbatim}%link")
True
>>> FDS.linking
True
```

%skip also ends a docstring block:

```
sage: FDS.ending_docstring("%skip")
True
```

```
>>> from sage.all import *
>>> FDS.ending_docstring("%skip")
True
```

start_finish_can_overlap = False

starting_docstring(*line*)

Determines whether the input line starts a docstring.

Docstring blocks in tex files are defined by verbatim or lstlisting environments, and can be linked together by adding %link immediately after the end{verbatim} or end{lstlisting}.

Within a verbatim (or lstlisting) block, you can tell Sage not to process the rest of the block by including a %skip line.

INPUT:

- `line` – string, one line of an input file

OUTPUT: boolean; whether the input line marks the start of a docstring (verbatim block)

EXAMPLES:

```
sage: from sage.doctest.control import DocTestDefaults
sage: from sage.doctest.sources import FileDocTestSource
sage: filename = "sage_paper.tex"
sage: FDS = FileDocTestSource(filename, DocTestDefaults())
sage: FDS._init()
```

```
>>> from sage.all import *
>>> from sage.doctest.control import DocTestDefaults
>>> from sage.doctest.sources import FileDocTestSource
>>> filename = "sage_paper.tex"
>>> FDS = FileDocTestSource(filename, DocTestDefaults())
>>> FDS._init()
```

We start docstrings with begin{verbatim} or begin{lstlisting}:

```
sage: FDS.starting_docstring(r"\begin{verbatim}")
True
sage: FDS.starting_docstring(r"\begin{lstlisting}")
True
sage: FDS.skipping
False
sage: FDS.ending_docstring("sage: 2+2")
False
sage: FDS.ending_docstring("4")
False
```

```
>>> from sage.all import *
>>> FDS.starting_docstring(r"\begin{verbatim}")
True
>>> FDS.starting_docstring(r"\begin{lstlisting}")
True
>>> FDS.skipping
False
>>> FDS.ending_docstring("sage: 2+2")
False
>>> FDS.ending_docstring("4")
False
```

To start ignoring the rest of the verbatim block, use %skip:

```
sage: FDS.ending_docstring("%skip")
True
sage: FDS.skipping
True
sage: FDS.starting_docstring("sage: raise RuntimeError")
False
```

```
>>> from sage.all import *
>>> FDS.ending_docstring("%skip")
True
>>> FDS.skipping
True
>>> FDS.starting_docstring("sage: raise RuntimeError")
False
```

You can even pretend to start another verbatim block while skipping:

```
sage: FDS.starting_docstring(r"\begin{verbatim}")
False
```

(continues on next page)

(continued from previous page)

```
sage: FDS.skipping
True
```

```
>>> from sage.all import *
>>> FDS.starting_docstring(r"\begin{verbatim}")
False
>>> FDS.skipping
True
```

To stop skipping end the verbatim block:

```
sage: FDS.starting_docstring(r"\end{verbatim} %link")
False
sage: FDS.skipping
False
```

```
>>> from sage.all import *
>>> FDS.starting_docstring(r"\end{verbatim} %link")
False
>>> FDS.skipping
False
```

Linking works even when the block was ended while skipping:

```
sage: FDS.linking
True
sage: FDS.starting_docstring(r"\begin{verbatim}")
True
```

```
>>> from sage.all import *
>>> FDS.linking
True
>>> FDS.starting_docstring(r"\begin{verbatim}")
True
```

sage.doctest.sources.**get_basename**(*path*)

This function returns the basename of the given path, e.g. `sage.doctest.sources` or `doc.ru.tutorial.tour_advanced`.

EXAMPLES:

```
sage: from sage.doctest.sources import get_basename
sage: import os
sage: get_basename(sage.doctest.sources.__file__)
'sage.doctest.sources'
```

```
>>> from sage.all import *
>>> from sage.doctest.sources import get_basename
>>> import os
>>> get_basename(sage.doctest.sources.__file__)
'sage.doctest.sources'
```

```
sage: # optional - !meson_editable
sage: get_basename(os.path.join(sage.structure.__path__[0], 'element.pxd'))
'sage.structure.element.pxd'
```

```
>>> from sage.all import *
>>> # optional - !meson_editable
>>> get_basename(os.path.join(sage.structure.__path__[Integer(0)], 'element.pxd'))
'sage.structure.element.pxd'
```

CHAPTER

THREE

PROCESSES FOR RUNNING DOCTESTS

This module controls the processes started by Sage that actually run the doctests.

EXAMPLES:

The following examples are used in doctesting this file:

```
sage: doctest_var = 42; doctest_var^2
1764
sage: R.<a> = ZZ[]
sage: a + doctest_var
a + 42
```

```
>>> from sage.all import *
>>> doctest_var = Integer(42); doctest_var**Integer(2)
1764
>>> R = ZZ['a']; (a,) = R._first_ngens(1)
>>> a + doctest_var
a + 42
```

AUTHORS:

- David Roe (2012-03-27) – initial version, based on Robert Bradshaw's code.
- Jeroen Demeyer (2013 and 2015) – major improvements to forking and logging

class sage.doctest.forker.**DocTestDispatcher**(*controller:* sage.doctest.control.DocTestController)

Bases: SageObject

Create parallel *DocTestWorker* processes and dispatches doctesting tasks.

dispatch()

Run the doctests for the controller's specified sources, by calling *parallel_dispatch()* or *serial_dispatch()* according to the `--serial` option.

EXAMPLES:

```
sage: from sage.doctest.control import DocTestController, DocTestDefaults
sage: from sage.doctest.forker import DocTestDispatcher
sage: from sage.doctest.reporting import DocTestReporter
sage: from sage.doctest.util import Timer
sage: import os
sage: freehom = os.path.join(SAGE_SRC, 'sage', 'modules', 'free_module_
↪homspace.py')
sage: bigo = os.path.join(SAGE_SRC, 'sage', 'rings', 'big_oh.py')
```

(continues on next page)

(continued from previous page)

```
sage: DC = DocTestController(DocTestDefaults(), [freehom, bigo])
sage: DC.expand_files_into_sources()
sage: DD = DocTestDispatcher(DC)
sage: DR = DocTestReporter(DC)
sage: DC.reporter = DR
sage: DC.dispatcher = DD
sage: DC.timer = Timer().start()
sage: DD.dispatch()
.../sage/modules/free_module_homspace.py
    [... tests, ...s wall]
.../sage/rings/big_oh.py
    [... tests, ...s wall]
```

```
>>> from sage.all import *
>>> from sage.doctest.control import DocTestController, DocTestDefaults
>>> from sage.doctest.forker import DocTestDispatcher
>>> from sage.doctest.reporting import DocTestReporter
>>> from sage.doctest.util import Timer
>>> import os
>>> freehom = os.path.join(SAGE_SRC, 'sage', 'modules', 'free_module_homspace.
↪py')
>>> bigo = os.path.join(SAGE_SRC, 'sage', 'rings', 'big_oh.py')
>>> DC = DocTestController(DocTestDefaults(), [freehom, bigo])
>>> DC.expand_files_into_sources()
>>> DD = DocTestDispatcher(DC)
>>> DR = DocTestReporter(DC)
>>> DC.reporter = DR
>>> DC.dispatcher = DD
>>> DC.timer = Timer().start()
>>> DD.dispatch()
.../sage/modules/free_module_homspace.py
    [... tests, ...s wall]
.../sage/rings/big_oh.py
    [... tests, ...s wall]
```

parallel_dispatch()

Run the doctests from the controller's specified sources in parallel.

This creates *DocTestWorker* subprocesses, while the master process checks for timeouts and collects and displays the results.

EXAMPLES:

```
sage: from sage.doctest.control import DocTestController, DocTestDefaults
sage: from sage.doctest.forker import DocTestDispatcher
sage: from sage.doctest.reporting import DocTestReporter
sage: from sage.doctest.util import Timer
sage: import os
sage: crem = os.path.join(SAGE_SRC, 'sage', 'databases', 'cremona.py')
sage: bigo = os.path.join(SAGE_SRC, 'sage', 'rings', 'big_oh.py')
sage: DC = DocTestController(DocTestDefaults(), [crem, bigo])
sage: DC.expand_files_into_sources()
sage: DD = DocTestDispatcher(DC)
```

(continues on next page)

(continued from previous page)

```
sage: DR = DocTestReporter(DC)
sage: DC.reporter = DR
sage: DC.dispatcher = DD
sage: DC.timer = Timer().start()
sage: DD.parallel_dispatch()
.../databases/cremona.py
    [... tests, ...s wall]
.../rings/big_oh.py
    [... tests, ...s wall]
```

```
>>> from sage.all import *
>>> from sage.doctest.control import DocTestController, DocTestDefaults
>>> from sage.doctest.forker import DocTestDispatcher
>>> from sage.doctest.reporting import DocTestReporter
>>> from sage.doctest.util import Timer
>>> import os
>>> crem = os.path.join(SAGE_SRC, 'sage', 'databases', 'cremona.py')
>>> bigo = os.path.join(SAGE_SRC, 'sage', 'rings', 'big_oh.py')
>>> DC = DocTestController(DocTestDefaults(), [crem, bigo])
>>> DC.expand_files_into_sources()
>>> DD = DocTestDispatcher(DC)
>>> DR = DocTestReporter(DC)
>>> DC.reporter = DR
>>> DC.dispatcher = DD
>>> DC.timer = Timer().start()
>>> DD.parallel_dispatch()
.../databases/cremona.py
    [... tests, ...s wall]
.../rings/big_oh.py
    [... tests, ...s wall]
```

If the `exitfirst=True` option is given, the results for a failing module will be immediately printed and any other ongoing tests canceled:

```
sage: from tempfile import NamedTemporaryFile as NTF
sage: with (NTF(suffix='.py', mode='w+t') as f1,
....:       NTF(suffix='.py', mode='w+t') as f2):
....:     _ = f1.write("'''\nsage: import time; time.sleep(60)\n'''")
....:     f1.flush()
....:     _ = f2.write("'''\nsage: True\nFalse\n'''")
....:     f2.flush()
....:     DC = DocTestController(DocTestDefaults(exitfirst=True,
....:                                            nthreads=2),
....:                            [f1.name, f2.name])
....:     DC.expand_files_into_sources()
....:     DD = DocTestDispatcher(DC)
....:     DR = DocTestReporter(DC)
....:     DC.reporter = DR
....:     DC.dispatcher = DD
....:     DC.timer = Timer().start()
....:     DD.parallel_dispatch()
...
```

(continues on next page)

(continued from previous page)

```
**********************************************************************
File "...", line 2, in ...
Failed example:
    True
Expected:
    False
Got:
    True
**********************************************************************
1 item had failures:
   1 of   1 in ...
    [1 test, 1 failure, ...s wall]
Killing test ...
```

```
>>> from sage.all import *
>>> from tempfile import NamedTemporaryFile as NTF
>>> with (NTF(suffix='.py', mode='w+t') as f1,
...       NTF(suffix='.py', mode='w+t') as f2):
...     _ = f1.write("'''\nsage: import time; time.sleep(60)\n'''")
...     f1.flush()
...     _ = f2.write("'''\nsage: True\nFalse\n'''")
...     f2.flush()
...     DC = DocTestController(DocTestDefaults(exitfirst=True,
...                                            nthreads=Integer(2)),
...                            [f1.name, f2.name])
...     DC.expand_files_into_sources()
...     DD = DocTestDispatcher(DC)
...     DR = DocTestReporter(DC)
...     DC.reporter = DR
...     DC.dispatcher = DD
...     DC.timer = Timer().start()
...     DD.parallel_dispatch()
...
**********************************************************************
File "...", line 2, in ...
Failed example:
    True
Expected:
    False
Got:
    True
**********************************************************************
1 item had failures:
   1 of   1 in ...
    [1 test, 1 failure, ...s wall]
Killing test ...
```

serial_dispatch()

Run the doctests from the controller's specified sources in series.

There is no graceful handling for signals, no possibility of interrupting tests and no timeout.

EXAMPLES:

```
sage: from sage.doctest.control import DocTestController, DocTestDefaults
sage: from sage.doctest.forker import DocTestDispatcher
sage: from sage.doctest.reporting import DocTestReporter
sage: from sage.doctest.util import Timer
sage: import os
sage: homset = os.path.join(SAGE_SRC, 'sage', 'rings', 'homset.py')
sage: ideal = os.path.join(SAGE_SRC, 'sage', 'rings', 'ideal.py')
sage: DC = DocTestController(DocTestDefaults(), [homset, ideal])
sage: DC.expand_files_into_sources()
sage: DD = DocTestDispatcher(DC)
sage: DR = DocTestReporter(DC)
sage: DC.reporter = DR
sage: DC.dispatcher = DD
sage: DC.timer = Timer().start()
sage: DD.serial_dispatch()
.../rings/homset.py
    [... tests, ...s wall]
.../rings/ideal.py
    [... tests, ...s wall]
```

```
>>> from sage.all import *
>>> from sage.doctest.control import DocTestController, DocTestDefaults
>>> from sage.doctest.forker import DocTestDispatcher
>>> from sage.doctest.reporting import DocTestReporter
>>> from sage.doctest.util import Timer
>>> import os
>>> homset = os.path.join(SAGE_SRC, 'sage', 'rings', 'homset.py')
>>> ideal = os.path.join(SAGE_SRC, 'sage', 'rings', 'ideal.py')
>>> DC = DocTestController(DocTestDefaults(), [homset, ideal])
>>> DC.expand_files_into_sources()
>>> DD = DocTestDispatcher(DC)
>>> DR = DocTestReporter(DC)
>>> DC.reporter = DR
>>> DC.dispatcher = DD
>>> DC.timer = Timer().start()
>>> DD.serial_dispatch()
.../rings/homset.py
    [... tests, ...s wall]
.../rings/ideal.py
    [... tests, ...s wall]
```

class sage.doctest.forker.**DocTestTask**(*source*)

Bases: object

This class encapsulates the tests from a single source.

This class does not insulate from problems in the source (e.g. entering an infinite loop or causing a segfault), that has to be dealt with at a higher level.

INPUT:

- source – a *sage.doctest.sources.DocTestSource* instance
- verbose – boolean, controls reporting of progress by doctest.DocTestRunner

EXAMPLES:

```
sage: from sage.doctest.forker import DocTestTask
sage: from sage.doctest.sources import FileDocTestSource
sage: from sage.doctest.control import DocTestDefaults, DocTestController
sage: import os
sage: filename = sage.doctest.sources.__file__
sage: DD = DocTestDefaults()
sage: FDS = FileDocTestSource(filename, DD)
sage: DTT = DocTestTask(FDS)
sage: DC = DocTestController(DD,[filename])
sage: ntests, results = DTT(options=DD)
sage: ntests >= 300 or ntests
True
sage: sorted(results.keys())
['cputime', 'err', 'failures', 'optionals', 'tests', 'walltime', 'walltime_skips']
```

```
>>> from sage.all import *
>>> from sage.doctest.forker import DocTestTask
>>> from sage.doctest.sources import FileDocTestSource
>>> from sage.doctest.control import DocTestDefaults, DocTestController
>>> import os
>>> filename = sage.doctest.sources.__file__
>>> DD = DocTestDefaults()
>>> FDS = FileDocTestSource(filename, DD)
>>> DTT = DocTestTask(FDS)
>>> DC = DocTestController(DD,[filename])
>>> ntests, results = DTT(options=DD)
>>> ntests >= Integer(300) or ntests
True
>>> sorted(results.keys())
['cputime', 'err', 'failures', 'optionals', 'tests', 'walltime', 'walltime_skips']
```

class sage.doctest.forker.**DocTestWorker**(*source*, *options*, *funclist=[]*, *baseline=None*)

Bases: Process

The DocTestWorker process runs one *DocTestTask* for a given source. It returns messages about doctest failures (or all tests if verbose doctesting) through a pipe and returns results through a multiprocessing.Queue instance (both these are created in the *start()* method).

It runs the task in its own process-group, such that killing the process group kills this process together with its child processes.

The class has additional methods and attributes for bookkeeping by the master process. Except in *run()*, nothing from this class should be accessed by the child process.

INPUT:

- source – a *DocTestSource* instance
- options – an object representing doctest options
- funclist – list of callables to be called at the start of the child process
- baseline – dictionary, the baseline_stats value

EXAMPLES:

```
sage: # long time
sage: from sage.doctest.forker import DocTestWorker, DocTestTask
sage: from sage.doctest.sources import FileDocTestSource
sage: from sage.doctest.reporting import DocTestReporter
sage: from sage.doctest.control import DocTestController, DocTestDefaults
sage: filename = sage.doctest.util.__file__
sage: DD = DocTestDefaults()
sage: FDS = FileDocTestSource(filename, DD)
sage: W = DocTestWorker(FDS, DD)
sage: W.start()
sage: DC = DocTestController(DD, filename)
sage: reporter = DocTestReporter(DC)
sage: W.join()  # Wait for worker to finish
sage: result = W.result_queue.get()
sage: reporter.report(FDS, False, W.exitcode, result, "")
    [... tests, ...s wall]
```

```
>>> from sage.all import *
>>> # long time
>>> from sage.doctest.forker import DocTestWorker, DocTestTask
>>> from sage.doctest.sources import FileDocTestSource
>>> from sage.doctest.reporting import DocTestReporter
>>> from sage.doctest.control import DocTestController, DocTestDefaults
>>> filename = sage.doctest.util.__file__
>>> DD = DocTestDefaults()
>>> FDS = FileDocTestSource(filename, DD)
>>> W = DocTestWorker(FDS, DD)
>>> W.start()
>>> DC = DocTestController(DD, filename)
>>> reporter = DocTestReporter(DC)
>>> W.join()  # Wait for worker to finish
>>> result = W.result_queue.get()
>>> reporter.report(FDS, False, W.exitcode, result, "")
    [... tests, ...s wall]
```

kill()

Kill this worker. Return `True` if the signal(s) are sent successfully or `False` if the worker process no longer exists.

This method is only called if there is something wrong with the worker. Under normal circumstances, the worker is supposed to exit by itself after finishing.

The first time this is called, use `SIGQUIT`. This will trigger the cysignals `SIGQUIT` handler and try to print an enhanced traceback.

Subsequent times, use `SIGKILL`. Also close the message pipe if it was still open.

EXAMPLES:

```
sage: import time
sage: from sage.doctest.forker import DocTestWorker, DocTestTask
sage: from sage.doctest.sources import FileDocTestSource
sage: from sage.doctest.reporting import DocTestReporter
sage: from sage.doctest.control import DocTestController, DocTestDefaults
```

(continues on next page)

(continued from previous page)

```
sage: filename = os.path.join(SAGE_SRC,'sage','doctest','tests','99seconds.rst
↪')
sage: DD = DocTestDefaults()
sage: FDS = FileDocTestSource(filename, DD)
```

```
>>> from sage.all import *
>>> import time
>>> from sage.doctest.forker import DocTestWorker, DocTestTask
>>> from sage.doctest.sources import FileDocTestSource
>>> from sage.doctest.reporting import DocTestReporter
>>> from sage.doctest.control import DocTestController, DocTestDefaults
>>> filename = os.path.join(SAGE_SRC,'sage','doctest','tests','99seconds.rst')
>>> DD = DocTestDefaults()
>>> FDS = FileDocTestSource(filename, DD)
```

We set up the worker to start by blocking `SIGQUIT`, such that killing will fail initially:

```
sage: from cysignals.pselect import PSelecter
sage: import signal
sage: def block_hup():
....:     # We never __exit__()
....:     PSelecter([signal.SIGQUIT]).__enter__()
sage: W = DocTestWorker(FDS, DD, [block_hup])
sage: W.start()
sage: W.killed
False
sage: W.kill()
True
sage: W.killed
True
sage: time.sleep(float(0.2))  # Worker doesn't die
sage: W.kill()          # Worker dies now
True
sage: time.sleep(1)
sage: W.is_alive()
False
```

```
>>> from sage.all import *
>>> from cysignals.pselect import PSelecter
>>> import signal
>>> def block_hup():
...     # We never __exit__()
...     PSelecter([signal.SIGQUIT]).__enter__()
>>> W = DocTestWorker(FDS, DD, [block_hup])
>>> W.start()
>>> W.killed
False
>>> W.kill()
True
>>> W.killed
True
>>> time.sleep(float(RealNumber('0.2')))  # Worker doesn't die
```

(continues on next page)

(continued from previous page)

```
>>> W.kill()          # Worker dies now
True
>>> time.sleep(Integer(1))
>>> W.is_alive()
False
```

read_messages()

In the master process, read from the pipe and store the data read in the `messages` attribute.

> **Note**
>
> This function may need to be called multiple times in order to read all of the messages.

EXAMPLES:

```
sage: # long time
sage: from sage.doctest.forker import DocTestWorker, DocTestTask
sage: from sage.doctest.sources import FileDocTestSource
sage: from sage.doctest.reporting import DocTestReporter
sage: from sage.doctest.control import DocTestController, DocTestDefaults
sage: filename = sage.doctest.util.__file__
sage: DD = DocTestDefaults(verbose=True,nthreads=2)
sage: FDS = FileDocTestSource(filename, DD)
sage: W = DocTestWorker(FDS, DD)
sage: W.start()
sage: while W.rmessages is not None:
....:     W.read_messages()
sage: W.join()
sage: len(W.messages) > 0
True
```

```
>>> from sage.all import *
>>> # long time
>>> from sage.doctest.forker import DocTestWorker, DocTestTask
>>> from sage.doctest.sources import FileDocTestSource
>>> from sage.doctest.reporting import DocTestReporter
>>> from sage.doctest.control import DocTestController, DocTestDefaults
>>> filename = sage.doctest.util.__file__
>>> DD = DocTestDefaults(verbose=True,nthreads=Integer(2))
>>> FDS = FileDocTestSource(filename, DD)
>>> W = DocTestWorker(FDS, DD)
>>> W.start()
>>> while W.rmessages is not None:
...     W.read_messages()
>>> W.join()
>>> len(W.messages) > Integer(0)
True
```

run()

Run the *DocTestTask* under its own PGID.

save_result_output()

Annotate `self` with `self.result` (the result read through the `result_queue` and with `self.output`, the complete contents of `self.outtmpfile`. Then close the Queue and `self.outtmpfile`.

EXAMPLES:

```
sage: # long time
sage: from sage.doctest.forker import DocTestWorker, DocTestTask
sage: from sage.doctest.sources import FileDocTestSource
sage: from sage.doctest.reporting import DocTestReporter
sage: from sage.doctest.control import DocTestController, DocTestDefaults
sage: filename = sage.doctest.util.__file__
sage: DD = DocTestDefaults()
sage: FDS = FileDocTestSource(filename, DD)
sage: W = DocTestWorker(FDS, DD)
sage: W.start()
sage: W.join()
sage: W.save_result_output()
sage: sorted(W.result[1].keys())
['cputime', 'err', 'failures', 'optionals', 'tests', 'walltime', 'walltime_
↪skips']
sage: len(W.output) > 0
True
```

```
>>> from sage.all import *
>>> # long time
>>> from sage.doctest.forker import DocTestWorker, DocTestTask
>>> from sage.doctest.sources import FileDocTestSource
>>> from sage.doctest.reporting import DocTestReporter
>>> from sage.doctest.control import DocTestController, DocTestDefaults
>>> filename = sage.doctest.util.__file__
>>> DD = DocTestDefaults()
>>> FDS = FileDocTestSource(filename, DD)
>>> W = DocTestWorker(FDS, DD)
>>> W.start()
>>> W.join()
>>> W.save_result_output()
>>> sorted(W.result[Integer(1)].keys())
['cputime', 'err', 'failures', 'optionals', 'tests', 'walltime', 'walltime_
↪skips']
>>> len(W.output) > Integer(0)
True
```

Note

This method is called from the parent process, not from the subprocess.

start()

Start the worker and close the writing end of the message pipe.

class sage.doctest.forker.**SageDocTestRunner**(**args*, ***kwds*)

Bases: DocTestRunner

A customized version of DocTestRunner that tracks dependencies of doctests.

INPUT:

- `stdout` – an open file to restore for debugging
- `checker` – `None`, or an instance of `doctest.OutputChecker`
- `verbose` – boolean, determines whether verbose printing is enabled
- `optionflags` – controls the comparison with the expected output. See `doctest.testmod()` for more information
- `baseline` – dictionary, the `baseline_stats` value

EXAMPLES:

```
sage: from sage.doctest.parsing import SageOutputChecker
sage: from sage.doctest.forker import SageDocTestRunner
sage: from sage.doctest.control import DocTestDefaults; DD = DocTestDefaults()
sage: import doctest, sys, os
sage: DTR = SageDocTestRunner(SageOutputChecker(), verbose=False, sage_options=DD,
↪ optionflags=doctest.NORMALIZE_WHITESPACE|doctest.ELLIPSIS)
sage: DTR
<sage.doctest.forker.SageDocTestRunner object at ...>
```

```
>>> from sage.all import *
>>> from sage.doctest.parsing import SageOutputChecker
>>> from sage.doctest.forker import SageDocTestRunner
>>> from sage.doctest.control import DocTestDefaults; DD = DocTestDefaults()
>>> import doctest, sys, os
>>> DTR = SageDocTestRunner(SageOutputChecker(), verbose=False, sage_options=DD,␣
↪optionflags=doctest.NORMALIZE_WHITESPACE|doctest.ELLIPSIS)
>>> DTR
<sage.doctest.forker.SageDocTestRunner object at ...>
```

compile_and_execute(*example*, *compiler*, *globs*)

Run the given example, recording dependencies.

Rather than using a basic dictionary, Sage's doctest runner uses a *sage.doctest.util.RecordingDict*, which records every time a value is set or retrieved. Executing the given code with this recording dictionary as the namespace allows Sage to track dependencies between doctest lines. For example, in the following two lines

```
sage: R.<x> = ZZ[]
sage: f = x^2 + 1
```

```
>>> from sage.all import *
>>> R = ZZ['x']; (x,) = R._first_ngens(1)
>>> f = x**Integer(2) + Integer(1)
```

the recording dictionary records that the second line depends on the first since the first INSERTS `x` into the global namespace and the second line RETRIEVES `x` from the global namespace.

INPUT:

- `example` – a `doctest.Example` instance
- `compiler` – a callable that, applied to example, produces a code object

- `globs` – dictionary in which to execute the code

OUTPUT: the output of the compiled code snippet

EXAMPLES:

```
sage: from sage.doctest.parsing import SageOutputChecker
sage: from sage.doctest.forker import SageDocTestRunner
sage: from sage.doctest.sources import FileDocTestSource
sage: from sage.doctest.util import RecordingDict
sage: from sage.doctest.control import DocTestDefaults; DD = DocTestDefaults()
sage: import doctest, sys, os, hashlib
sage: DTR = SageDocTestRunner(SageOutputChecker(), verbose=False, sage_
↪options=DD,
....:            optionflags=doctest.NORMALIZE_WHITESPACE|doctest.ELLIPSIS)
sage: DTR.running_doctest_digest = hashlib.md5()
sage: filename = sage.doctest.forker.__file__
sage: FDS = FileDocTestSource(filename, DD)
sage: globs = RecordingDict(globals())
sage: 'doctest_var' in globs
False
sage: doctests, extras = FDS.create_doctests(globs)
sage: ex0 = doctests[0].examples[0]
sage: flags = 524288
sage: def compiler(ex):
....:     return compile(ex.source, '<doctest sage.doctest.forker[0]>',
....:                    'single', flags, 1)
sage: DTR.compile_and_execute(ex0, compiler, globs)
1764
sage: globs['doctest_var']
42
sage: globs.set
{'doctest_var'}
sage: globs.got
{'Integer'}
```

```
>>> from sage.all import *
>>> from sage.doctest.parsing import SageOutputChecker
>>> from sage.doctest.forker import SageDocTestRunner
>>> from sage.doctest.sources import FileDocTestSource
>>> from sage.doctest.util import RecordingDict
>>> from sage.doctest.control import DocTestDefaults; DD = DocTestDefaults()
>>> import doctest, sys, os, hashlib
>>> DTR = SageDocTestRunner(SageOutputChecker(), verbose=False, sage_
↪options=DD,
...            optionflags=doctest.NORMALIZE_WHITESPACE|doctest.ELLIPSIS)
>>> DTR.running_doctest_digest = hashlib.md5()
>>> filename = sage.doctest.forker.__file__
>>> FDS = FileDocTestSource(filename, DD)
>>> globs = RecordingDict(globals())
>>> 'doctest_var' in globs
False
>>> doctests, extras = FDS.create_doctests(globs)
>>> ex0 = doctests[Integer(0)].examples[Integer(0)]
```

(continues on next page)

(continued from previous page)

```
>>> flags = Integer(524288)
>>> def compiler(ex):
...     return compile(ex.source, '<doctest sage.doctest.forker[0]>',
...                    'single', flags, Integer(1))
>>> DTR.compile_and_execute(ex0, compiler, globs)
1764
>>> globs['doctest_var']
42
>>> globs.set
{'doctest_var'}
>>> globs.got
{'Integer'}
```

Now we can execute some more doctests to see the dependencies.

```
sage: ex1 = doctests[0].examples[1]
sage: def compiler(ex):
....:     return compile(ex.source, '<doctest sage.doctest.forker[1]>',
....:                    'single', flags, 1)
sage: DTR.compile_and_execute(ex1, compiler, globs)
sage: sorted(list(globs.set))
['R', 'a']
sage: globs.got
{'ZZ'}
sage: ex1.predecessors
[]
```

```
>>> from sage.all import *
>>> ex1 = doctests[Integer(0)].examples[Integer(1)]
>>> def compiler(ex):
...     return compile(ex.source, '<doctest sage.doctest.forker[1]>',
...                    'single', flags, Integer(1))
>>> DTR.compile_and_execute(ex1, compiler, globs)
>>> sorted(list(globs.set))
['R', 'a']
>>> globs.got
{'ZZ'}
>>> ex1.predecessors
[]
```

```
sage: ex2 = doctests[0].examples[2]
sage: def compiler(ex):
....:     return compile(ex.source, '<doctest sage.doctest.forker[2]>',
....:                    'single', flags, 1)
sage: DTR.compile_and_execute(ex2, compiler, globs)
a + 42
sage: list(globs.set)
[]
sage: sorted(list(globs.got))
['a', 'doctest_var']
sage: set(ex2.predecessors) == set([ex0,ex1])
True
```

```
>>> from sage.all import *
>>> ex2 = doctests[Integer(0)].examples[Integer(2)]
>>> def compiler(ex):
...     return compile(ex.source, '<doctest sage.doctest.forker[2]>',
...                    'single', flags, Integer(1))
>>> DTR.compile_and_execute(ex2, compiler, globs)
a + 42
>>> list(globs.set)
[]
>>> sorted(list(globs.got))
['a', 'doctest_var']
>>> set(ex2.predecessors) == set([ex0,ex1])
True
```

report_failure(*out*, *test*, *example*, *got*, *globs*)

Called when a doctest fails.

INPUT:

- `out` – a function for printing
- `test` – a `doctest.DocTest` instance
- `example` – a `doctest.Example` instance in `test`
- `got` – string, the result of running `example`
- `globs` – dictionary of globals, used if in debugging mode

OUTPUT: prints a report to `out`

EXAMPLES:

```
sage: from sage.doctest.parsing import SageOutputChecker
sage: from sage.doctest.forker import SageDocTestRunner
sage: from sage.doctest.sources import FileDocTestSource
sage: from sage.doctest.control import DocTestDefaults; DD = DocTestDefaults()
sage: import doctest, sys, os
sage: DTR = SageDocTestRunner(SageOutputChecker(), verbose=True, sage_
↪options=DD, optionflags=doctest.NORMALIZE_WHITESPACE|doctest.ELLIPSIS)
sage: filename = sage.doctest.forker.__file__
sage: FDS = FileDocTestSource(filename, DD)
sage: doctests, extras = FDS.create_doctests(globals())
sage: ex = doctests[0].examples[0]
sage: DTR.no_failure_yet = True
sage: DTR.report_failure(sys.stdout.write, doctests[0], ex, 'BAD ANSWER\n', {}
↪)
**********************************************************************
File ".../sage/doctest/forker.py", line ..., in sage.doctest.forker
Failed example:
    doctest_var = 42; doctest_var^2
Expected:
    1764
Got:
    BAD ANSWER
```

```
>>> from sage.all import *
>>> from sage.doctest.parsing import SageOutputChecker
>>> from sage.doctest.forker import SageDocTestRunner
>>> from sage.doctest.sources import FileDocTestSource
>>> from sage.doctest.control import DocTestDefaults; DD = DocTestDefaults()
>>> import doctest, sys, os
>>> DTR = SageDocTestRunner(SageOutputChecker(), verbose=True, sage_
↪options=DD, optionflags=doctest.NORMALIZE_WHITESPACE|doctest.ELLIPSIS)
>>> filename = sage.doctest.forker.__file__
>>> FDS = FileDocTestSource(filename, DD)
>>> doctests, extras = FDS.create_doctests(globals())
>>> ex = doctests[Integer(0)].examples[Integer(0)]
>>> DTR.no_failure_yet = True
>>> DTR.report_failure(sys.stdout.write, doctests[Integer(0)], ex, 'BAD␣
↪ANSWER\n', {})
**********************************************************************
File ".../sage/doctest/forker.py", line ..., in sage.doctest.forker
Failed example:
    doctest_var = 42; doctest_var^2
Expected:
    1764
Got:
    BAD ANSWER
```

If debugging is turned on this function starts an IPython prompt when a test returns an incorrect answer:

```
sage: sage0.quit()
sage: _ = sage0.eval("import doctest, sys, os, multiprocessing, subprocess")
sage: _ = sage0.eval("from sage.doctest.parsing import SageOutputChecker")
sage: _ = sage0.eval("import sage.doctest.forker as sdf")
sage: _ = sage0.eval("from sage.doctest.control import DocTestDefaults")
sage: _ = sage0.eval("DD = DocTestDefaults(debug=True)")
sage: _ = sage0.eval("ex1 = doctest.Example('a = 17', '')")
sage: _ = sage0.eval("ex2 = doctest.Example('2*a', '1')")
sage: _ = sage0.eval("DT = doctest.DocTest([ex1,ex2], globals(), 'doubling',␣
↪None, 0, None)")
sage: _ = sage0.eval("DTR = sdf.SageDocTestRunner(SageOutputChecker(),␣
↪verbose=False, sage_options=DD, optionflags=doctest.NORMALIZE_
↪WHITESPACE|doctest.ELLIPSIS)")
sage: print(sage0.eval("sdf.init_sage(); DTR.run(DT, clear_globs=False)")) #␣
↪indirect doctest
**********************************************************************
Line 1, in doubling
Failed example:
    2*a
Expected:
    1
Got:
    34
**********************************************************************
Previously executed commands:
sage: sage0._expect.expect('sage: ')    # sage0 just mis-identified the output␣
↪as prompt, synchronize
```

(continues on next page)

(continued from previous page)

```
0
sage: sage0.eval("a")
'...17'
sage: sage0.eval("quit")
'Returning to doctests...TestResults(failed=1, attempted=2)'
```

```
>>> from sage.all import *
>>> sage0.quit()
>>> _ = sage0.eval("import doctest, sys, os, multiprocessing, subprocess")
>>> _ = sage0.eval("from sage.doctest.parsing import SageOutputChecker")
>>> _ = sage0.eval("import sage.doctest.forker as sdf")
>>> _ = sage0.eval("from sage.doctest.control import DocTestDefaults")
>>> _ = sage0.eval("DD = DocTestDefaults(debug=True)")
>>> _ = sage0.eval("ex1 = doctest.Example('a = 17', '')")
>>> _ = sage0.eval("ex2 = doctest.Example('2*a', '1')")
>>> _ = sage0.eval("DT = doctest.DocTest([ex1,ex2], globals(), 'doubling',␣
↪None, 0, None)")
>>> _ = sage0.eval("DTR = sdf.SageDocTestRunner(SageOutputChecker(),␣
↪verbose=False, sage_options=DD, optionflags=doctest.NORMALIZE_
↪WHITESPACE|doctest.ELLIPSIS)")
>>> print(sage0.eval("sdf.init_sage(); DTR.run(DT, clear_globs=False)")) #␣
↪indirect doctest
**********************************************************************
Line 1, in doubling
Failed example:
    2*a
Expected:
    1
Got:
    34
**********************************************************************
Previously executed commands:
>>> sage0._expect.expect('sage: ')    # sage0 just mis-identified the output␣
↪as prompt, synchronize
0
>>> sage0.eval("a")
'...17'
>>> sage0.eval("quit")
'Returning to doctests...TestResults(failed=1, attempted=2)'
```

report_overtime(*out*, *test*, *example*, *got*, *check_timer*)

Called when the `warn_long` option flag is set and a doctest runs longer than the specified time.

INPUT:

- `out` – a function for printing
- `test` – a `doctest.DocTest` instance
- `example` – a `doctest.Example` instance in `test`
- `got` – string; the result of running `example`
- `check_timer` – a *sage.doctest.util.Timer* (default: `None`) that measures the time spent checking whether or not the output was correct

OUTPUT: prints a report to `out`

EXAMPLES:

```
sage: from sage.doctest.parsing import SageOutputChecker
sage: from sage.doctest.forker import SageDocTestRunner
sage: from sage.doctest.sources import FileDocTestSource
sage: from sage.doctest.control import DocTestDefaults; DD = DocTestDefaults()
sage: from sage.doctest.util import Timer
sage: import doctest, sys, os
sage: DTR = SageDocTestRunner(SageOutputChecker(), verbose=True, sage_
↪options=DD, optionflags=doctest.NORMALIZE_WHITESPACE|doctest.ELLIPSIS)
sage: filename = sage.doctest.forker.__file__
sage: FDS = FileDocTestSource(filename, DD)
sage: doctests, extras = FDS.create_doctests(globals())
sage: ex = doctests[0].examples[0]
sage: ex.cputime = 1.23
sage: ex.walltime = 2.50
sage: check = Timer()
sage: check.cputime = 2.34
sage: check.walltime = 3.12
sage: DTR.report_overtime(sys.stdout.write, doctests[0], ex, 'BAD ANSWER\n',␣
↪check_timer=check)
**********************************************************************
File ".../sage/doctest/forker.py", line ..., in sage.doctest.forker
Warning: slow doctest:
    doctest_var = 42; doctest_var^2
Test ran for 1.23s cpu, 2.50s wall
Check ran for 2.34s cpu, 3.12s wall
```

```
>>> from sage.all import *
>>> from sage.doctest.parsing import SageOutputChecker
>>> from sage.doctest.forker import SageDocTestRunner
>>> from sage.doctest.sources import FileDocTestSource
>>> from sage.doctest.control import DocTestDefaults; DD = DocTestDefaults()
>>> from sage.doctest.util import Timer
>>> import doctest, sys, os
>>> DTR = SageDocTestRunner(SageOutputChecker(), verbose=True, sage_
↪options=DD, optionflags=doctest.NORMALIZE_WHITESPACE|doctest.ELLIPSIS)
>>> filename = sage.doctest.forker.__file__
>>> FDS = FileDocTestSource(filename, DD)
>>> doctests, extras = FDS.create_doctests(globals())
>>> ex = doctests[Integer(0)].examples[Integer(0)]
>>> ex.cputime = RealNumber('1.23')
>>> ex.walltime = RealNumber('2.50')
>>> check = Timer()
>>> check.cputime = RealNumber('2.34')
>>> check.walltime = RealNumber('3.12')
>>> DTR.report_overtime(sys.stdout.write, doctests[Integer(0)], ex, 'BAD␣
↪ANSWER\n', check_timer=check)
**********************************************************************
File ".../sage/doctest/forker.py", line ..., in sage.doctest.forker
Warning: slow doctest:
    doctest_var = 42; doctest_var^2
```

(continues on next page)

(continued from previous page)

```
Test ran for 1.23s cpu, 2.50s wall
Check ran for 2.34s cpu, 3.12s wall
```

report_start(*out*, *test*, *example*)

Called when an example starts.

INPUT:

- `out` – a function for printing
- `test` – a `doctest.DocTest` instance
- `example` – a `doctest.Example` instance in `test`

OUTPUT: prints a report to `out`

EXAMPLES:

```
sage: from sage.doctest.parsing import SageOutputChecker
sage: from sage.doctest.forker import SageDocTestRunner
sage: from sage.doctest.sources import FileDocTestSource
sage: from sage.doctest.control import DocTestDefaults; DD = DocTestDefaults()
sage: import doctest, sys, os
sage: DTR = SageDocTestRunner(SageOutputChecker(), verbose=True, sage_
↪options=DD, optionflags=doctest.NORMALIZE_WHITESPACE|doctest.ELLIPSIS)
sage: filename = sage.doctest.forker.__file__
sage: FDS = FileDocTestSource(filename, DD)
sage: doctests, extras = FDS.create_doctests(globals())
sage: ex = doctests[0].examples[0]
sage: DTR.report_start(sys.stdout.write, doctests[0], ex)
Trying (line ...):    doctest_var = 42; doctest_var^2
Expecting:
    1764
```

```
>>> from sage.all import *
>>> from sage.doctest.parsing import SageOutputChecker
>>> from sage.doctest.forker import SageDocTestRunner
>>> from sage.doctest.sources import FileDocTestSource
>>> from sage.doctest.control import DocTestDefaults; DD = DocTestDefaults()
>>> import doctest, sys, os
>>> DTR = SageDocTestRunner(SageOutputChecker(), verbose=True, sage_
↪options=DD, optionflags=doctest.NORMALIZE_WHITESPACE|doctest.ELLIPSIS)
>>> filename = sage.doctest.forker.__file__
>>> FDS = FileDocTestSource(filename, DD)
>>> doctests, extras = FDS.create_doctests(globals())
>>> ex = doctests[Integer(0)].examples[Integer(0)]
>>> DTR.report_start(sys.stdout.write, doctests[Integer(0)], ex)
Trying (line ...):    doctest_var = 42; doctest_var^2
Expecting:
    1764
```

report_success(*out*, *test*, *example*, *got*, *check_timer*)

Called when an example succeeds.

INPUT:

- `out` – a function for printing
- `test` – a `doctest.DocTest` instance
- `example` – a `doctest.Example` instance in `test`
- `got` – string; the result of running `example`
- `check_timer` – a *`sage.doctest.util.Timer`* (default: `None`) that measures the time spent checking whether or not the output was correct

OUTPUT: prints a report to `out`; if in debugging mode, starts an IPython prompt at the point of the failure

EXAMPLES:

```
sage: from sage.doctest.parsing import SageOutputChecker
sage: from sage.doctest.forker import SageDocTestRunner
sage: from sage.doctest.sources import FileDocTestSource
sage: from sage.doctest.control import DocTestDefaults; DD = DocTestDefaults()
sage: from sage.doctest.util import Timer
sage: import doctest, sys, os
sage: DTR = SageDocTestRunner(SageOutputChecker(), verbose=True, sage_
↪options=DD, optionflags=doctest.NORMALIZE_WHITESPACE|doctest.ELLIPSIS)
sage: filename = sage.doctest.forker.__file__
sage: FDS = FileDocTestSource(filename, DD)
sage: doctests, extras = FDS.create_doctests(globals())
sage: ex = doctests[0].examples[0]
sage: ex.cputime = 1.01
sage: ex.walltime = 1.12
sage: check = Timer()
sage: check.cputime = 2.14
sage: check.walltime = 2.71
sage: DTR.report_success(sys.stdout.write, doctests[0], ex, '1764',
....:                    check_timer=check)
ok [3.83s wall]
```

```
>>> from sage.all import *
>>> from sage.doctest.parsing import SageOutputChecker
>>> from sage.doctest.forker import SageDocTestRunner
>>> from sage.doctest.sources import FileDocTestSource
>>> from sage.doctest.control import DocTestDefaults; DD = DocTestDefaults()
>>> from sage.doctest.util import Timer
>>> import doctest, sys, os
>>> DTR = SageDocTestRunner(SageOutputChecker(), verbose=True, sage_
↪options=DD, optionflags=doctest.NORMALIZE_WHITESPACE|doctest.ELLIPSIS)
>>> filename = sage.doctest.forker.__file__
>>> FDS = FileDocTestSource(filename, DD)
>>> doctests, extras = FDS.create_doctests(globals())
>>> ex = doctests[Integer(0)].examples[Integer(0)]
>>> ex.cputime = RealNumber('1.01')
>>> ex.walltime = RealNumber('1.12')
>>> check = Timer()
>>> check.cputime = RealNumber('2.14')
>>> check.walltime = RealNumber('2.71')
>>> DTR.report_success(sys.stdout.write, doctests[Integer(0)], ex, '1764',
...                    check_timer=check)
```

(continues on next page)

(continued from previous page)

```
ok [3.83s wall]
```

report_unexpected_exception(*out*, *test*, *example*, *exc_info*)

Called when a doctest raises an exception that's not matched by the expected output.

If debugging has been turned on, starts an interactive debugger.

INPUT:

- `out` – a function for printing
- `test` – a `doctest.DocTest` instance
- `example` – a `doctest.Example` instance in `test`
- `exc_info` – the result of `sys.exc_info()`

OUTPUT: prints a report to `out`

- if in debugging mode, starts PDB with the given traceback

EXAMPLES:

```
sage: from sage.interfaces.sage0 import sage0
sage: sage0.quit()
sage: _ = sage0.eval("import doctest, sys, os, multiprocessing, subprocess")
sage: _ = sage0.eval("from sage.doctest.parsing import SageOutputChecker")
sage: _ = sage0.eval("import sage.doctest.forker as sdf")
sage: _ = sage0.eval("from sage.doctest.control import DocTestDefaults")
sage: _ = sage0.eval("DD = DocTestDefaults(debug=True)")
sage: _ = sage0.eval("ex = doctest.Example('E = EllipticCurve([0,0]); E', 'A␣
↪singular Elliptic Curve')")
sage: _ = sage0.eval("DT = doctest.DocTest([ex], globals(), 'singular_curve',␣
↪None, 0, None)")
sage: _ = sage0.eval("DTR = sdf.SageDocTestRunner(SageOutputChecker(),␣
↪verbose=False, sage_options=DD, optionflags=doctest.NORMALIZE_
↪WHITESPACE|doctest.ELLIPSIS)")
sage: old_prompt = sage0._prompt
sage: sage0._prompt = r"\(Pdb\) "
sage: sage0.eval("DTR.run(DT, clear_globs=False)") # indirect doctest
'... ArithmeticError(self._equation_string() + " defines a singular curve")'
sage: sage0.eval("l")
'...if self.discriminant() == 0:...raise ArithmeticError...'
sage: sage0.eval("u")
'...-> super().__init__(R, data, category=category)'
sage: sage0.eval("u")
'...EllipticCurve_field.__init__(self, K, ainvs)'
sage: sage0.eval("p ainvs")
'(0, 0, 0, 0, 0)'
sage: sage0._prompt = old_prompt
sage: sage0.eval("quit")
'TestResults(failed=1, attempted=1)'
```

```
>>> from sage.all import *
>>> from sage.interfaces.sage0 import sage0
>>> sage0.quit()
```

(continues on next page)

(continued from previous page)

```
>>> _ = sage0.eval("import doctest, sys, os, multiprocessing, subprocess")
>>> _ = sage0.eval("from sage.doctest.parsing import SageOutputChecker")
>>> _ = sage0.eval("import sage.doctest.forker as sdf")
>>> _ = sage0.eval("from sage.doctest.control import DocTestDefaults")
>>> _ = sage0.eval("DD = DocTestDefaults(debug=True)")
>>> _ = sage0.eval("ex = doctest.Example('E = EllipticCurve([0,0]); E', 'A␣
↪singular Elliptic Curve')")
>>> _ = sage0.eval("DT = doctest.DocTest([ex], globals(), 'singular_curve',␣
↪None, 0, None)")
>>> _ = sage0.eval("DTR = sdf.SageDocTestRunner(SageOutputChecker(),␣
↪verbose=False, sage_options=DD, optionflags=doctest.NORMALIZE_
↪WHITESPACE|doctest.ELLIPSIS)")
>>> old_prompt = sage0._prompt
>>> sage0._prompt = r"\(Pdb\) "
>>> sage0.eval("DTR.run(DT, clear_globs=False)") # indirect doctest
'... ArithmeticError(self._equation_string() + " defines a singular curve")'
>>> sage0.eval("l")
'...if self.discriminant() == 0:...raise ArithmeticError...'
>>> sage0.eval("u")
'...-> super().__init__(R, data, category=category)'
>>> sage0.eval("u")
'...EllipticCurve_field.__init__(self, K, ainvs)'
>>> sage0.eval("p ainvs")
'(0, 0, 0, 0, 0)'
>>> sage0._prompt = old_prompt
>>> sage0.eval("quit")
'TestResults(failed=1, attempted=1)'
```

run(*test*, *compileflags=0*, *out=None*, *clear_globs=True*)

Run the examples in a given doctest.

This function replaces `doctest.DocTestRunner.run()` since it needs to handle spoofing. It also leaves the display hook in place.

INPUT:

- `test` – an instance of `doctest.DocTest`
- `compileflags` – integer (default: 0) the set of compiler flags used to execute examples (passed in to the `compile()`)
- `out` – a function for writing the output (defaults to `sys.stdout.write`)
- `clear_globs` – boolean (default: `True`); whether to clear the namespace after running this doctest

OUTPUT:

- `f` – integer, the number of examples that failed
- `t` – the number of examples tried

EXAMPLES:

```
sage: from sage.doctest.parsing import SageOutputChecker
sage: from sage.doctest.forker import SageDocTestRunner
sage: from sage.doctest.sources import FileDocTestSource
sage: from sage.doctest.control import DocTestDefaults; DD = DocTestDefaults()
```

(continues on next page)

(continued from previous page)

```
sage: import doctest, sys, os
sage: DTR = SageDocTestRunner(SageOutputChecker(), verbose=False, sage_
↪options=DD,
....:                         optionflags=doctest.NORMALIZE_
↪WHITESPACE|doctest.ELLIPSIS)
sage: filename = sage.doctest.forker.__file__
sage: FDS = FileDocTestSource(filename, DD)
sage: doctests, extras = FDS.create_doctests(globals())
sage: DTR.run(doctests[0], clear_globs=False)
TestResults(failed=0, attempted=4)
```

```
>>> from sage.all import *
>>> from sage.doctest.parsing import SageOutputChecker
>>> from sage.doctest.forker import SageDocTestRunner
>>> from sage.doctest.sources import FileDocTestSource
>>> from sage.doctest.control import DocTestDefaults; DD = DocTestDefaults()
>>> import doctest, sys, os
>>> DTR = SageDocTestRunner(SageOutputChecker(), verbose=False, sage_
↪options=DD,
...                         optionflags=doctest.NORMALIZE_WHITESPACE|doctest.
↪ELLIPSIS)
>>> filename = sage.doctest.forker.__file__
>>> FDS = FileDocTestSource(filename, DD)
>>> doctests, extras = FDS.create_doctests(globals())
>>> DTR.run(doctests[Integer(0)], clear_globs=False)
TestResults(failed=0, attempted=4)
```

summarize(*verbose=None*)

Print results of testing to `self.msgfile` and return number of failures and tests run.

INPUT:

- `verbose` – whether to print lots of stuff

OUTPUT:

- returns `(f, t)`, a *TestResults* instance giving the number of failures and the total number of tests run.

EXAMPLES:

```
sage: from sage.doctest.parsing import SageOutputChecker
sage: from sage.doctest.forker import SageDocTestRunner
sage: from sage.doctest.control import DocTestDefaults; DD = DocTestDefaults()
sage: import doctest, sys, os
sage: DTR = SageDocTestRunner(SageOutputChecker(), verbose=False, sage_
↪options=DD, optionflags=doctest.NORMALIZE_WHITESPACE|doctest.ELLIPSIS)
sage: DTR._stats['sage.doctest.forker'] = (1,120)
sage: results = DTR.summarize()
**********************************************************************
1 item had failures:
    1 of 120 in sage.doctest.forker
sage: results
TestResults(failed=1, attempted=120)
```

```
>>> from sage.all import *
>>> from sage.doctest.parsing import SageOutputChecker
>>> from sage.doctest.forker import SageDocTestRunner
>>> from sage.doctest.control import DocTestDefaults; DD = DocTestDefaults()
>>> import doctest, sys, os
>>> DTR = SageDocTestRunner(SageOutputChecker(), verbose=False, sage_
↪options=DD, optionflags=doctest.NORMALIZE_WHITESPACE|doctest.ELLIPSIS)
>>> DTR._stats['sage.doctest.forker'] = (Integer(1),Integer(120))
>>> results = DTR.summarize()
**********************************************************************
1 item had failures:
    1 of 120 in sage.doctest.forker
>>> results
TestResults(failed=1, attempted=120)
```

update_digests (*example*)

Update global and doctest digests.

Sage's doctest runner tracks the state of doctests so that their dependencies are known. For example, in the following two lines

```
sage: R.<x> = ZZ[]
sage: f = x^2 + 1
```

```
>>> from sage.all import *
>>> R = ZZ['x']; (x,) = R._first_ngens(1)
>>> f = x**Integer(2) + Integer(1)
```

it records that the second line depends on the first since the first INSERTS `x` into the global namespace and the second line RETRIEVES `x` from the global namespace.

This function updates the hashes that record these dependencies.

INPUT:

- `example` – a `doctest.Example` instance

EXAMPLES:

```
sage: from sage.doctest.parsing import SageOutputChecker
sage: from sage.doctest.forker import SageDocTestRunner
sage: from sage.doctest.sources import FileDocTestSource
sage: from sage.doctest.control import DocTestDefaults; DD = DocTestDefaults()
sage: import doctest, sys, os, hashlib
sage: DTR = SageDocTestRunner(SageOutputChecker(), verbose=False, sage_
↪options=DD, optionflags=doctest.NORMALIZE_WHITESPACE|doctest.ELLIPSIS)
sage: filename = sage.doctest.forker.__file__
sage: FDS = FileDocTestSource(filename, DD)
sage: doctests, extras = FDS.create_doctests(globals())
sage: DTR.running_global_digest.hexdigest()
'd41d8cd98f00b204e9800998ecf8427e'
sage: DTR.running_doctest_digest = hashlib.md5()
sage: ex = doctests[0].examples[0]; ex.predecessors = None
sage: DTR.update_digests(ex)
```

(continues on next page)

(continued from previous page)

```
sage: DTR.running_global_digest.hexdigest()
'3cb44104292c3a3ab4da3112ce5dc35c'
```

```
>>> from sage.all import *
>>> from sage.doctest.parsing import SageOutputChecker
>>> from sage.doctest.forker import SageDocTestRunner
>>> from sage.doctest.sources import FileDocTestSource
>>> from sage.doctest.control import DocTestDefaults; DD = DocTestDefaults()
>>> import doctest, sys, os, hashlib
>>> DTR = SageDocTestRunner(SageOutputChecker(), verbose=False, sage_
↪options=DD, optionflags=doctest.NORMALIZE_WHITESPACE|doctest.ELLIPSIS)
>>> filename = sage.doctest.forker.__file__
>>> FDS = FileDocTestSource(filename, DD)
>>> doctests, extras = FDS.create_doctests(globals())
>>> DTR.running_global_digest.hexdigest()
'd41d8cd98f00b204e9800998ecf8427e'
>>> DTR.running_doctest_digest = hashlib.md5()
>>> ex = doctests[Integer(0)].examples[Integer(0)]; ex.predecessors = None
>>> DTR.update_digests(ex)
>>> DTR.running_global_digest.hexdigest()
'3cb44104292c3a3ab4da3112ce5dc35c'
```

update_results(*D*)

When returning results we pick out the results of interest since many attributes are not pickleable.

INPUT:

- `D` – dictionary to update with cputime and walltime

OUTPUT: the number of failures (or `False` if there is no failure attribute)

EXAMPLES:

```
sage: from sage.doctest.parsing import SageOutputChecker
sage: from sage.doctest.forker import SageDocTestRunner
sage: from sage.doctest.sources import FileDocTestSource, DictAsObject
sage: from sage.doctest.control import DocTestDefaults; DD = DocTestDefaults()
sage: import doctest, sys, os
sage: DTR = SageDocTestRunner(SageOutputChecker(), verbose=False, sage_
↪options=DD, optionflags=doctest.NORMALIZE_WHITESPACE|doctest.ELLIPSIS)
sage: filename = sage.doctest.forker.__file__
sage: FDS = FileDocTestSource(filename, DD)
sage: doctests, extras = FDS.create_doctests(globals())
sage: from sage.doctest.util import Timer
sage: T = Timer().start()
sage: DTR.run(doctests[0])
TestResults(failed=0, attempted=4)
sage: T.stop().annotate(DTR)
sage: D = DictAsObject({'cputime': [], 'walltime': [], 'err': None})
sage: DTR.update_results(D)
0
sage: sorted(list(D.items()))
[('cputime', [...]), ('err', None), ('failures', 0), ('tests', 4),
 ('walltime', [...]), ('walltime_skips', 0)]
```

```
>>> from sage.all import *
>>> from sage.doctest.parsing import SageOutputChecker
>>> from sage.doctest.forker import SageDocTestRunner
>>> from sage.doctest.sources import FileDocTestSource, DictAsObject
>>> from sage.doctest.control import DocTestDefaults; DD = DocTestDefaults()
>>> import doctest, sys, os
>>> DTR = SageDocTestRunner(SageOutputChecker(), verbose=False, sage_
↪options=DD, optionflags=doctest.NORMALIZE_WHITESPACE|doctest.ELLIPSIS)
>>> filename = sage.doctest.forker.__file__
>>> FDS = FileDocTestSource(filename, DD)
>>> doctests, extras = FDS.create_doctests(globals())
>>> from sage.doctest.util import Timer
>>> T = Timer().start()
>>> DTR.run(doctests[Integer(0)])
TestResults(failed=0, attempted=4)
>>> T.stop().annotate(DTR)
>>> D = DictAsObject({'cputime': [], 'walltime': [], 'err': None})
>>> DTR.update_results(D)
0
>>> sorted(list(D.items()))
[('cputime', [...]), ('err', None), ('failures', 0), ('tests', 4),
 ('walltime', [...]), ('walltime_skips', 0)]
```

class sage.doctest.forker.**SageSpoofInOut**(*outfile=None*, *infile=None*)

Bases: SageObject

We replace the standard `doctest._SpoofOut` for three reasons:

- we need to divert the output of C programs that don't print through sys.stdout,
- we want the ability to recover partial output from doctest processes that segfault.
- we also redirect stdin (usually from /dev/null) during doctests.

This class defines streams `self.real_stdin`, `self.real_stdout` and `self.real_stderr` which refer to the original streams.

INPUT:

- `outfile` – (default: `tempfile.TemporaryFile()`) a seekable open file object to which stdout and stderr should be redirected
- `infile` – (default: `open(os.devnull)`) an open file object from which stdin should be redirected

EXAMPLES:

```
sage: import subprocess, tempfile
sage: from sage.doctest.forker import SageSpoofInOut
sage: O = tempfile.TemporaryFile()
sage: S = SageSpoofInOut(O)
sage: try:
....:     S.start_spoofing()
....:     print("hello world")
....: finally:
....:     S.stop_spoofing()
....:
sage: S.getvalue()
```

(continues on next page)

(continued from previous page)

```
'hello world\n'
sage: _ = O.seek(0)
sage: S = SageSpoofInOut(outfile=sys.stdout, infile=O)
sage: try:
....:     S.start_spoofing()
....:     _ = subprocess.check_call("cat")
....: finally:
....:     S.stop_spoofing()
....:
hello world
sage: O.close()
```

```
>>> from sage.all import *
>>> import subprocess, tempfile
>>> from sage.doctest.forker import SageSpoofInOut
>>> O = tempfile.TemporaryFile()
>>> S = SageSpoofInOut(O)
>>> try:
...     S.start_spoofing()
...     print("hello world")
... finally:
...     S.stop_spoofing()
....:
>>> S.getvalue()
'hello world\n'
>>> _ = O.seek(Integer(0))
>>> S = SageSpoofInOut(outfile=sys.stdout, infile=O)
>>> try:
...     S.start_spoofing()
...     _ = subprocess.check_call("cat")
... finally:
...     S.stop_spoofing()
....:
hello world
>>> O.close()
```

getvalue()

Get the value that has been printed to `outfile` since the last time this function was called.

EXAMPLES:

```
sage: from sage.doctest.forker import SageSpoofInOut
sage: S = SageSpoofInOut()
sage: try:
....:     S.start_spoofing()
....:     print("step 1")
....: finally:
....:     S.stop_spoofing()
....:
sage: S.getvalue()
'step 1\n'
sage: try:
```

(continues on next page)

(continued from previous page)

```
....:     S.start_spoofing()
....:     print("step 2")
....: finally:
....:     S.stop_spoofing()
....:
sage: S.getvalue()
'step 2\n'
```

```
>>> from sage.all import *
>>> from sage.doctest.forker import SageSpoofInOut
>>> S = SageSpoofInOut()
>>> try:
...     S.start_spoofing()
...     print("step 1")
... finally:
...     S.stop_spoofing()
....:
>>> S.getvalue()
'step 1\n'
>>> try:
...     S.start_spoofing()
...     print("step 2")
... finally:
...     S.stop_spoofing()
....:
>>> S.getvalue()
'step 2\n'
```

start_spoofing()

Set stdin to read from `self.infile` and stdout to print to `self.outfile`.

EXAMPLES:

```
sage: import os, tempfile
sage: from sage.doctest.forker import SageSpoofInOut
sage: O = tempfile.TemporaryFile()
sage: S = SageSpoofInOut(O)
sage: try:
....:     S.start_spoofing()
....:     print("this is not printed")
....: finally:
....:     S.stop_spoofing()
....:
sage: S.getvalue()
'this is not printed\n'
sage: _ = O.seek(0)
sage: S = SageSpoofInOut(infile=O)
sage: try:
....:     S.start_spoofing()
....:     v = sys.stdin.read()
....: finally:
....:     S.stop_spoofing()
```

(continues on next page)

(continued from previous page)

```
....:
sage: v
'this is not printed\n'
```

```
>>> from sage.all import *
>>> import os, tempfile
>>> from sage.doctest.forker import SageSpoofInOut
>>> O = tempfile.TemporaryFile()
>>> S = SageSpoofInOut(O)
>>> try:
...     S.start_spoofing()
...     print("this is not printed")
... finally:
...     S.stop_spoofing()
....:
>>> S.getvalue()
'this is not printed\n'
>>> _ = O.seek(Integer(0))
>>> S = SageSpoofInOut(infile=O)
>>> try:
...     S.start_spoofing()
...     v = sys.stdin.read()
... finally:
...     S.stop_spoofing()
....:
>>> v
'this is not printed\n'
```

We also catch non-Python output:

```
sage: try:
....:     S.start_spoofing()
....:     retval = os.system('''echo "Hello there"\nif [ $? -eq 0 ]; then\
↪necho "good"\nfi''')
....: finally:
....:     S.stop_spoofing()
....:
sage: S.getvalue()
'Hello there\ngood\n'
sage: O.close()
```

```
>>> from sage.all import *
>>> try:
...     S.start_spoofing()
...     retval = os.system('''echo "Hello there"\nif [ $? -eq 0 ]; then\necho
↪"good"\nfi''')
... finally:
...     S.stop_spoofing()
....:
>>> S.getvalue()
'Hello there\ngood\n'
>>> O.close()
```

stop_spoofing()

Reset stdin and stdout to their original values.

EXAMPLES:

```
sage: from sage.doctest.forker import SageSpoofInOut
sage: S = SageSpoofInOut()
sage: try:
....:     S.start_spoofing()
....:     print("this is not printed")
....: finally:
....:     S.stop_spoofing()
....:
sage: print("this is now printed")
this is now printed
```

```
>>> from sage.all import *
>>> from sage.doctest.forker import SageSpoofInOut
>>> S = SageSpoofInOut()
>>> try:
...     S.start_spoofing()
...     print("this is not printed")
... finally:
...     S.stop_spoofing()
....:
>>> print("this is now printed")
this is now printed
```

class sage.doctest.forker.**TestResults**(*failed*, *attempted*)

Bases: tuple

attempted

Alias for field number 1

failed

Alias for field number 0

sage.doctest.forker.**dummy_handler**(*sig*, *frame*)

Dummy signal handler for SIGCHLD (just to ensure the signal isn't ignored).

sage.doctest.forker.**init_sage**(*controller=None*)

Import the Sage library.

This function is called once at the beginning of a doctest run (rather than once for each file). It imports the Sage library, sets DOCTEST_MODE to True, and invalidates any interfaces.

EXAMPLES:

```
sage: from sage.doctest.forker import init_sage
sage: sage.doctest.DOCTEST_MODE = False
sage: init_sage()
sage: sage.doctest.DOCTEST_MODE
True
```

```
>>> from sage.all import *
>>> from sage.doctest.forker import init_sage
>>> sage.doctest.DOCTEST_MODE = False
>>> init_sage()
>>> sage.doctest.DOCTEST_MODE
True
```

Check that pexpect interfaces are invalidated, but still work:

```
sage: gap.eval("my_test_var := 42;")
'42'
sage: gap.eval("my_test_var;")
'42'
sage: init_sage()
sage: gap('Group((1,2,3)(4,5), (3,4))')
Group( [ (1,2,3)(4,5), (3,4) ] )
sage: gap.eval("my_test_var;")
Traceback (most recent call last):
...
RuntimeError: Gap produced error output...
```

```
>>> from sage.all import *
>>> gap.eval("my_test_var := 42;")
'42'
>>> gap.eval("my_test_var;")
'42'
>>> init_sage()
>>> gap('Group((1,2,3)(4,5), (3,4))')
Group( [ (1,2,3)(4,5), (3,4) ] )
>>> gap.eval("my_test_var;")
Traceback (most recent call last):
...
RuntimeError: Gap produced error output...
```

Check that SymPy equation pretty printer is limited in doctest mode to default width (80 chars):

```
sage: # needs sympy
sage: from sympy import sympify
sage: from sympy.printing.pretty.pretty import PrettyPrinter
sage: s = sympify('+x^'.join(str(i) for i in range(30)))
sage: print(PrettyPrinter(settings={'wrap_line': True}).doprint(s))
 29    28    27    26    25    24    23    22    21    20    19    18    17...
x   + x   + x   + x   + x   + x   + x   + x   + x   + x   + x   + x   + x...

... 16    15    14    13    12    11    10    9    8    7    6    5    4    3...
...x   + x   + x   + x   + x   + x   + x   + x  + x  + x  + x  + x  + x  + x...

...
```

```
>>> from sage.all import *
>>> # needs sympy
>>> from sympy import sympify
>>> from sympy.printing.pretty.pretty import PrettyPrinter
```

(continues on next page)

(continued from previous page)

```
>>> s = sympify('+x^'.join(str(i) for i in range(Integer(30))))
>>> print(PrettyPrinter(settings={'wrap_line': True}).doprint(s))
 29    28    27    26    25    24    23    22    21    20    19    18    17...
x   + x   + x   + x   + x   + x   + x   + x   + x   + x   + x   + x   + x...
<BLANKLINE>
... 16    15    14    13    12    11    10    9    8    7    6    5    4    3...
...x   + x   + x   + x   + x   + x   + x   + x  + x  + x  + x  + x  + x  + x...
<BLANKLINE>
...
```

The displayhook sorts dictionary keys to simplify doctesting of dictionary output:

```
sage: {'a':23, 'b':34, 'au':56, 'bbf':234, 'aaa':234}
{'a': 23, 'aaa': 234, 'au': 56, 'b': 34, 'bbf': 234}
```

```
>>> from sage.all import *
>>> {'a':Integer(23), 'b':Integer(34), 'au':Integer(56), 'bbf':Integer(234), 'aaa
↪':Integer(234)}
{'a': 23, 'aaa': 234, 'au': 56, 'b': 34, 'bbf': 234}
```

sage.doctest.forker.**showwarning_with_traceback**(*message*, *category*, *filename*, *lineno*, *file=None*, *line=None*)

Displays a warning message with a traceback.

INPUT: see `warnings.showwarning()` with the difference that with `file=None`
the message will be written to stdout.

OUTPUT: none

EXAMPLES:

```
sage: from sage.doctest.forker import showwarning_with_traceback
sage: showwarning_with_traceback("bad stuff", UserWarning, "myfile.py", 0)
doctest:warning...
  File "<doctest sage.doctest.forker.showwarning_with_traceback[1]>", line 1, in
↪<module>
    showwarning_with_traceback("bad stuff", UserWarning, "myfile.py", Integer(0))
:
UserWarning: bad stuff
```

```
>>> from sage.all import *
>>> from sage.doctest.forker import showwarning_with_traceback
>>> showwarning_with_traceback("bad stuff", UserWarning, "myfile.py", Integer(0))
doctest:warning...
  File "<doctest sage.doctest.forker.showwarning_with_traceback[1]>", line 1, in
↪<module>
    showwarning_with_traceback("bad stuff", UserWarning, "myfile.py", Integer(0))
:
UserWarning: bad stuff
```

CHAPTER

FOUR

PARSING DOCSTRINGS

This module contains functions and classes that parse docstrings.

AUTHORS:

- David Roe (2012-03-27) – initial version, based on Robert Bradshaw's code.
- Jeroen Demeyer(2014-08-28) – much improved handling of tolerances using interval arithmetic (Issue #16889).

class sage.doctest.parsing.**OriginalSource**(*example*)

Bases: object

Context swapping out the pre-parsed source with the original for better reporting.

EXAMPLES:

```
sage: from sage.doctest.sources import FileDocTestSource
sage: from sage.doctest.control import DocTestDefaults
sage: filename = sage.doctest.forker.__file__
sage: FDS = FileDocTestSource(filename, DocTestDefaults())
sage: doctests, extras = FDS.create_doctests(globals())
sage: ex = doctests[0].examples[0]
sage: ex.sage_source
'doctest_var = 42; doctest_var^2\n'
sage: ex.source
'doctest_var = Integer(42); doctest_var**Integer(2)\n'
sage: from sage.doctest.parsing import OriginalSource
sage: with OriginalSource(ex):
....:     ex.source
'doctest_var = 42; doctest_var^2\n'
```

```
>>> from sage.all import *
>>> from sage.doctest.sources import FileDocTestSource
>>> from sage.doctest.control import DocTestDefaults
>>> filename = sage.doctest.forker.__file__
>>> FDS = FileDocTestSource(filename, DocTestDefaults())
>>> doctests, extras = FDS.create_doctests(globals())
>>> ex = doctests[Integer(0)].examples[Integer(0)]
>>> ex.sage_source
'doctest_var = 42; doctest_var^2\n'
>>> ex.source
'doctest_var = Integer(42); doctest_var**Integer(2)\n'
>>> from sage.doctest.parsing import OriginalSource
>>> with OriginalSource(ex):
```

(continues on next page)

(continued from previous page)

```
...       ex.source
'doctest_var = 42; doctest_var^2\n'
```

class sage.doctest.parsing.**SageDocTestParser**(*optional_tags=()*, *long=False*, ** (Keyword-only parameters separator (PEP 3102))*, *probed_tags=()*, *file_optional_tags=()*)

Bases: DocTestParser

A version of the standard doctest parser which handles Sage's custom options and tolerances in floating point arithmetic.

file_optional_tags: set[str]

long: bool

optional_only: bool

optional_tags: bool | set[str]

optionals: dict[str, int]

parse(*string*, **args*)

A Sage specialization of doctest.DocTestParser.

INPUT:

- string – the string to parse
- name – (optional) string giving the name identifying string, to be used in error messages

OUTPUT:

- A list consisting of strings and doctest.Example instances. There will be at least one string between successive examples (exactly one unless long or optional tests are removed), and it will begin and end with a string.

EXAMPLES:

```
sage: from sage.doctest.parsing import SageDocTestParser
sage: DTP = SageDocTestParser(('sage','magma','guava'))
sage: example = 'Explanatory text::\n\n    sage: E = magma("EllipticCurve([1,␣
↪1, 1, -10, -10])") # optional: magma\n\nLater text'
sage: parsed = DTP.parse(example)
sage: parsed[0]
'Explanatory text::\n\n'
sage: parsed[1].sage_source
'E = magma("EllipticCurve([1, 1, 1, -10, -10])") # optional: magma\n'
sage: parsed[2]
'\nLater text'
```

```
>>> from sage.all import *
>>> from sage.doctest.parsing import SageDocTestParser
>>> DTP = SageDocTestParser(('sage','magma','guava'))
>>> example = 'Explanatory text::\n\n    sage: E = magma("EllipticCurve([1, 1,
↪ 1, -10, -10])") # optional: magma\n\nLater text'
>>> parsed = DTP.parse(example)
>>> parsed[Integer(0)]
```

(continues on next page)

(continued from previous page)

```
'Explanatory text::\n\n'
>>> parsed[Integer(1)].sage_source
'E = magma("EllipticCurve([1, 1, 1, -10, -10])") # optional: magma\n'
>>> parsed[Integer(2)]
'\nLater text'
```

If the doctest parser is not created to accept a given optional argument, the corresponding examples will just be removed:

```
sage: DTP2 = SageDocTestParser(('sage',))
sage: parsed2 = DTP2.parse(example)
sage: parsed2
['Explanatory text::\n\n', '\nLater text']
```

```
>>> from sage.all import *
>>> DTP2 = SageDocTestParser(('sage',))
>>> parsed2 = DTP2.parse(example)
>>> parsed2
['Explanatory text::\n\n', '\nLater text']
```

You can mark doctests as having a particular tolerance:

```
sage: example2 = 'sage: gamma(1.6) # tol 2.0e-11\n0.893515349287690'
sage: ex = DTP.parse(example2)[1]
sage: ex.sage_source
'gamma(1.6) # tol 2.0e-11\n'
sage: ex.want
'0.893515349287690\n'
sage: type(ex.want)
<class 'sage.doctest.marked_output.MarkedOutput'>
sage: ex.want.tol
2.000000000000000000...?e-11
```

```
>>> from sage.all import *
>>> example2 = 'sage: gamma(1.6) # tol 2.0e-11\n0.893515349287690'
>>> ex = DTP.parse(example2)[Integer(1)]
>>> ex.sage_source
'gamma(1.6) # tol 2.0e-11\n'
>>> ex.want
'0.893515349287690\n'
>>> type(ex.want)
<class 'sage.doctest.marked_output.MarkedOutput'>
>>> ex.want.tol
2.000000000000000000...?e-11
```

You can use continuation lines:

```
sage: s = "sage: for i in range(4):\n....:     print(i)\n....:\n"
sage: ex = DTP2.parse(s)[1]
sage: ex.source
'for i in range(Integer(4)):\n    print(i)\n'
```

```
>>> from sage.all import *
>>> s = "sage: for i in range(4):\n....:     print(i)\n....:\n"
>>> ex = DTP2.parse(s)[Integer(1)]
>>> ex.source
'for i in range(Integer(4)):\n    print(i)\n'
```

Optional tags at the start of an example block persist to the end of the block (delimited by a blank line):

```
sage: # long time, needs sage.rings.number_field
sage: QQbar(I)^10000
1
sage: QQbar(I)^10000  # not tested
I

sage: # needs sage.rings.finite_rings
sage: GF(7)
Finite Field of size 7
sage: GF(10)
Traceback (most recent call last):
...
ValueError: the order of a finite field must be a prime power
```

```
>>> from sage.all import *
>>> # long time, needs sage.rings.number_field
>>> QQbar(I)**Integer(10000)
1
>>> QQbar(I)**Integer(10000)  # not tested
I

>>> # needs sage.rings.finite_rings
>>> GF(Integer(7))
Finite Field of size 7
>>> GF(Integer(10))
Traceback (most recent call last):
...
ValueError: the order of a finite field must be a prime power
```

Test that Issue #26575 is resolved:

```
sage: example3 = 'sage: Zp(5,4,print_mode="digits")(5)\n...00010'
sage: parsed3 = DTP.parse(example3)
sage: dte = parsed3[1]
sage: dte.sage_source
'Zp(5,4,print_mode="digits")(5)\n'
sage: dte.want
'...00010\n'
```

```
>>> from sage.all import *
>>> example3 = 'sage: Zp(5,4,print_mode="digits")(5)\n...00010'
>>> parsed3 = DTP.parse(example3)
>>> dte = parsed3[Integer(1)]
>>> dte.sage_source
'Zp(5,4,print_mode="digits")(5)\n'
```

(continues on next page)

(continued from previous page)

```
>>> dte.want
'...00010\n'
```

Style warnings:

```
sage: def parse(test_string):
....:     return [x if isinstance(x, str)
....:                else (getattr(x, 'warnings', None), x.sage_source, x.
↪source)
....:             for x in DTP.parse(test_string)]

sage: parse('sage: 1 # optional guava mango\nsage: 2 # optional guava\nsage:␣
↪3 # optional guava\nsage: 4 # optional guava\nsage: 5 # optional guava\n\
↪nsage: 11 # optional guava')
['',
 (["Consider using a block-scoped tag by inserting the line 'sage: # optional␣
↪- guava' just before this line to avoid repeating the tag 5 times"],
  '1 # optional guava mango\n',
  'None  # virtual doctest'),
 '',
 (None, '2 # optional guava\n', 'Integer(2) # optional guava\n'),
 '',
 (None, '3 # optional guava\n', 'Integer(3) # optional guava\n'),
 '',
 (None, '4 # optional guava\n', 'Integer(4) # optional guava\n'),
 '',
 (None, '5 # optional guava\n', 'Integer(5) # optional guava\n'),
 '\n',
 (None, '11 # optional guava\n', 'Integer(11) # optional guava\n'),
 '']

sage: parse('sage: 1 # optional guava\nsage: 2 # optional guava mango\nsage:␣
↪3 # optional guava\nsage: 4 # optional guava\nsage: 5 # optional guava\n')
['',
 (["Consider using a block-scoped tag by inserting the line 'sage: # optional␣
↪- guava' just before this line to avoid repeating the tag 5 times"],
  '1 # optional guava\n',
  'Integer(1) # optional guava\n'),
 '',
 '',
 (None, '3 # optional guava\n', 'Integer(3) # optional guava\n'),
 '',
 (None, '4 # optional guava\n', 'Integer(4) # optional guava\n'),
 '',
 (None, '5 # optional guava\n', 'Integer(5) # optional guava\n'),
 '']

sage: parse('sage: # optional mango\nsage: 1 # optional guava\nsage: 2 #␣
↪optional guava mango\nsage: 3 # optional guava\nsage: 4 # optional guava\n␣
↪sage: 5 # optional guava\n')  # optional - guava mango
['',
 (["Consider updating this block-scoped tag to 'sage: # optional - guava mango
```

(continues on next page)

(continued from previous page)

```
↪' to avoid repeating the tag 5 times"],
  '# optional mango\n',
  'None  # virtual doctest'),
 '',
 '',
 '',
 '',
 '',
 '']

sage: parse('::\n\n    sage: 1 # optional guava\n    sage: 2 # optional guava␣
↪mango\n    sage: 3 # optional guava\n\n::\n\n    sage: 4 # optional guava\n␣
↪    sage: 5 # optional guava\n')
['::\n\n',
(None, '1 # optional guava\n', 'Integer(1) # optional guava\n'),
'',
'',
(None, '3 # optional guava\n', 'Integer(3) # optional guava\n'),
'\n::\n\n',
(None, '4 # optional guava\n', 'Integer(4) # optional guava\n'),
'',
(None, '5 # optional guava\n', 'Integer(5) # optional guava\n'),
'']
```

```
>>> from sage.all import *
>>> def parse(test_string):
...     return [x if isinstance(x, str)
...               else (getattr(x, 'warnings', None), x.sage_source, x.source)
...             for x in DTP.parse(test_string)]

>>> parse('sage: 1 # optional guava mango\nsage: 2 # optional guava\nsage: 3
↪# optional guava\nsage: 4 # optional guava\nsage: 5 # optional guava\n\
↪nsage: 11 # optional guava')
['',
 (["Consider using a block-scoped tag by inserting the line 'sage: # optional␣
↪- guava' just before this line to avoid repeating the tag 5 times"],
  '1 # optional guava mango\n',
  'None  # virtual doctest'),
 '',
 (None, '2 # optional guava\n', 'Integer(2) # optional guava\n'),
 '',
 (None, '3 # optional guava\n', 'Integer(3) # optional guava\n'),
 '',
 (None, '4 # optional guava\n', 'Integer(4) # optional guava\n'),
 '',
 (None, '5 # optional guava\n', 'Integer(5) # optional guava\n'),
 '\n',
 (None, '11 # optional guava\n', 'Integer(11) # optional guava\n'),
 '']

>>> parse('sage: 1 # optional guava\nsage: 2 # optional guava mango\nsage: 3
↪# optional guava\nsage: 4 # optional guava\nsage: 5 # optional guava\n')
```

(continues on next page)

(continued from previous page)

```
['',
 (["Consider using a block-scoped tag by inserting the line 'sage: # optional
↪- guava' just before this line to avoid repeating the tag 5 times"],
  '1 # optional guava\n',
  'Integer(1) # optional guava\n'),
 '',
 '',
 (None, '3 # optional guava\n', 'Integer(3) # optional guava\n'),
 '',
 (None, '4 # optional guava\n', 'Integer(4) # optional guava\n'),
 '',
 (None, '5 # optional guava\n', 'Integer(5) # optional guava\n'),
 '']

>>> parse('sage: # optional mango\nsage: 1 # optional guava\nsage: 2 #
↪optional guava mango\nsage: 3 # optional guava\nsage: 4 # optional guava\n
↪sage: 5 # optional guava\n')  # optional - guava mango
['',
 (["Consider updating this block-scoped tag to 'sage: # optional - guava mango
↪' to avoid repeating the tag 5 times"],
  '# optional mango\n',
  'None  # virtual doctest'),
 '',
 '',
 '',
 '',
 '',
 '']

>>> parse('::\n\n    sage: 1 # optional guava\n    sage: 2 # optional guava
↪mango\n    sage: 3 # optional guava\n\n::\n\n    sage: 4 # optional guava\n
↪    sage: 5 # optional guava\n')
['::\n\n',
(None, '1 # optional guava\n', 'Integer(1) # optional guava\n'),
'',
'',
(None, '3 # optional guava\n', 'Integer(3) # optional guava\n'),
'\n::\n\n',
(None, '4 # optional guava\n', 'Integer(4) # optional guava\n'),
'',
(None, '5 # optional guava\n', 'Integer(5) # optional guava\n'),
'']
```

probed_tags: bool | set[str]

class sage.doctest.parsing.**SageOutputChecker**

Bases: OutputChecker

A modification of the doctest OutputChecker that can check relative and absolute tolerance of answers.

EXAMPLES:

```
sage: from sage.doctest.parsing import SageOutputChecker, MarkedOutput,
```

(continues on next page)

(continued from previous page)

```
↪SageDocTestParser
sage: import doctest
sage: optflag = doctest.NORMALIZE_WHITESPACE|doctest.ELLIPSIS
sage: DTP = SageDocTestParser(('sage','magma','guava'))
sage: OC = SageOutputChecker()
sage: example2 = 'sage: gamma(1.6) # tol 2.0e-11\n0.893515349287690'
sage: ex = DTP.parse(example2)[1]
sage: ex.sage_source
'gamma(1.6) # tol 2.0e-11\n'
sage: ex.want
'0.893515349287690\n'
sage: type(ex.want)
<class 'sage.doctest.marked_output.MarkedOutput'>
sage: ex.want.tol
2.000000000000000000...?e-11
sage: OC.check_output(ex.want, '0.893515349287690', optflag)
True
sage: OC.check_output(ex.want, '0.8935153492877', optflag)
True
sage: OC.check_output(ex.want, '0', optflag)
False
sage: OC.check_output(ex.want, 'x + 0.8935153492877', optflag)
False
```

```
>>> from sage.all import *
>>> from sage.doctest.parsing import SageOutputChecker, MarkedOutput,␣
↪SageDocTestParser
>>> import doctest
>>> optflag = doctest.NORMALIZE_WHITESPACE|doctest.ELLIPSIS
>>> DTP = SageDocTestParser(('sage','magma','guava'))
>>> OC = SageOutputChecker()
>>> example2 = 'sage: gamma(1.6) # tol 2.0e-11\n0.893515349287690'
>>> ex = DTP.parse(example2)[Integer(1)]
>>> ex.sage_source
'gamma(1.6) # tol 2.0e-11\n'
>>> ex.want
'0.893515349287690\n'
>>> type(ex.want)
<class 'sage.doctest.marked_output.MarkedOutput'>
>>> ex.want.tol
2.000000000000000000...?e-11
>>> OC.check_output(ex.want, '0.893515349287690', optflag)
True
>>> OC.check_output(ex.want, '0.8935153492877', optflag)
True
>>> OC.check_output(ex.want, '0', optflag)
False
>>> OC.check_output(ex.want, 'x + 0.8935153492877', optflag)
False
```

check_output(*want*, *got*, *optionflags*)

Check to see if the output matches the desired output.

If `want` is a *MarkedOutput* instance, takes into account the desired tolerance.

INPUT:

- `want` – string or *MarkedOutput*
- `got` – string
- `optionflags` – integer; passed down to `doctest.OutputChecker`

OUTPUT: boolean; whether `got` matches `want` up to the specified tolerance

EXAMPLES:

```
sage: from sage.doctest.parsing import MarkedOutput, SageOutputChecker
sage: import doctest
sage: optflag = doctest.NORMALIZE_WHITESPACE|doctest.ELLIPSIS
sage: rndstr = MarkedOutput("I'm wrong!").update(random=True)
sage: tentol = MarkedOutput("10.0").update(tol=.1)
sage: tenabs = MarkedOutput("10.0").update(abs_tol=.1)
sage: tenrel = MarkedOutput("10.0").update(rel_tol=.1)
sage: zerotol = MarkedOutput("0.0").update(tol=.1)
sage: zeroabs = MarkedOutput("0.0").update(abs_tol=.1)
sage: zerorel = MarkedOutput("0.0").update(rel_tol=.1)
sage: zero = "0.0"
sage: nf = "9.5"
sage: ten = "10.05"
sage: eps = "-0.05"
sage: OC = SageOutputChecker()
```

```
>>> from sage.all import *
>>> from sage.doctest.parsing import MarkedOutput, SageOutputChecker
>>> import doctest
>>> optflag = doctest.NORMALIZE_WHITESPACE|doctest.ELLIPSIS
>>> rndstr = MarkedOutput("I'm wrong!").update(random=True)
>>> tentol = MarkedOutput("10.0").update(tol=RealNumber('.1'))
>>> tenabs = MarkedOutput("10.0").update(abs_tol=RealNumber('.1'))
>>> tenrel = MarkedOutput("10.0").update(rel_tol=RealNumber('.1'))
>>> zerotol = MarkedOutput("0.0").update(tol=RealNumber('.1'))
>>> zeroabs = MarkedOutput("0.0").update(abs_tol=RealNumber('.1'))
>>> zerorel = MarkedOutput("0.0").update(rel_tol=RealNumber('.1'))
>>> zero = "0.0"
>>> nf = "9.5"
>>> ten = "10.05"
>>> eps = "-0.05"
>>> OC = SageOutputChecker()
```

```
sage: OC.check_output(rndstr,nf,optflag)
True

sage: OC.check_output(tentol,nf,optflag)
True
sage: OC.check_output(tentol,ten,optflag)
True
sage: OC.check_output(tentol,zero,optflag)
False
```

(continues on next page)

(continued from previous page)

```
sage: OC.check_output(tenabs,nf,optflag)
False
sage: OC.check_output(tenabs,ten,optflag)
True
sage: OC.check_output(tenabs,zero,optflag)
False

sage: OC.check_output(tenrel,nf,optflag)
True
sage: OC.check_output(tenrel,ten,optflag)
True
sage: OC.check_output(tenrel,zero,optflag)
False

sage: OC.check_output(zerotol,zero,optflag)
True
sage: OC.check_output(zerotol,eps,optflag)
True
sage: OC.check_output(zerotol,ten,optflag)
False

sage: OC.check_output(zeroabs,zero,optflag)
True
sage: OC.check_output(zeroabs,eps,optflag)
True
sage: OC.check_output(zeroabs,ten,optflag)
False

sage: OC.check_output(zerorel,zero,optflag)
True
sage: OC.check_output(zerorel,eps,optflag)
False
sage: OC.check_output(zerorel,ten,optflag)
False
```

```
>>> from sage.all import *
>>> OC.check_output(rndstr,nf,optflag)
True

>>> OC.check_output(tentol,nf,optflag)
True
>>> OC.check_output(tentol,ten,optflag)
True
>>> OC.check_output(tentol,zero,optflag)
False

>>> OC.check_output(tenabs,nf,optflag)
False
>>> OC.check_output(tenabs,ten,optflag)
True
>>> OC.check_output(tenabs,zero,optflag)
```

(continues on next page)

(continued from previous page)

```
False

>>> OC.check_output(tenrel,nf,optflag)
True
>>> OC.check_output(tenrel,ten,optflag)
True
>>> OC.check_output(tenrel,zero,optflag)
False

>>> OC.check_output(zerotol,zero,optflag)
True
>>> OC.check_output(zerotol,eps,optflag)
True
>>> OC.check_output(zerotol,ten,optflag)
False

>>> OC.check_output(zeroabs,zero,optflag)
True
>>> OC.check_output(zeroabs,eps,optflag)
True
>>> OC.check_output(zeroabs,ten,optflag)
False

>>> OC.check_output(zerorel,zero,optflag)
True
>>> OC.check_output(zerorel,eps,optflag)
False
>>> OC.check_output(zerorel,ten,optflag)
False
```

More explicit tolerance checks:

```
sage: _ = x  # rel tol 1e10                                                    #␣
↪needs sage.symbolic
sage: raise RuntimeError   # rel tol 1e10
Traceback (most recent call last):
...
RuntimeError
sage: 1  # abs tol 2
-0.5
sage: print("0.9999")    # rel tol 1e-4
1.0
sage: print("1.00001")   # abs tol 1e-5
1.0
sage: 0  # rel tol 1
1
```

```
>>> from sage.all import *
>>> _ = x  # rel tol 1e10                                                      #␣
↪needs sage.symbolic
>>> raise RuntimeError   # rel tol 1e10
Traceback (most recent call last):
```

(continues on next page)

(continued from previous page)

```
...
RuntimeError
>>> Integer(1)  # abs tol 2
-0.5
>>> print("0.9999")    # rel tol 1e-4
1.0
>>> print("1.00001")   # abs tol 1e-5
1.0
>>> Integer(0)  # rel tol 1
1
```

Abs tol checks over the complex domain:

```
sage: [1, -1.3, -1.5 + 0.1*I, 0.5 - 0.1*I, -1.5*I]  # abs tol 1.0
[1, -1, -1, 1, -I]
```

```
>>> from sage.all import *
>>> [Integer(1), -RealNumber('1.3'), -RealNumber('1.5') + RealNumber('0.1')*I,
↪ RealNumber('0.5') - RealNumber('0.1')*I, -RealNumber('1.5')*I]  # abs tol␣
↪1.0
[1, -1, -1, 1, -I]
```

Spaces before numbers or between the sign and number are ignored:

```
sage: print("[ - 1, 2]")  # abs tol 1e-10
[-1,2]
```

```
>>> from sage.all import *
>>> print("[ - 1, 2]")  # abs tol 1e-10
[-1,2]
```

Tolerance on Python 3 for string results with unicode prefix:

```
sage: a = 'Cyrano'; a
'Cyrano'
sage: b = ['Fermat', 'Euler']; b
['Fermat',  'Euler']
sage: c = 'you'; c
'you'
```

```
>>> from sage.all import *
>>> a = 'Cyrano'; a
'Cyrano'
>>> b = ['Fermat', 'Euler']; b
['Fermat',  'Euler']
>>> c = 'you'; c
'you'
```

This illustrates that Issue #33588 is fixed:

```
sage: from sage.doctest.parsing import SageOutputChecker, SageDocTestParser
sage: import doctest
```

(continues on next page)

(continued from previous page)

```
sage: optflag = doctest.NORMALIZE_WHITESPACE|doctest.ELLIPSIS
sage: DTP = SageDocTestParser(('sage','magma','guava'))
sage: OC = SageOutputChecker()
sage: example = "sage: 1.3090169943749475 # tol 1e-8\n1.3090169943749475"
sage: ex = DTP.parse(example)[1]
sage: OC.check_output(ex.want, '1.3090169943749475', optflag)
True
sage: OC.check_output(ex.want, 'ANYTHING1.3090169943749475', optflag)
False
sage: OC.check_output(ex.want, 'Long-step dual simplex will be used\n1.
↪3090169943749475', optflag)
True
```

```
>>> from sage.all import *
>>> from sage.doctest.parsing import SageOutputChecker, SageDocTestParser
>>> import doctest
>>> optflag = doctest.NORMALIZE_WHITESPACE|doctest.ELLIPSIS
>>> DTP = SageDocTestParser(('sage','magma','guava'))
>>> OC = SageOutputChecker()
>>> example = "sage: 1.3090169943749475 # tol 1e-8\n1.3090169943749475"
>>> ex = DTP.parse(example)[Integer(1)]
>>> OC.check_output(ex.want, '1.3090169943749475', optflag)
True
>>> OC.check_output(ex.want, 'ANYTHING1.3090169943749475', optflag)
False
>>> OC.check_output(ex.want, 'Long-step dual simplex will be used\n1.
↪3090169943749475', optflag)
True
```

do_fixup(*want*, *got*)

Perform few changes to the strings `want` and `got`.

For example, remove warnings to be ignored.

INPUT:

- `want` – string or *MarkedOutput*
- `got` – string

OUTPUT: a tuple:

- boolean, `True` when some fixup were performed and `False` otherwise
- string, edited wanted string
- string, edited got string

Note

Currently, the code only possibly changes the string `got` while keeping `want` invariant. We keep open the possibility of adding a regular expression which would also change the `want` string. This is why `want` is an input and an output of the method even if currently kept invariant.

EXAMPLES:

```
sage: from sage.doctest.parsing import SageOutputChecker
sage: OC = SageOutputChecker()
sage: OC.do_fixup('1.3090169943749475','1.3090169943749475')
(False, '1.3090169943749475', '1.3090169943749475')
sage: OC.do_fixup('1.3090169943749475','ANYTHING1.3090169943749475')
(False, '1.3090169943749475', 'ANYTHING1.3090169943749475')
sage: OC.do_fixup('1.3090169943749475','Long-step dual simplex will be used\
↪\n1.3090169943749475')
(True, '1.3090169943749475', '\n1.3090169943749475')
```

```
>>> from sage.all import *
>>> from sage.doctest.parsing import SageOutputChecker
>>> OC = SageOutputChecker()
>>> OC.do_fixup('1.3090169943749475','1.3090169943749475')
(False, '1.3090169943749475', '1.3090169943749475')
>>> OC.do_fixup('1.3090169943749475','ANYTHING1.3090169943749475')
(False, '1.3090169943749475', 'ANYTHING1.3090169943749475')
>>> OC.do_fixup('1.3090169943749475','Long-step dual simplex will be used\n1.
↪3090169943749475')
(True, '1.3090169943749475', '\n1.3090169943749475')
```

When `want` is an instance of class *MarkedOutput*:

```
sage: from sage.doctest.parsing import SageOutputChecker, SageDocTestParser
sage: import doctest
sage: optflag = doctest.NORMALIZE_WHITESPACE|doctest.ELLIPSIS
sage: DTP = SageDocTestParser(('sage','magma','guava'))
sage: OC = SageOutputChecker()
sage: example = "sage: 1.3090169943749475\n1.3090169943749475"
sage: ex = DTP.parse(example)[1]
sage: ex.want
'1.3090169943749475\n'
sage: OC.do_fixup(ex.want,'1.3090169943749475')
(False, '1.3090169943749475\n', '1.3090169943749475')
sage: OC.do_fixup(ex.want,'ANYTHING1.3090169943749475')
(False, '1.3090169943749475\n', 'ANYTHING1.3090169943749475')
sage: OC.do_fixup(ex.want,'Long-step dual simplex will be used\n1.
↪3090169943749475')
(True, '1.3090169943749475\n', '\n1.3090169943749475')
```

```
>>> from sage.all import *
>>> from sage.doctest.parsing import SageOutputChecker, SageDocTestParser
>>> import doctest
>>> optflag = doctest.NORMALIZE_WHITESPACE|doctest.ELLIPSIS
>>> DTP = SageDocTestParser(('sage','magma','guava'))
>>> OC = SageOutputChecker()
>>> example = "sage: 1.3090169943749475\n1.3090169943749475"
>>> ex = DTP.parse(example)[Integer(1)]
>>> ex.want
'1.3090169943749475\n'
>>> OC.do_fixup(ex.want,'1.3090169943749475')
(False, '1.3090169943749475\n', '1.3090169943749475')
```

(continues on next page)

(continued from previous page)

```
>>> OC.do_fixup(ex.want,'ANYTHING1.3090169943749475')
(False, '1.3090169943749475\n', 'ANYTHING1.3090169943749475')
>>> OC.do_fixup(ex.want,'Long-step dual simplex will be used\n1.
↪3090169943749475')
(True, '1.3090169943749475\n', '\n1.3090169943749475')
```

human_readable_escape_sequences(*string*)

Make ANSI escape sequences human readable.

EXAMPLES:

```
sage: print('This is \x1b[1mbold\x1b[0m text')
This is <CSI-1m>bold<CSI-0m> text
```

```
>>> from sage.all import *
>>> print('This is \x1b[1mbold\x1b[0m text')
This is <CSI-1m>bold<CSI-0m> text
```

output_difference(*example*, *got*, *optionflags*)

Report on the differences between the desired result and what was actually obtained.

If `want` is a *MarkedOutput* instance, takes into account the desired tolerance.

INPUT:

- `example` – a `doctest.Example` instance
- `got` – string
- `optionflags` – integer; passed down to `doctest.OutputChecker`

OUTPUT: string, describing how `got` fails to match `example.want`

EXAMPLES:

```
sage: from sage.doctest.parsing import MarkedOutput, SageOutputChecker
sage: import doctest
sage: optflag = doctest.NORMALIZE_WHITESPACE|doctest.ELLIPSIS
sage: tentol = doctest.Example('',MarkedOutput("10.0\n").update(tol=.1))
sage: tenabs = doctest.Example('',MarkedOutput("10.0\n").update(abs_tol=.1))
sage: tenrel = doctest.Example('',MarkedOutput("10.0\n").update(rel_tol=.1))
sage: zerotol = doctest.Example('',MarkedOutput("0.0\n").update(tol=.1))
sage: zeroabs = doctest.Example('',MarkedOutput("0.0\n").update(abs_tol=.1))
sage: zerorel = doctest.Example('',MarkedOutput("0.0\n").update(rel_tol=.1))
sage: tlist = doctest.Example('',MarkedOutput("[10.0, 10.0, 10.0, 10.0, 10.0,␣
↪10.0]\n").update(abs_tol=0.987))
sage: zero = "0.0"
sage: nf = "9.5"
sage: ten = "10.05"
sage: eps = "-0.05"
sage: L = "[9.9, 8.7, 10.3, 11.2, 10.8, 10.0]"
sage: OC = SageOutputChecker()
```

```
>>> from sage.all import *
>>> from sage.doctest.parsing import MarkedOutput, SageOutputChecker
```

(continues on next page)

(continued from previous page)

```
>>> import doctest
>>> optflag = doctest.NORMALIZE_WHITESPACE|doctest.ELLIPSIS
>>> tentol = doctest.Example('',MarkedOutput("10.0\n").update(tol=RealNumber(
↪'.1')))
>>> tenabs = doctest.Example('',MarkedOutput("10.0\n").update(abs_
↪tol=RealNumber('.1')))
>>> tenrel = doctest.Example('',MarkedOutput("10.0\n").update(rel_
↪tol=RealNumber('.1')))
>>> zerotol = doctest.Example('',MarkedOutput("0.0\n").update(tol=RealNumber(
↪'.1')))
>>> zeroabs = doctest.Example('',MarkedOutput("0.0\n").update(abs_
↪tol=RealNumber('.1')))
>>> zerorel = doctest.Example('',MarkedOutput("0.0\n").update(rel_
↪tol=RealNumber('.1')))
>>> tlist = doctest.Example('',MarkedOutput("[10.0, 10.0, 10.0, 10.0, 10.0,␣
↪10.0]\n").update(abs_tol=RealNumber('0.987')))
>>> zero = "0.0"
>>> nf = "9.5"
>>> ten = "10.05"
>>> eps = "-0.05"
>>> L = "[9.9, 8.7, 10.3, 11.2, 10.8, 10.0]"
>>> OC = SageOutputChecker()
```

```
sage: print(OC.output_difference(tenabs,nf,optflag))
Expected:
    10.0
Got:
    9.5
Tolerance exceeded:
    10.0 vs 9.5, tolerance 5e-1 > 1e-1

sage: print(OC.output_difference(tentol,zero,optflag))
Expected:
    10.0
Got:
    0.0
Tolerance exceeded:
    10.0 vs 0.0, tolerance 1e0 > 1e-1

sage: print(OC.output_difference(tentol,eps,optflag))
Expected:
    10.0
Got:
    -0.05
Tolerance exceeded:
    10.0 vs -0.05, tolerance 2e0 > 1e-1

sage: print(OC.output_difference(tlist,L,optflag))
Expected:
    [10.0, 10.0, 10.0, 10.0, 10.0, 10.0]
Got:
    [9.9, 8.7, 10.3, 11.2, 10.8, 10.0]
```

(continues on next page)

(continued from previous page)

```
Tolerance exceeded in 2 of 6:
    10.0 vs 8.7, tolerance 2e0 > 9.87e-1
    10.0 vs 11.2, tolerance 2e0 > 9.87e-1
```

```
>>> from sage.all import *
>>> print(OC.output_difference(tenabs,nf,optflag))
Expected:
    10.0
Got:
    9.5
Tolerance exceeded:
    10.0 vs 9.5, tolerance 5e-1 > 1e-1

>>> print(OC.output_difference(tentol,zero,optflag))
Expected:
    10.0
Got:
    0.0
Tolerance exceeded:
    10.0 vs 0.0, tolerance 1e0 > 1e-1

>>> print(OC.output_difference(tentol,eps,optflag))
Expected:
    10.0
Got:
    -0.05
Tolerance exceeded:
    10.0 vs -0.05, tolerance 2e0 > 1e-1

>>> print(OC.output_difference(tlist,L,optflag))
Expected:
    [10.0, 10.0, 10.0, 10.0, 10.0, 10.0]
Got:
    [9.9, 8.7, 10.3, 11.2, 10.8, 10.0]
Tolerance exceeded in 2 of 6:
    10.0 vs 8.7, tolerance 2e0 > 9.87e-1
    10.0 vs 11.2, tolerance 2e0 > 9.87e-1
```

`sage.doctest.parsing.`**`get_source`**(*example*)

Return the source with the leading 'sage: ' stripped off.

EXAMPLES:

```
sage: from sage.doctest.parsing import get_source
sage: from sage.doctest.sources import DictAsObject
sage: example = DictAsObject({})
sage: example.sage_source = "2 + 2"
sage: example.source = "sage: 2 + 2"
sage: get_source(example)
'2 + 2'
sage: example = DictAsObject({})
sage: example.source = "3 + 3"
```

(continues on next page)

(continued from previous page)

```
sage: get_source(example)
'3 + 3'
```

```
>>> from sage.all import *
>>> from sage.doctest.parsing import get_source
>>> from sage.doctest.sources import DictAsObject
>>> example = DictAsObject({})
>>> example.sage_source = "2 + 2"
>>> example.source = "sage: 2 + 2"
>>> get_source(example)
'2 + 2'
>>> example = DictAsObject({})
>>> example.source = "3 + 3"
>>> get_source(example)
'3 + 3'
```

sage.doctest.parsing.**parse_file_optional_tags**(*lines*)

Scan the first few lines for file-level doctest directives.

INPUT:

- `lines` – iterable of pairs `(lineno, line)`

OUTPUT: dictionary whose keys are strings (tags); see *parse_optional_tags()*

EXAMPLES:

```
sage: from sage.doctest.parsing import parse_file_optional_tags
sage: filename = tmp_filename(ext='.pyx')
sage: with open(filename, "r") as f:
....:     parse_file_optional_tags(enumerate(f))
{}
sage: with open(filename, "w") as f:
....:     _ = f.write("# nodoctest")
sage: with open(filename, "r") as f:
....:     parse_file_optional_tags(enumerate(f))
{'not tested': None}
sage: with open(filename, "w") as f:
....:     _ = f.write("# sage.doctest: "    # broken in two source lines to avoid␣
↪the pattern
....:                 "optional - xyz")     # of relint (multiline_doctest_
↪comment)
sage: with open(filename, "r") as f:
....:     parse_file_optional_tags(enumerate(f))
{'xyz': None}
```

```
>>> from sage.all import *
>>> from sage.doctest.parsing import parse_file_optional_tags
>>> filename = tmp_filename(ext='.pyx')
>>> with open(filename, "r") as f:
...     parse_file_optional_tags(enumerate(f))
{}
>>> with open(filename, "w") as f:
```

(continues on next page)

(continued from previous page)

```
...       _ = f.write("# nodoctest")
>>> with open(filename, "r") as f:
...       parse_file_optional_tags(enumerate(f))
{'not tested': None}
>>> with open(filename, "w") as f:
...       _ = f.write("# sage.doctest: "     # broken in two source lines to avoid␣
↪the pattern
...                   "optional - xyz")      # of relint (multiline_doctest_comment)
>>> with open(filename, "r") as f:
...       parse_file_optional_tags(enumerate(f))
{'xyz': None}
```

sage.doctest.parsing.**parse_optional_tags**(*string: str*) → dict[str, str | None]

sage.doctest.parsing.**parse_optional_tags**(*string: str*, *, *return_string_sans_tags: Literal[True]*) → tuple[dict[str, str | None], str, bool]

Return a dictionary whose keys are optional tags from the following set that occur in a comment on the first line of the input string.

- `'long time'`
- `'not implemented'`
- `'not tested'`
- `'known bug'` (possible values are `None`, `linux` and `macos`)
- `'py2'`
- `'optional -- FEATURE...'` or `'needs FEATURE...'` – the dictionary will just have the key `'FEATURE'`

The values, if non-`None`, are strings with optional explanations for a tag, which may appear in parentheses after the tag in `string`.

INPUT:

- `string` – string
- `return_string_sans_tags` – boolean (default: `False`); whether to additionally return `string` with the optional tags removed but other comments kept and a boolean `is_persistent`

EXAMPLES:

```
sage: from sage.doctest.parsing import parse_optional_tags
sage: parse_optional_tags("sage: magma('2 + 2')# optional: magma")
{'magma': None}
sage: parse_optional_tags("sage: #optional -- mypkg")
{'mypkg': None}
sage: parse_optional_tags("sage: print(1)  # parentheses are optional here")
{}
sage: parse_optional_tags("sage: print(1)  # optional")
{}
sage: sorted(list(parse_optional_tags("sage: #optional -- foo bar, baz")))
['bar', 'foo']
sage: parse_optional_tags("sage: #optional -- foo.bar, baz")
{'foo.bar': None}
sage: parse_optional_tags("sage: #needs foo.bar, baz")
```

(continues on next page)

(continued from previous page)

```
{'foo.bar': None}
sage: sorted(list(parse_optional_tags("    sage: factor(10^(10^10) + 1) # LoNg␣
↪TiME, NoT TeSTED; OptioNAL -- P4cka9e")))
['long time', 'not tested', 'p4cka9e']
sage: parse_optional_tags("    sage: raise RuntimeError # known bug")
{'bug': None}
sage: sorted(list(parse_optional_tags("    sage: determine_meaning_of_life() #␣
↪long time, not implemented")))
['long time', 'not implemented']
```

```
>>> from sage.all import *
>>> from sage.doctest.parsing import parse_optional_tags
>>> parse_optional_tags("sage: magma('2 + 2')# optional: magma")
{'magma': None}
>>> parse_optional_tags("sage: #optional -- mypkg")
{'mypkg': None}
>>> parse_optional_tags("sage: print(1)  # parentheses are optional here")
{}
>>> parse_optional_tags("sage: print(1)  # optional")
{}
>>> sorted(list(parse_optional_tags("sage: #optional -- foo bar, baz")))
['bar', 'foo']
>>> parse_optional_tags("sage: #optional -- foo.bar, baz")
{'foo.bar': None}
>>> parse_optional_tags("sage: #needs foo.bar, baz")
{'foo.bar': None}
>>> sorted(list(parse_optional_tags("    sage: factor(10^(10^10) + 1) # LoNg TiME,
↪ NoT TeSTED; OptioNAL -- P4cka9e")))
['long time', 'not tested', 'p4cka9e']
>>> parse_optional_tags("    sage: raise RuntimeError # known bug")
{'bug': None}
>>> sorted(list(parse_optional_tags("    sage: determine_meaning_of_life() # long␣
↪time, not implemented")))
['long time', 'not implemented']
```

We don't parse inside strings:

```
sage: parse_optional_tags("    sage: print('  # long time')")
{}
sage: parse_optional_tags("    sage: print('  # long time')  # not tested")
{'not tested': None}
```

```
>>> from sage.all import *
>>> parse_optional_tags("    sage: print('  # long time')")
{}
>>> parse_optional_tags("    sage: print('  # long time')  # not tested")
{'not tested': None}
```

UTF-8 works:

```
sage: parse_optional_tags("'ěščřžýáíédĎ'")
{}
```

```
>>> from sage.all import *
>>> parse_optional_tags("'ěščřžýáíéďŤ'")
{}
```

Tags with parenthesized explanations:

```
sage: parse_optional_tags("    sage: 1 + 1  # long time (1 year, 2 months??),␣
↪optional - bliss (because)")
{'bliss': 'because', 'long time': '1 year, 2 months??'}
```

```
>>> from sage.all import *
>>> parse_optional_tags("    sage: 1 + 1  # long time (1 year, 2 months??),␣
↪optional - bliss (because)")
{'bliss': 'because', 'long time': '1 year, 2 months??'}
```

With `return_string_sans_tags=True`:

```
sage: parse_optional_tags("sage: print(1)  # very important 1  # optional - foo",
....:                     return_string_sans_tags=True)
({'foo': None}, 'sage: print(1)  # very important 1  ', False)
sage: parse_optional_tags("sage: print(    # very important too  # optional - foo\
↪n....:     2)",
....:                     return_string_sans_tags=True)
({'foo': None}, 'sage: print(    # very important too  \n....:     2)', False)
sage: parse_optional_tags("sage: #this is persistent #needs scipy",
....:                     return_string_sans_tags=True)
({'scipy': None}, 'sage: #this is persistent ', True)
sage: parse_optional_tags("sage: #this is not #needs scipy\n....: import scipy",
....:                     return_string_sans_tags=True)
({'scipy': None}, 'sage: #this is not \n....: import scipy', False)
```

```
>>> from sage.all import *
>>> parse_optional_tags("sage: print(1)  # very important 1  # optional - foo",
...                     return_string_sans_tags=True)
({'foo': None}, 'sage: print(1)  # very important 1  ', False)
>>> parse_optional_tags("sage: print(    # very important too  # optional - foo\n.
↪...:     2)",
...                     return_string_sans_tags=True)
({'foo': None}, 'sage: print(    # very important too  \n....:     2)', False)
>>> parse_optional_tags("sage: #this is persistent #needs scipy",
...                     return_string_sans_tags=True)
({'scipy': None}, 'sage: #this is persistent ', True)
>>> parse_optional_tags("sage: #this is not #needs scipy\n....: import scipy",
...                     return_string_sans_tags=True)
({'scipy': None}, 'sage: #this is not \n....: import scipy', False)
```

`sage.doctest.parsing.`**`parse_tolerance`**(*source*, *want*)

Return a version of `want` marked up with the tolerance tags specified in `source`.

INPUT:

- `source` – string, the source of a doctest
- `want` – string, the desired output of the doctest

OUTPUT: `want` if there are no tolerance tags specified; a *MarkedOutput* version otherwise

EXAMPLES:

```
sage: from sage.doctest.parsing import parse_tolerance
sage: marked = parse_tolerance("sage: s.update(abs_tol = .0000001)", "")
sage: type(marked)
<class 'str'>
sage: marked = parse_tolerance("sage: s.update(tol = 0.1); s.rel_tol # abs tol  ␣
↪ 0.01 ", "")
sage: marked.tol
0
sage: marked.rel_tol
0
sage: marked.abs_tol
0.010000000000000000000...?
```

```
>>> from sage.all import *
>>> from sage.doctest.parsing import parse_tolerance
>>> marked = parse_tolerance("sage: s.update(abs_tol = .0000001)", "")
>>> type(marked)
<class 'str'>
>>> marked = parse_tolerance("sage: s.update(tol = 0.1); s.rel_tol # abs tol  ␣
↪0.01 ", "")
>>> marked.tol
0
>>> marked.rel_tol
0
>>> marked.abs_tol
0.010000000000000000000...?
```

sage.doctest.parsing.**pre_hash**(*s*)

Prepends a string with its length.

EXAMPLES:

```
sage: from sage.doctest.parsing import pre_hash
sage: pre_hash("abc")
'3:abc'
```

```
>>> from sage.all import *
>>> from sage.doctest.parsing import pre_hash
>>> pre_hash("abc")
'3:abc'
```

sage.doctest.parsing.**reduce_hex**(*fingerprints*)

Return a symmetric function of the arguments as hex strings.

The arguments should be 32 character strings consisting of hex digits: 0-9 and a-f.

EXAMPLES:

```
sage: from sage.doctest.parsing import reduce_hex
sage: reduce_hex(["abc", "12399aedf"])
'00000000000000000000000012399a463'
```

(continues on next page)

(continued from previous page)

```
sage: reduce_hex(["12399aedf","abc"])
'00000000000000000000000012399a463'
```

```
>>> from sage.all import *
>>> from sage.doctest.parsing import reduce_hex
>>> reduce_hex(["abc", "12399aedf"])
'00000000000000000000000012399a463'
>>> reduce_hex(["12399aedf","abc"])
'00000000000000000000000012399a463'
```

sage.doctest.parsing.**unparse_optional_tags**(*tags*, *prefix='# '*)

Return a comment string that sets `tags`.

INPUT:

- `tags` – dictionary or iterable of tags, as output by *parse_optional_tags()*
- `prefix` – to be put before a nonempty string

EXAMPLES:

```
sage: from sage.doctest.parsing import unparse_optional_tags
sage: unparse_optional_tags({})
''
sage: unparse_optional_tags({'magma': None})
'# optional - magma'
sage: unparse_optional_tags({'fictional_optional': None,
....:                        'sage.rings.number_field': None,
....:                        'scipy': 'just because',
....:                        'bliss': None})
'# optional - bliss fictional_optional, needs scipy (just because) sage.rings.
↪number_field'
sage: unparse_optional_tags(['long time', 'not tested', 'p4cka9e'], prefix='')
'long time, not tested, optional - p4cka9e'
```

```
>>> from sage.all import *
>>> from sage.doctest.parsing import unparse_optional_tags
>>> unparse_optional_tags({})
''
>>> unparse_optional_tags({'magma': None})
'# optional - magma'
>>> unparse_optional_tags({'fictional_optional': None,
...                        'sage.rings.number_field': None,
...                        'scipy': 'just because',
...                        'bliss': None})
'# optional - bliss fictional_optional, needs scipy (just because) sage.rings.
↪number_field'
>>> unparse_optional_tags(['long time', 'not tested', 'p4cka9e'], prefix='')
'long time, not tested, optional - p4cka9e'
```

sage.doctest.parsing.**update_optional_tags**(*line*, *tags*, *add_tags*, *remove_tags*, *force_rewrite=None*)

Return the doctest `line` with tags changed.

EXAMPLES:

```
sage: from sage.doctest.parsing import update_optional_tags, optional_tag_columns,
↪ standard_tag_columns
sage: ruler = ''
sage: for column in optional_tag_columns:
....:     ruler += ' ' * (column - len(ruler)) + 'V'
sage: for column in standard_tag_columns:
....:     ruler += ' ' * (column - len(ruler)) + 'v'
sage: def print_with_ruler(lines):
....:     print('|' + ruler)
....:     for line in lines:
....:         print('|' + line)
sage: print_with_ruler([  # the tags are obscured in the source file to avoid␣
↪relint warnings
....:     update_optional_tags('    sage: something()  # opt' 'ional - latte_int',
....:                          remove_tags=['latte_int', 'wasnt_even_there']),
....:     update_optional_tags('    sage: nothing_to_be_seen_here()',
....:                          tags=['scipy', 'long time']),
....:     update_optional_tags('    sage: nothing_to_be_seen_here(honestly=True)',
....:                          add_tags=['scipy', 'long time']),
....:     update_optional_tags('    sage: nothing_to_be_seen_here(honestly=True,␣
↪very=True)',
....:                          add_tags=['scipy', 'long time']),
....:     update_optional_tags('    sage: no_there_is_absolutely_nothing_to_be_
↪seen_here_i_am_serious()#opt' 'ional:bliss',
....:                          add_tags=['scipy', 'long time']),
....:     update_optional_tags('    sage: ntbsh()  # abbrv for above#opt'
↪'ional:bliss',
....:                          add_tags={'scipy': None, 'long time': '30s on the␣
↪highest setting'}),
....:     update_optional_tags('    sage: no_there_is_absolutely_nothing_to_be_
↪seen_here_i_am_serious()  # really, you can trust me here',
....:                          add_tags=['scipy']),
....: ])
|                                                V       V       V       V      ␣
↪V   V   v           v                   v                                      ␣
↪v
|     sage: something()
|     sage: nothing_to_be_seen_here()             # long time                    ␣
↪       # needs scipy
|     sage: nothing_to_be_seen_here(honestly=True)        # long time            ␣
↪       # needs scipy
|     sage: nothing_to_be_seen_here(honestly=True, very=True)     # long time     ␣
↪       # needs scipy
|     sage: no_there_is_absolutely_nothing_to_be_seen_here_i_am_serious()
↪# long time, optional - bliss, needs scipy
|     sage: ntbsh()  # abbrv for above            # long time (30s on the highest␣
↪setting), optional - bliss, needs scipy
|     sage: no_there_is_absolutely_nothing_to_be_seen_here_i_am_serious()  #␣
↪really, you can trust me here                # needs scipy
```

```
>>> from sage.all import *
>>> from sage.doctest.parsing import update_optional_tags, optional_tag_columns,␣
```

(continues on next page)

(continued from previous page)

```
↪standard_tag_columns
>>> ruler = ''
>>> for column in optional_tag_columns:
...     ruler += ' ' * (column - len(ruler)) + 'V'
>>> for column in standard_tag_columns:
...     ruler += ' ' * (column - len(ruler)) + 'v'
>>> def print_with_ruler(lines):
...     print('|' + ruler)
...     for line in lines:
...         print('|' + line)
>>> print_with_ruler([  # the tags are obscured in the source file to avoid␣
↪relint warnings
...     update_optional_tags('    sage: something()  # opt' 'ional - latte_int',
...                          remove_tags=['latte_int', 'wasnt_even_there']),
...     update_optional_tags('    sage: nothing_to_be_seen_here()',
...                          tags=['scipy', 'long time']),
...     update_optional_tags('    sage: nothing_to_be_seen_here(honestly=True)',
...                          add_tags=['scipy', 'long time']),
...     update_optional_tags('    sage: nothing_to_be_seen_here(honestly=True,␣
↪very=True)',
...                          add_tags=['scipy', 'long time']),
...     update_optional_tags('    sage: no_there_is_absolutely_nothing_to_be_seen_
↪here_i_am_serious()#opt' 'ional:bliss',
...                          add_tags=['scipy', 'long time']),
...     update_optional_tags('    sage: ntbsh()  # abbrv for above#opt'
↪'ional:bliss',
...                          add_tags={'scipy': None, 'long time': '30s on the␣
↪highest setting'}),
...     update_optional_tags('    sage: no_there_is_absolutely_nothing_to_be_seen_
↪here_i_am_serious()  # really, you can trust me here',
...                          add_tags=['scipy']),
... ])
|                                                V       V       V       V      ␣
↪V   V   v           v                   v                                      ␣
↪v
|    sage: something()
|    sage: nothing_to_be_seen_here()             # long time                     ␣
↪       # needs scipy
|    sage: nothing_to_be_seen_here(honestly=True)        # long time             ␣
↪       # needs scipy
|    sage: nothing_to_be_seen_here(honestly=True, very=True)     # long time     ␣
↪       # needs scipy
|    sage: no_there_is_absolutely_nothing_to_be_seen_here_i_am_serious()
↪# long time, optional - bliss, needs scipy
|    sage: ntbsh()  # abbrv for above            # long time (30s on the highest␣
↪setting), optional - bliss, needs scipy
|    sage: no_there_is_absolutely_nothing_to_be_seen_here_i_am_serious()  #␣
↪really, you can trust me here                # needs scipy
```

When no tags are changed, by default, the unchanged input is returned. We can force a rewrite; unconditionally or whenever standard tags are involved. But even when forced, if comments are already aligned at one of the standard alignment columns, this alignment is kept even if we would normally realign farther to the left:

```
sage: print_with_ruler([
....:     update_optional_tags('    sage: unforced()       # opt' 'ional - latte_
↪int'),
....:     update_optional_tags('    sage: unforced()  # opt' 'ional - latte_int',
....:                          add_tags=['latte_int']),
....:     update_optional_tags('    sage: forced()#opt' 'ional- latte_int',
....:                          force_rewrite=True),
....:     update_optional_tags('    sage: forced()  # opt' 'ional - scipy',
....:                          force_rewrite='standard'),
....:     update_optional_tags('    sage: aligned_with_below()                    ␣
↪              # opt' 'ional - 4ti2',
....:                          force_rewrite=True),
....:     update_optional_tags('    sage: aligned_with_above()                    ␣
↪              # opt' 'ional - 4ti2',
....:                          force_rewrite=True),
....:     update_optional_tags('    sage: also_already_aligned()                  ␣
↪                                                                  # ne'
↪'eds scipy',
....:                          force_rewrite='standard'),
....:     update_optional_tags('    sage: two_columns_first_preserved()          #␣
↪lo' 'ng time                             # ne' 'eds scipy',
....:                          force_rewrite='standard'),
....:     update_optional_tags('    sage: two_columns_first_preserved()          ␣
↪      # lo' 'ng time                                  # ne' 'eds scipy',
....:                          force_rewrite='standard'),
....: ])
|                                                  V       V       V       V      ␣
↪V   V   v           v                   v                                        ␣
↪v
|     sage: unforced()       # optional - latte_int
|     sage: unforced()  # optional - latte_int
|     sage: forced()                                 # optional - latte_int
|     sage: forced()                                                              ␣
↪       # needs scipy
|     sage: aligned_with_below()                                    # optional - 4ti2
|     sage: aligned_with_above()                                    # optional - 4ti2
|     sage: also_already_aligned()                                                ␣
↪                                       # needs scipy
|     sage: two_columns_first_preserved()          # long time                    ␣
↪       # needs scipy
|     sage: two_columns_first_preserved()                 # long time             ␣
↪       # needs scipy
```

```
>>> from sage.all import *
>>> print_with_ruler([
...     update_optional_tags('    sage: unforced()       # opt' 'ional - latte_int
↪'),
...     update_optional_tags('    sage: unforced()  # opt' 'ional - latte_int',
...                          add_tags=['latte_int']),
...     update_optional_tags('    sage: forced()#opt' 'ional- latte_int',
...                          force_rewrite=True),
...     update_optional_tags('    sage: forced()  # opt' 'ional - scipy',
```

(continues on next page)

(continued from previous page)

```
...                           force_rewrite='standard'),
...      update_optional_tags('    sage: aligned_with_below()                    ␣
↪            # opt' 'ional - 4ti2',
...                           force_rewrite=True),
...      update_optional_tags('    sage: aligned_with_above()                    ␣
↪            # opt' 'ional - 4ti2',
...                           force_rewrite=True),
...      update_optional_tags('    sage: also_already_aligned()                  ␣
↪                                                                  # ne' 'eds␣
↪scipy',
...                           force_rewrite='standard'),
...      update_optional_tags('    sage: two_columns_first_preserved()         # lo
↪' 'ng time                              # ne' 'eds scipy',
...                           force_rewrite='standard'),
...      update_optional_tags('    sage: two_columns_first_preserved()           ␣
↪    # lo' 'ng time                                    # ne' 'eds scipy',
...                           force_rewrite='standard'),
... ])
|                                                  V       V       V       V      ␣
↪V   V   v           v                   v                                        ␣
↪v
|    sage: unforced()       # optional - latte_int
|    sage: unforced()  # optional - latte_int
|    sage: forced()                                # optional - latte_int
|    sage: forced()                                                               ␣
↪       # needs scipy
|    sage: aligned_with_below()                                    # optional - 4ti2
|    sage: aligned_with_above()                                    # optional - 4ti2
|    sage: also_already_aligned()                                                 ␣
↪                                        # needs scipy
|    sage: two_columns_first_preserved()         # long time                      ␣
↪       # needs scipy
|    sage: two_columns_first_preserved()                 # long time             ␣
↪       # needs scipy
```

Rewriting a persistent (block-scoped) tag:

```
sage: print_with_ruler([
....:     update_optional_tags('    sage:    #opt' 'ional:magma sage.symbolic',
....:                          force_rewrite=True),
....: ])
|                                                  V       V       V       V      ␣
↪V   V   v           v                   v                                        ␣
↪v
|    sage: # optional - magma, needs sage.symbolic
```

```
>>> from sage.all import *
>>> print_with_ruler([
...     update_optional_tags('    sage:    #opt' 'ional:magma sage.symbolic',
...                          force_rewrite=True),
... ])
|                                                  V       V       V       V      ␣
```

(continues on next page)

(continued from previous page)

```
↪V   V   v           v                   v                                    ␣
↪v
|    sage: # optional - magma, needs sage.symbolic
```

CHAPTER

FIVE

HELPER FOR ATTACHING TOLERANCE INFORMATION TO STRINGS

class sage.doctest.marked_output.**MarkedOutput**

Bases: str

A subclass of string with context for whether another string matches it.

EXAMPLES:

```
sage: from sage.doctest.marked_output import MarkedOutput
sage: s = MarkedOutput("abc")
sage: s.rel_tol
0
sage: s.update(rel_tol = .05)
'abc'
sage: s.rel_tol
0.0500000000000000

sage: MarkedOutput("56 µs")
'56 µs'
```

```
>>> from sage.all import *
>>> from sage.doctest.marked_output import MarkedOutput
>>> s = MarkedOutput("abc")
>>> s.rel_tol
0
>>> s.update(rel_tol = RealNumber('.05'))
'abc'
>>> s.rel_tol
0.0500000000000000

>>> MarkedOutput("56 µs")
'56 µs'
```

abs_tol = 0

random = False

rel_tol = 0

tol = 0

update(***kwds*)

EXAMPLES:

```
sage: from sage.doctest.marked_output import MarkedOutput
sage: s = MarkedOutput("0.0007401")
sage: s.update(abs_tol = .0000001)
'0.0007401'
sage: s.rel_tol
0
sage: s.abs_tol
1.00000000000000e-7
```

```
>>> from sage.all import *
>>> from sage.doctest.marked_output import MarkedOutput
>>> s = MarkedOutput("0.0007401")
>>> s.update(abs_tol = RealNumber('.0000001'))
'0.0007401'
>>> s.rel_tol
0
>>> s.abs_tol
1.00000000000000e-7
```

sage.doctest.marked_output.**make_marked_output**(*s*, *D*)

Auxiliary function for pickling.

EXAMPLES:

```
sage: from sage.doctest.marked_output import make_marked_output
sage: s = make_marked_output("0.0007401", {'abs_tol':.0000001})
sage: s
'0.0007401'
sage: s.abs_tol
1.00000000000000e-7
```

```
>>> from sage.all import *
>>> from sage.doctest.marked_output import make_marked_output
>>> s = make_marked_output("0.0007401", {'abs_tol':RealNumber('.0000001')})
>>> s
'0.0007401'
>>> s.abs_tol
1.00000000000000e-7
```

CHAPTER

SIX

REPORTING DOCTEST RESULTS

This module determines how doctest results are reported to the user.

It also computes the exit status in the `error_status` attribute of *DocTestReporter*. This is a bitwise OR of the following bits:

- 1: Doctest failure
- 2: Bad command line syntax or invalid options
- 4: Test timed out
- 8: Test exited with nonzero status
- 16: Test crashed with a signal (e.g. segmentation fault)
- 32: TAB character found
- 64: Internal error in the doctesting framework
- 128: Testing interrupted, not all tests run
- 256: Doctest contains explicit source line number

AUTHORS:

- David Roe (2012-03-27) – initial version, based on Robert Bradshaw's code.

class sage.doctest.reporting.**DocTestReporter**(*controller*)

Bases: SageObject

This class reports to the users on the results of doctests.

finalize()

Print out the postscript that summarizes the doctests that were run.

EXAMPLES:

First we have to set up a bunch of stuff:

```
sage: from sage.doctest.reporting import DocTestReporter
sage: from sage.doctest.control import DocTestController, DocTestDefaults
sage: from sage.doctest.sources import FileDocTestSource, DictAsObject
sage: from sage.doctest.forker import SageDocTestRunner
sage: from sage.doctest.parsing import SageOutputChecker
sage: from sage.doctest.util import Timer
sage: import doctest
sage: filename = sage.doctest.reporting.__file__
sage: DD = DocTestDefaults()
```

(continues on next page)

(continued from previous page)

```
sage: FDS = FileDocTestSource(filename, DD)
sage: DC = DocTestController(DD, [filename])
sage: DTR = DocTestReporter(DC)
```

```
>>> from sage.all import *
>>> from sage.doctest.reporting import DocTestReporter
>>> from sage.doctest.control import DocTestController, DocTestDefaults
>>> from sage.doctest.sources import FileDocTestSource, DictAsObject
>>> from sage.doctest.forker import SageDocTestRunner
>>> from sage.doctest.parsing import SageOutputChecker
>>> from sage.doctest.util import Timer
>>> import doctest
>>> filename = sage.doctest.reporting.__file__
>>> DD = DocTestDefaults()
>>> FDS = FileDocTestSource(filename, DD)
>>> DC = DocTestController(DD, [filename])
>>> DTR = DocTestReporter(DC)
```

Now we pretend to run some doctests:

```
sage: DTR.report(FDS, True, 0, None, "Output so far...", pid=1234)
    Timed out
**********************************************************************
Tests run before process (pid=1234) timed out:
Output so far...
**********************************************************************
sage: DTR.report(FDS, False, 3, None, "Output before bad exit")
    Bad exit: 3
**********************************************************************
Tests run before process failed:
Output before bad exit
**********************************************************************
sage: doctests, extras = FDS.create_doctests(globals())
sage: runner = SageDocTestRunner(
....:     SageOutputChecker(), verbose=False, sage_options=DD,
....:     optionflags=doctest.NORMALIZE_WHITESPACE|doctest.ELLIPSIS)
sage: t = Timer().start().stop()
sage: t.annotate(runner)
sage: DC.timer = t
sage: D = DictAsObject({'err':None})
sage: runner.update_results(D)
0
sage: DTR.report(FDS, False, 0, (sum([len(t.examples) for t in doctests]), D),
....:            "Good tests")
    [... tests, ...s wall]
sage: runner.failures = 1
sage: runner.update_results(D)
1
sage: DTR.report(FDS, False, 0, (sum([len(t.examples) for t in doctests]), D),
....:            "Doctest output including the failure...")
    [... tests, 1 failure, ...s wall]
```

```
>>> from sage.all import *
>>> DTR.report(FDS, True, Integer(0), None, "Output so far...",␣
↪pid=Integer(1234))
    Timed out
**********************************************************************
Tests run before process (pid=1234) timed out:
Output so far...
**********************************************************************
>>> DTR.report(FDS, False, Integer(3), None, "Output before bad exit")
    Bad exit: 3
**********************************************************************
Tests run before process failed:
Output before bad exit
**********************************************************************
>>> doctests, extras = FDS.create_doctests(globals())
>>> runner = SageDocTestRunner(
...     SageOutputChecker(), verbose=False, sage_options=DD,
...     optionflags=doctest.NORMALIZE_WHITESPACE|doctest.ELLIPSIS)
>>> t = Timer().start().stop()
>>> t.annotate(runner)
>>> DC.timer = t
>>> D = DictAsObject({'err':None})
>>> runner.update_results(D)
0
>>> DTR.report(FDS, False, Integer(0), (sum([len(t.examples) for t in␣
↪doctests]), D),
...            "Good tests")
    [... tests, ...s wall]
>>> runner.failures = Integer(1)
>>> runner.update_results(D)
1
>>> DTR.report(FDS, False, Integer(0), (sum([len(t.examples) for t in␣
↪doctests]), D),
...            "Doctest output including the failure...")
    [... tests, 1 failure, ...s wall]
```

Now we can show the output of finalize:

```
sage: DC.sources = [None] * 4 # to fool the finalize method
sage: DTR.finalize()
----------------------------------------------------------------------
.../sage/doctest/reporting.py  # Timed out
.../sage/doctest/reporting.py  # Bad exit: 3
.../sage/doctest/reporting.py  # 1 doctest failed
----------------------------------------------------------------------
Total time for all tests: 0.0 seconds
    cpu time: 0.0 seconds
    cumulative wall time: 0.0 seconds
```

```
>>> from sage.all import *
>>> DC.sources = [None] * Integer(4) # to fool the finalize method
>>> DTR.finalize()
----------------------------------------------------------------------
```

(continues on next page)

(continued from previous page)

```
.../sage/doctest/reporting.py  # Timed out
.../sage/doctest/reporting.py  # Bad exit: 3
.../sage/doctest/reporting.py  # 1 doctest failed
----------------------------------------------------------------------
Total time for all tests: 0.0 seconds
    cpu time: 0.0 seconds
    cumulative wall time: 0.0 seconds
```

If we interrupted doctests, then the number of files tested will not match the number of sources on the controller:

```
sage: DC.sources = [None] * 6
sage: DTR.finalize()

----------------------------------------------------------------------
.../sage/doctest/reporting.py  # Timed out
.../sage/doctest/reporting.py  # Bad exit: 3
.../sage/doctest/reporting.py  # 1 doctest failed
Doctests interrupted: 4/6 files tested
----------------------------------------------------------------------
Total time for all tests: 0.0 seconds
    cpu time: 0.0 seconds
    cumulative wall time: 0.0 seconds
```

```
>>> from sage.all import *
>>> DC.sources = [None] * Integer(6)
>>> DTR.finalize()
<BLANKLINE>
----------------------------------------------------------------------
.../sage/doctest/reporting.py  # Timed out
.../sage/doctest/reporting.py  # Bad exit: 3
.../sage/doctest/reporting.py  # 1 doctest failed
Doctests interrupted: 4/6 files tested
----------------------------------------------------------------------
Total time for all tests: 0.0 seconds
    cpu time: 0.0 seconds
    cumulative wall time: 0.0 seconds
```

report(*source*, *timeout*, *return_code*, *results*, *output*, *pid*, *process_tree_before_kill=None*)

Report on the result of running doctests on a given source.

This doesn't print the *report_head()*, which is assumed to be printed already.

INPUT:

- `source` – a source from *sage.doctest.sources*
- `timeout` – boolean; whether doctests timed out
- `return_code` – integer; the return code of the process running doctests on that file
- `results` – (irrelevant if `timeout` or `return_code`) a tuple
  - `ntests` – the number of doctests
  - `timings` – a *sage.doctest.sources.DictAsObject* instance storing timing data

- `output` – string; printed if there was some kind of failure
- `pid` – integer (default: `None`); the pid of the worker process

EXAMPLES:

```
sage: from sage.doctest.reporting import DocTestReporter
sage: from sage.doctest.control import DocTestController, DocTestDefaults
sage: from sage.doctest.sources import FileDocTestSource, DictAsObject
sage: from sage.doctest.forker import SageDocTestRunner
sage: from sage.doctest.parsing import SageOutputChecker
sage: from sage.doctest.util import Timer
sage: import doctest
sage: filename = sage.doctest.reporting.__file__
sage: DD = DocTestDefaults()
sage: FDS = FileDocTestSource(filename, DD)
sage: DC = DocTestController(DD, [filename])
sage: DTR = DocTestReporter(DC)
```

```
>>> from sage.all import *
>>> from sage.doctest.reporting import DocTestReporter
>>> from sage.doctest.control import DocTestController, DocTestDefaults
>>> from sage.doctest.sources import FileDocTestSource, DictAsObject
>>> from sage.doctest.forker import SageDocTestRunner
>>> from sage.doctest.parsing import SageOutputChecker
>>> from sage.doctest.util import Timer
>>> import doctest
>>> filename = sage.doctest.reporting.__file__
>>> DD = DocTestDefaults()
>>> FDS = FileDocTestSource(filename, DD)
>>> DC = DocTestController(DD, [filename])
>>> DTR = DocTestReporter(DC)
```

You can report a timeout:

```
sage: DTR.report(FDS, True, 0, None, "Output so far...", pid=1234)
    Timed out
**********************************************************************
Tests run before process (pid=1234) timed out:
Output so far...
**********************************************************************
sage: DTR.stats
{'sage.doctest.reporting': {'failed': True,
                            'ntests': 0,
                            'walltime': 1000000.0}}
```

```
>>> from sage.all import *
>>> DTR.report(FDS, True, Integer(0), None, "Output so far...",␣
↪pid=Integer(1234))
    Timed out
**********************************************************************
Tests run before process (pid=1234) timed out:
Output so far...
**********************************************************************
```

(continues on next page)

(continued from previous page)

```
>>> DTR.stats
{'sage.doctest.reporting': {'failed': True,
                            'ntests': 0,
                            'walltime': 1000000.0}}
```

Or a process that returned a bad exit code:

```
sage: DTR.report(FDS, False, 3, None, "Output before trouble")
    Bad exit: 3
**********************************************************************
Tests run before process failed:
Output before trouble
**********************************************************************
sage: DTR.stats
{'sage.doctest.reporting': {'failed': True,
                            'ntests': 0,
                            'walltime': 1000000.0}}
```

```
>>> from sage.all import *
>>> DTR.report(FDS, False, Integer(3), None, "Output before trouble")
    Bad exit: 3
**********************************************************************
Tests run before process failed:
Output before trouble
**********************************************************************
>>> DTR.stats
{'sage.doctest.reporting': {'failed': True,
                            'ntests': 0,
                            'walltime': 1000000.0}}
```

Or a process that segfaulted:

```
sage: from signal import SIGSEGV
sage: DTR.report(FDS, False, -SIGSEGV, None, "Output before trouble")
    Killed due to segmentation fault
**********************************************************************
Tests run before process failed:
Output before trouble
**********************************************************************
sage: DTR.stats
{'sage.doctest.reporting': {'failed': True,
                            'ntests': 0,
                            'walltime': 1000000.0}}
```

```
>>> from sage.all import *
>>> from signal import SIGSEGV
>>> DTR.report(FDS, False, -SIGSEGV, None, "Output before trouble")
    Killed due to segmentation fault
**********************************************************************
Tests run before process failed:
Output before trouble
**********************************************************************
```

(continues on next page)

(continued from previous page)

```
>>> DTR.stats
{'sage.doctest.reporting': {'failed': True,
                            'ntests': 0,
                            'walltime': 1000000.0}}
```

Report a timeout with results and a `SIGKILL`:

```
sage: from signal import SIGKILL
sage: DTR.report(FDS, True, -SIGKILL, (1,None), "Output before trouble")
    Timed out after testing finished (and interrupt failed)
**********************************************************************
Tests run before process timed out:
Output before trouble
**********************************************************************
sage: DTR.stats
{'sage.doctest.reporting': {'failed': True,
                            'ntests': 1,
                            'walltime': 1000000.0}}
```

```
>>> from sage.all import *
>>> from signal import SIGKILL
>>> DTR.report(FDS, True, -SIGKILL, (Integer(1),None), "Output before trouble
↪")
    Timed out after testing finished (and interrupt failed)
**********************************************************************
Tests run before process timed out:
Output before trouble
**********************************************************************
>>> DTR.stats
{'sage.doctest.reporting': {'failed': True,
                            'ntests': 1,
                            'walltime': 1000000.0}}
```

This is an internal error since results is None:

```
sage: DTR.report(FDS, False, 0, None, "All output")
    Error in doctesting framework (bad result returned)
**********************************************************************
Tests run before error:
All output
**********************************************************************
sage: DTR.stats
{'sage.doctest.reporting': {'failed': True,
                            'ntests': 1,
                            'walltime': 1000000.0}}
```

```
>>> from sage.all import *
>>> DTR.report(FDS, False, Integer(0), None, "All output")
    Error in doctesting framework (bad result returned)
**********************************************************************
Tests run before error:
All output
```

(continues on next page)

(continued from previous page)

```
**********************************************************************
>>> DTR.stats
{'sage.doctest.reporting': {'failed': True,
                            'ntests': 1,
                            'walltime': 1000000.0}}
```

Or tell the user that everything succeeded:

```
sage: doctests, extras = FDS.create_doctests(globals())
sage: runner = SageDocTestRunner(
....:     SageOutputChecker(), verbose=False, sage_options=DD,
....:     optionflags=doctest.NORMALIZE_WHITESPACE|doctest.ELLIPSIS)
sage: Timer().start().stop().annotate(runner)
sage: D = DictAsObject({'err':None})
sage: runner.update_results(D)
0
sage: DTR.report(FDS, False, 0, (sum([len(t.examples) for t in doctests]), D),
....:            "Good tests")
    [... tests, ...s wall]
sage: DTR.stats
{'sage.doctest.reporting': {'ntests': ..., 'walltime': ...}}
```

```
>>> from sage.all import *
>>> doctests, extras = FDS.create_doctests(globals())
>>> runner = SageDocTestRunner(
...     SageOutputChecker(), verbose=False, sage_options=DD,
...     optionflags=doctest.NORMALIZE_WHITESPACE|doctest.ELLIPSIS)
>>> Timer().start().stop().annotate(runner)
>>> D = DictAsObject({'err':None})
>>> runner.update_results(D)
0
>>> DTR.report(FDS, False, Integer(0), (sum([len(t.examples) for t in␣
↪doctests]), D),
...             "Good tests")
    [... tests, ...s wall]
>>> DTR.stats
{'sage.doctest.reporting': {'ntests': ..., 'walltime': ...}}
```

Or inform the user that some doctests failed:

```
sage: runner.failures = 1
sage: runner.update_results(D)
1
sage: DTR.report(FDS, False, 0, (sum([len(t.examples) for t in doctests]), D),
....:            "Doctest output including the failure...")
    [... tests, 1 failure, ...s wall]
```

```
>>> from sage.all import *
>>> runner.failures = Integer(1)
>>> runner.update_results(D)
1
>>> DTR.report(FDS, False, Integer(0), (sum([len(t.examples) for t in␣
```

(continues on next page)

(continued from previous page)

```
↪doctests]), D),
...            "Doctest output including the failure...")
    [... tests, 1 failure, ...s wall]
```

If the user has requested that we report on skipped doctests, we do so:

```
sage: DC.options = DocTestDefaults(show_skipped=True)
sage: from collections import defaultdict
sage: optionals = defaultdict(int)
sage: optionals['magma'] = 5; optionals['long time'] = 4; optionals[''] = 1;␣
↪optionals['not tested'] = 2
sage: D = DictAsObject(dict(err=None, optionals=optionals))
sage: runner.failures = 0
sage: runner.update_results(D)
0
sage: DTR.report(FDS, False, 0, (sum([len(t.examples) for t in doctests]), D),
↪ "Good tests")
    1 unlabeled test not run
    4 long tests not run
    5 magma tests not run
    2 not tested tests not run
    0 tests not run because we ran out of time
    [... tests, ...s wall]
```

```
>>> from sage.all import *
>>> DC.options = DocTestDefaults(show_skipped=True)
>>> from collections import defaultdict
>>> optionals = defaultdict(int)
>>> optionals['magma'] = Integer(5); optionals['long time'] = Integer(4);␣
↪optionals[''] = Integer(1); optionals['not tested'] = Integer(2)
>>> D = DictAsObject(dict(err=None, optionals=optionals))
>>> runner.failures = Integer(0)
>>> runner.update_results(D)
0
>>> DTR.report(FDS, False, Integer(0), (sum([len(t.examples) for t in␣
↪doctests]), D), "Good tests")
    1 unlabeled test not run
    4 long tests not run
    5 magma tests not run
    2 not tested tests not run
    0 tests not run because we ran out of time
    [... tests, ...s wall]
```

Test an internal error in the reporter:

```
sage: DTR.report(None, None, None, None, None)
Traceback (most recent call last):
...
AttributeError: 'NoneType' object has no attribute 'basename'...
```

```
>>> from sage.all import *
>>> DTR.report(None, None, None, None, None)
```

(continues on next page)

(continued from previous page)

```
Traceback (most recent call last):
...
AttributeError: 'NoneType' object has no attribute 'basename'...
```

The only-errors mode does not output anything on success:

```
sage: DD = DocTestDefaults(only_errors=True)
sage: FDS = FileDocTestSource(filename, DD)
sage: DC = DocTestController(DD, [filename])
sage: DTR = DocTestReporter(DC)
sage: doctests, extras = FDS.create_doctests(globals())
sage: runner = SageDocTestRunner(
....:     SageOutputChecker(), verbose=False, sage_options=DD,
....:     optionflags=doctest.NORMALIZE_WHITESPACE|doctest.ELLIPSIS)
sage: Timer().start().stop().annotate(runner)
sage: D = DictAsObject({'err':None})
sage: runner.update_results(D)
0
sage: DTR.report(FDS, False, 0, (sum([len(t.examples) for t in doctests]), D),
....:            "Good tests")
```

```
>>> from sage.all import *
>>> DD = DocTestDefaults(only_errors=True)
>>> FDS = FileDocTestSource(filename, DD)
>>> DC = DocTestController(DD, [filename])
>>> DTR = DocTestReporter(DC)
>>> doctests, extras = FDS.create_doctests(globals())
>>> runner = SageDocTestRunner(
...     SageOutputChecker(), verbose=False, sage_options=DD,
...     optionflags=doctest.NORMALIZE_WHITESPACE|doctest.ELLIPSIS)
>>> Timer().start().stop().annotate(runner)
>>> D = DictAsObject({'err':None})
>>> runner.update_results(D)
0
>>> DTR.report(FDS, False, Integer(0), (sum([len(t.examples) for t in␣
↪doctests]), D),
...            "Good tests")
```

However, failures are still output in the errors-only mode:

```
sage: runner.failures = 1
sage: runner.update_results(D)
1
sage: DTR.report(FDS, False, 0, (sum([len(t.examples) for t in doctests]), D),
....:            "Failed test")
    [... tests, 1 failure, ...s wall]
```

```
>>> from sage.all import *
>>> runner.failures = Integer(1)
>>> runner.update_results(D)
1
>>> DTR.report(FDS, False, Integer(0), (sum([len(t.examples) for t in␣
```

(continues on next page)

(continued from previous page)

```
↪doctests]), D),
...             "Failed test")
    [... tests, 1 failure, ...s wall]
```

report_head(*source*, *fail_msg=None*)

Return the `sage -t [options] file.py` line as string.

INPUT:

- `source` – a source from *sage.doctest.sources*
- `fail_msg` – `None` or a string

EXAMPLES:

```
sage: from sage.doctest.reporting import DocTestReporter
sage: from sage.doctest.control import DocTestController, DocTestDefaults
sage: from sage.doctest.sources import FileDocTestSource
sage: from sage.doctest.forker import SageDocTestRunner
sage: filename = sage.doctest.reporting.__file__
sage: DD = DocTestDefaults()
sage: FDS = FileDocTestSource(filename, DD)
sage: DC = DocTestController(DD, [filename])
sage: DTR = DocTestReporter(DC)
sage: print(DTR.report_head(FDS))
.../sage/doctest/reporting.py
```

```
>>> from sage.all import *
>>> from sage.doctest.reporting import DocTestReporter
>>> from sage.doctest.control import DocTestController, DocTestDefaults
>>> from sage.doctest.sources import FileDocTestSource
>>> from sage.doctest.forker import SageDocTestRunner
>>> filename = sage.doctest.reporting.__file__
>>> DD = DocTestDefaults()
>>> FDS = FileDocTestSource(filename, DD)
>>> DC = DocTestController(DD, [filename])
>>> DTR = DocTestReporter(DC)
>>> print(DTR.report_head(FDS))
.../sage/doctest/reporting.py
```

The same with various options:

```
sage: DD.long = True
sage: print(DTR.report_head(FDS))
... --long .../sage/doctest/reporting.py
sage: print(DTR.report_head(FDS, "Failed by self-sabotage"))
... --long .../sage/doctest/reporting.py  # Failed by self-sabotage
```

```
>>> from sage.all import *
>>> DD.long = True
>>> print(DTR.report_head(FDS))
... --long .../sage/doctest/reporting.py
>>> print(DTR.report_head(FDS, "Failed by self-sabotage"))
... --long .../sage/doctest/reporting.py  # Failed by self-sabotage
```

were_doctests_with_optional_tag_run(*tag*)

Return whether doctests marked with this tag were run.

INPUT:

- `tag` – string

EXAMPLES:

```
sage: from sage.doctest.reporting import DocTestReporter
sage: from sage.doctest.control import DocTestController, DocTestDefaults
sage: filename = sage.doctest.reporting.__file__
sage: DC = DocTestController(DocTestDefaults(), [filename])
sage: DTR = DocTestReporter(DC)
```

```
>>> from sage.all import *
>>> from sage.doctest.reporting import DocTestReporter
>>> from sage.doctest.control import DocTestController, DocTestDefaults
>>> filename = sage.doctest.reporting.__file__
>>> DC = DocTestController(DocTestDefaults(), [filename])
>>> DTR = DocTestReporter(DC)
```

```
sage: DTR.were_doctests_with_optional_tag_run('sage')
True
sage: DTR.were_doctests_with_optional_tag_run('nice_unavailable_package')
False
```

```
>>> from sage.all import *
>>> DTR.were_doctests_with_optional_tag_run('sage')
True
>>> DTR.were_doctests_with_optional_tag_run('nice_unavailable_package')
False
```

sage.doctest.reporting.**signal_name**(*sig*)

Return a string describing a signal number.

EXAMPLES:

```
sage: from signal import SIGSEGV
sage: from sage.doctest.reporting import signal_name
sage: signal_name(SIGSEGV)
'segmentation fault'
sage: signal_name(9)
'kill signal'
sage: signal_name(12345)
'signal 12345'
```

```
>>> from sage.all import *
>>> from signal import SIGSEGV
>>> from sage.doctest.reporting import signal_name
>>> signal_name(SIGSEGV)
'segmentation fault'
>>> signal_name(Integer(9))
'kill signal'
```

(continues on next page)

(continued from previous page)

```
>>> signal_name(Integer(12345))
'signal 12345'
```

CHAPTER

SEVEN

DETECTING EXTERNAL SOFTWARE

This module makes up a list of external software that Sage interfaces. Availability of each software is tested only when necessary. This is mainly used for the doctests which require certain external software installed on the system.

Even though the functions in this module should also work when an external software is not present, most doctests in this module are only tested if testing of external software is explicitly enabled in order to avoid invoking external software otherwise. See Issue #28819 for details.

AUTHORS:

- Kwankyu Lee (2016-03-09) – initial version, based on code by Robert Bradshaw and Nathann Cohen

class sage.doctest.external.**AvailableSoftware**

Bases: object

This class keeps the set of available software whose availability is detected lazily from the list of external software.

EXAMPLES:

```
sage: from sage.doctest.external import external_software, available_software
sage: external_software
['cplex',
 'dvips',
 'ffmpeg',
 'gurobi',
 'internet',
 'latex',
 'latex_package_tkz_graph',
 'lualatex',
 'macaulay2',
 'magma',
 'maple',
 'mathematica',
 'matlab',
 'octave',
 'pdflatex',
 'scilab',
 'xelatex']
sage: 'internet' in available_software # random, optional - internet
True
sage: available_software.issuperset(set(['internet','latex'])) # random, optional␣
↪- internet latex
True
```

```
>>> from sage.all import *
>>> from sage.doctest.external import external_software, available_software
>>> external_software
['cplex',
 'dvips',
 'ffmpeg',
 'gurobi',
 'internet',
 'latex',
 'latex_package_tkz_graph',
 'lualatex',
 'macaulay2',
 'magma',
 'maple',
 'mathematica',
 'matlab',
 'octave',
 'pdflatex',
 'scilab',
 'xelatex']
>>> 'internet' in available_software # random, optional - internet
True
>>> available_software.issuperset(set(['internet','latex'])) # random, optional -␣
↪internet latex
True
```

detectable()

Return the list of names of those features for which testing their presence is allowed.

hidden()

Return the list of detected hidden external software.

EXAMPLES:

```
sage: # needs conway_polynomials database_cremona_mini_ellcurve database_
↪ellcurves database_graphs
sage: from sage.doctest.external import available_software
sage: from sage.features.databases import all_features
sage: for f in all_features():
....:     f.hide()
....:     if f._spkg_type() == 'standard':
....:         test = f.name in available_software
....:     f.unhide()
sage: sorted(available_software.hidden())
[...'conway_polynomials',...
 'database_cremona_mini_ellcurve',...
 'database_ellcurves',...
 'database_graphs'...]
```

```
>>> from sage.all import *
>>> # needs conway_polynomials database_cremona_mini_ellcurve database_
↪ellcurves database_graphs
>>> from sage.doctest.external import available_software
```

(continues on next page)

(continued from previous page)

```
>>> from sage.features.databases import all_features
>>> for f in all_features():
...     f.hide()
...     if f._spkg_type() == 'standard':
...         test = f.name in available_software
...     f.unhide()
>>> sorted(available_software.hidden())
[...'conway_polynomials',...
 'database_cremona_mini_ellcurve',...
 'database_ellcurves',...
 'database_graphs'...]
```

issuperset(*other*)

Return `True` if `other` is a subset of `self`.

EXAMPLES:

```
sage: from sage.doctest.external import available_software
sage: available_software.issuperset(set(['internet','latex','magma'])) #␣
↪random, optional - internet latex magma
True
```

```
>>> from sage.all import *
>>> from sage.doctest.external import available_software
>>> available_software.issuperset(set(['internet','latex','magma'])) # random,
↪ optional - internet latex magma
True
```

seen()

Return the list of detected external software.

EXAMPLES:

```
sage: from sage.doctest.external import available_software
sage: available_software.seen() # random
['internet', 'latex', 'magma']
```

```
>>> from sage.all import *
>>> from sage.doctest.external import available_software
>>> available_software.seen() # random
['internet', 'latex', 'magma']
```

sage.doctest.external.**external_features**()

Generate the features that are only to be tested if `--optional=external` is used.

See also

`sage.features.all.all_features()`

EXAMPLES:

```
sage: from sage.doctest.external import external_features
sage: next(external_features())
Feature('internet')
```

```
>>> from sage.all import *
>>> from sage.doctest.external import external_features
>>> next(external_features())
Feature('internet')
```

sage.doctest.external.**has_4ti2**()

Test if the 4ti2 package is available.

EXAMPLES:

```
sage: from sage.doctest.external import has_4ti2
sage: has_4ti2()    # optional -- 4ti2
FeatureTestResult('4ti2', True)
```

```
>>> from sage.all import *
>>> from sage.doctest.external import has_4ti2
>>> has_4ti2()    # optional -- 4ti2
FeatureTestResult('4ti2', True)
```

sage.doctest.external.**has_cplex**()

Test if CPLEX is available.

EXAMPLES:

```
sage: from sage.doctest.external import has_cplex
sage: has_cplex() # random, optional - CPLEX
FeatureTestResult('cplex', True)
```

```
>>> from sage.all import *
>>> from sage.doctest.external import has_cplex
>>> has_cplex() # random, optional - CPLEX
FeatureTestResult('cplex', True)
```

sage.doctest.external.**has_ffmpeg**()

Test if ffmpeg is available.

EXAMPLES:

```
sage: from sage.doctest.external import has_ffmpeg
sage: has_ffmpeg()       # optional -- ffmpeg
FeatureTestResult('ffmpeg', True)
```

```
>>> from sage.all import *
>>> from sage.doctest.external import has_ffmpeg
>>> has_ffmpeg()       # optional -- ffmpeg
FeatureTestResult('ffmpeg', True)
```

sage.doctest.external.**has_graphviz**()

Test if graphviz (dot, twopi, neato) are available.

EXAMPLES:

```
sage: from sage.doctest.external import has_graphviz
sage: has_graphviz()    # optional -- graphviz
FeatureTestResult('graphviz', True)
```

```
>>> from sage.all import *
>>> from sage.doctest.external import has_graphviz
>>> has_graphviz()    # optional -- graphviz
FeatureTestResult('graphviz', True)
```

sage.doctest.external.**has_gurobi**()

Test if Gurobi is available.

EXAMPLES:

```
sage: from sage.doctest.external import has_gurobi
sage: has_gurobi() # random, optional - Gurobi
FeatureTestResult('gurobi', True)
```

```
>>> from sage.all import *
>>> from sage.doctest.external import has_gurobi
>>> has_gurobi() # random, optional - Gurobi
FeatureTestResult('gurobi', True)
```

sage.doctest.external.**has_imagemagick**()

Test if ImageMagick (command magick or convert) is available.

EXAMPLES:

```
sage: from sage.doctest.external import has_imagemagick
sage: has_imagemagick() # optional -- imagemagick
FeatureTestResult('imagemagick', True)
```

```
>>> from sage.all import *
>>> from sage.doctest.external import has_imagemagick
>>> has_imagemagick() # optional -- imagemagick
FeatureTestResult('imagemagick', True)
```

sage.doctest.external.**has_internet**()

Test if Internet is available.

Failure of connecting to the site "https://www.sagemath.org" within a second is regarded as internet being not available.

EXAMPLES:

```
sage: from sage.doctest.external import has_internet
sage: has_internet()  # random, optional -- internet
FeatureTestResult('internet', True)
```

```
>>> from sage.all import *
>>> from sage.doctest.external import has_internet
>>> has_internet()  # random, optional -- internet
FeatureTestResult('internet', True)
```

sage.doctest.external.**has_latex**()

Test if Latex is available.

EXAMPLES:

```
sage: from sage.doctest.external import has_latex
sage: has_latex() # optional - latex
FeatureTestResult('latex', True)
```

```
>>> from sage.all import *
>>> from sage.doctest.external import has_latex
>>> has_latex() # optional - latex
FeatureTestResult('latex', True)
```

sage.doctest.external.**has_lualatex**()

Test if lualatex is available.

EXAMPLES:

```
sage: from sage.doctest.external import has_lualatex
sage: has_lualatex()   # optional - lualatex
FeatureTestResult('lualatex', True)
```

```
>>> from sage.all import *
>>> from sage.doctest.external import has_lualatex
>>> has_lualatex()   # optional - lualatex
FeatureTestResult('lualatex', True)
```

sage.doctest.external.**has_macaulay2**()

Test if Macaulay2 is available.

EXAMPLES:

```
sage: from sage.doctest.external import has_macaulay2
sage: has_macaulay2() # random, optional - macaulay2
True
```

```
>>> from sage.all import *
>>> from sage.doctest.external import has_macaulay2
>>> has_macaulay2() # random, optional - macaulay2
True
```

sage.doctest.external.**has_magma**()

Test if Magma is available.

EXAMPLES:

```
sage: from sage.doctest.external import has_magma
sage: has_magma() # random, optional - magma
True
```

```
>>> from sage.all import *
>>> from sage.doctest.external import has_magma
>>> has_magma() # random, optional - magma
True
```

sage.doctest.external.**has_maple**()

Test if Maple is available.

EXAMPLES:

```
sage: from sage.doctest.external import has_maple
sage: has_maple() # random, optional - maple
True
```

```
>>> from sage.all import *
>>> from sage.doctest.external import has_maple
>>> has_maple() # random, optional - maple
True
```

sage.doctest.external.**has_mathematica**()

Test if Mathematica is available.

EXAMPLES:

```
sage: from sage.doctest.external import has_mathematica
sage: has_mathematica() # random, optional - mathematica
True
```

```
>>> from sage.all import *
>>> from sage.doctest.external import has_mathematica
>>> has_mathematica() # random, optional - mathematica
True
```

sage.doctest.external.**has_matlab**()

Test if Matlab is available.

EXAMPLES:

```
sage: from sage.doctest.external import has_matlab
sage: has_matlab() # random, optional - matlab
True
```

```
>>> from sage.all import *
>>> from sage.doctest.external import has_matlab
>>> has_matlab() # random, optional - matlab
True
```

sage.doctest.external.**has_octave**()

Test if Octave is available.

EXAMPLES:

```
sage: from sage.doctest.external import has_octave
sage: has_octave() # random, optional - octave
True
```

```
>>> from sage.all import *
>>> from sage.doctest.external import has_octave
>>> has_octave() # random, optional - octave
True
```

sage.doctest.external.**has_pandoc**()

Test if pandoc is available.

EXAMPLES:

```
sage: from sage.doctest.external import has_pandoc
sage: has_pandoc()      # optional -- pandoc
FeatureTestResult('pandoc', True)
```

```
>>> from sage.all import *
>>> from sage.doctest.external import has_pandoc
>>> has_pandoc()      # optional -- pandoc
FeatureTestResult('pandoc', True)
```

sage.doctest.external.**has_pdflatex**()

Test if pdflatex is available.

EXAMPLES:

```
sage: from sage.doctest.external import has_pdflatex
sage: has_pdflatex()    # optional - pdflatex
FeatureTestResult('pdflatex', True)
```

```
>>> from sage.all import *
>>> from sage.doctest.external import has_pdflatex
>>> has_pdflatex()    # optional - pdflatex
FeatureTestResult('pdflatex', True)
```

sage.doctest.external.**has_rubiks**()

Test if the rubiks package (`cu2`, `cubex`, `dikcube`, `mcube`, `optimal`, and `size222`) is available.

EXAMPLES:

```
sage: from sage.doctest.external import has_rubiks
sage: has_rubiks()    # optional -- rubiks
FeatureTestResult('rubiks', True)
```

```
>>> from sage.all import *
>>> from sage.doctest.external import has_rubiks
>>> has_rubiks()    # optional -- rubiks
FeatureTestResult('rubiks', True)
```

sage.doctest.external.**has_scilab**()

Test if Scilab is available.

EXAMPLES:

```
sage: from sage.doctest.external import has_scilab
sage: has_scilab() # random, optional - scilab
True
```

```
>>> from sage.all import *
>>> from sage.doctest.external import has_scilab
>>> has_scilab() # random, optional - scilab
True
```

sage.doctest.external.**has_xelatex**()

Test if xelatex is available.

EXAMPLES:

```
sage: from sage.doctest.external import has_xelatex
sage: has_xelatex()   # optional - xelatex
FeatureTestResult('xelatex', True)
```

```
>>> from sage.all import *
>>> from sage.doctest.external import has_xelatex
>>> has_xelatex()   # optional - xelatex
FeatureTestResult('xelatex', True)
```

CHAPTER

EIGHT

TEST THE DOCTESTING FRAMEWORK

Many tests (with expected failures or crashes) are run in a subprocess, those tests can be found in the `tests/` subdirectory.

EXAMPLES:

```
sage: import signal
sage: import subprocess
sage: import time
sage: import os
sage: from sage.env import SAGE_SRC
sage: tests_dir = os.path.join(SAGE_SRC, 'sage', 'doctest', 'tests')
sage: tests_env = dict(os.environ)
```

```
>>> from sage.all import *
>>> import signal
>>> import subprocess
>>> import time
>>> import os
>>> from sage.env import SAGE_SRC
>>> tests_dir = os.path.join(SAGE_SRC, 'sage', 'doctest', 'tests')
>>> tests_env = dict(os.environ)
```

Unset `TERM` when running doctests, see Issue #14370:

```
sage: try:
....:     del tests_env['TERM']
....: except KeyError:
....:     pass
sage: kwds = {'cwd': tests_dir, 'env':tests_env}
```

```
>>> from sage.all import *
>>> try:
...     del tests_env['TERM']
... except KeyError:
...     pass
>>> kwds = {'cwd': tests_dir, 'env':tests_env}
```

Check that Issue #2235 has been fixed:

```
sage: subprocess.call(["python3", "-m", "sage.doctest", "--warn-long", "0",     # long␣
↪time
....:         "--random-seed=0", "--optional=sage", "longtime.rst"], **kwds)
```

(continues on next page)

(continued from previous page)

```
Running doctests...
Doctesting 1 file.
... --warn-long 0.0 --random-seed=0 longtime.rst
[0 tests, ...s wall]
----------------------------------------------------------------------
All tests passed!
----------------------------------------------------------------------
...
0
sage: subprocess.call(["python3", "-m", "sage.doctest", "--warn-long", "0",     # long␣
↪time
....:       "--random-seed=0", "--optional=sage", "-l", "longtime.rst"], **kwds)
Running doctests...
Doctesting 1 file.
... --long --warn-long 0.0 --random-seed=0 longtime.rst
[1 test, ...s wall]
----------------------------------------------------------------------
All tests passed!
----------------------------------------------------------------------
...
0
```

```
>>> from sage.all import *
>>> subprocess.call(["python3", "-m", "sage.doctest", "--warn-long", "0",     # long␣
↪time
...        "--random-seed=0", "--optional=sage", "longtime.rst"], **kwds)
Running doctests...
Doctesting 1 file.
... --warn-long 0.0 --random-seed=0 longtime.rst
[0 tests, ...s wall]
----------------------------------------------------------------------
All tests passed!
----------------------------------------------------------------------
...
0
>>> subprocess.call(["python3", "-m", "sage.doctest", "--warn-long", "0",     # long␣
↪time
...       "--random-seed=0", "--optional=sage", "-l", "longtime.rst"], **kwds)
Running doctests...
Doctesting 1 file.
... --long --warn-long 0.0 --random-seed=0 longtime.rst
[1 test, ...s wall]
----------------------------------------------------------------------
All tests passed!
----------------------------------------------------------------------
...
0
```

Check slow doctest warnings are correctly raised:

```
sage: subprocess.call(["python3", "-m", "sage.doctest", "--warn-long",      # long time
....:       "--random-seed=0", "--optional=sage", "sleep2.rst"], **kwds)
```

(continues on next page)

(continued from previous page)

```
Running doctests...
Doctesting 1 file.
... --warn-long --random-seed=0 sleep2.rst
**********************************************************************
File "sleep2.rst", line 4, in sage.doctest.tests.sleep2
Warning: slow doctest:
    while walltime(t) < 2: pass
Test ran for ...s cpu, ...s wall
Check ran for ...s cpu, ...s wall
    [2 tests, ...s wall]
----------------------------------------------------------------------
All tests passed!
----------------------------------------------------------------------
...
0
sage: subprocess.call(["python3", "-m", "sage.doctest", "--format=github", "--warn-
↪long",     # long time
....:       "--random-seed=0", "--optional=sage", "sleep2.rst"], **kwds)
Running doctests...
Doctesting 1 file.
... --warn-long --random-seed=0 sleep2.rst
**********************************************************************
::warning title=Warning: slow doctest:,file=sleep2.rst,line=4::slow doctest:: Test␣
↪ran for ...s cpu, ...s wall%0ACheck ran for ...s cpu, ...s wall%0A
    while walltime(t) < 2: pass
Test ran for ...s cpu, ...s wall
Check ran for ...s cpu, ...s wall
    [2 tests, ...s wall]
----------------------------------------------------------------------
All tests passed!
----------------------------------------------------------------------
...
0
```

```
>>> from sage.all import *
>>> subprocess.call(["python3", "-m", "sage.doctest", "--warn-long",     # long time
...       "--random-seed=0", "--optional=sage", "sleep2.rst"], **kwds)
Running doctests...
Doctesting 1 file.
... --warn-long --random-seed=0 sleep2.rst
**********************************************************************
File "sleep2.rst", line 4, in sage.doctest.tests.sleep2
Warning: slow doctest:
    while walltime(t) < 2: pass
Test ran for ...s cpu, ...s wall
Check ran for ...s cpu, ...s wall
    [2 tests, ...s wall]
----------------------------------------------------------------------
All tests passed!
----------------------------------------------------------------------
...
0
```

(continues on next page)

(continued from previous page)

```
>>> subprocess.call(["python3", "-m", "sage.doctest", "--format=github", "--warn-long
↪",     # long time
...       "--random-seed=0", "--optional=sage", "sleep2.rst"], **kwds)
Running doctests...
Doctesting 1 file.
... --warn-long --random-seed=0 sleep2.rst
**********************************************************************
::warning title=Warning: slow doctest:,file=sleep2.rst,line=4::slow doctest:: Test␣
↪ran for ...s cpu, ...s wall%0ACheck ran for ...s cpu, ...s wall%0A
    while walltime(t) < 2: pass
Test ran for ...s cpu, ...s wall
Check ran for ...s cpu, ...s wall
    [2 tests, ...s wall]
----------------------------------------------------------------------
All tests passed!
----------------------------------------------------------------------
...
0
```

Check handling of tolerances:

```
sage: subprocess.call(["python3", "-m", "sage.doctest", "--warn-long", "0",     # long␣
↪time
....:       "--random-seed=0", "--optional=sage", "tolerance.rst"], **kwds)
Running doctests...
Doctesting 1 file.
... --warn-long 0.0 --random-seed=0 tolerance.rst
**********************************************************************
File "tolerance.rst", line ..., in sage.doctest.tests.tolerance
Failed example:
    print(":-(")    # abs tol 0.1
Expected:
    :-)
Got:
    :-(
**********************************************************************
File "tolerance.rst", line ..., in sage.doctest.tests.tolerance
Failed example:
    print("1.0 2.0 3.0")  # abs tol 0.1
Expected:
    4.0 5.0
Got:
    1.0 2.0 3.0
**********************************************************************
File "tolerance.rst", line ..., in sage.doctest.tests.tolerance
Failed example:
    print("Hello")  # abs tol 0.1
Expected:
    1.0
Got:
    Hello
**********************************************************************
```

(continues on next page)

(continued from previous page)

```
File "tolerance.rst", line ..., in sage.doctest.tests.tolerance
Failed example:
    print("1.0")  # abs tol 0.1
Expected:
    Hello
Got:
    1.0
**********************************************************************
File "tolerance.rst", line ..., in sage.doctest.tests.tolerance
Failed example:
    print("Hello 1.1")  # abs tol 0.1
Expected:
    Goodbye 1.0
Got:
    Hello 1.1
**********************************************************************
File "tolerance.rst", line ..., in sage.doctest.tests.tolerance
Failed example:
    print("Hello 1.0")  # rel tol 1e-6
Expected:
    Goodbye 0.999999
Got:
    Hello 1.0
Tolerance exceeded:
    0.999999 vs 1.0, tolerance 2e-6 > 1e-6
**********************************************************************
File "tolerance.rst", line ..., in sage.doctest.tests.tolerance
Failed example:
    print("Hello 1.0")  # rel tol 1e-6
Expected:
    Hello ...
Got:
    Hello 1.0
Note: combining tolerance (# tol) with ellipsis (...) is not supported
**********************************************************************
...
1
```

```
>>> from sage.all import *
>>> subprocess.call(["python3", "-m", "sage.doctest", "--warn-long", "0",    # long␣
↪time
...      "--random-seed=0", "--optional=sage", "tolerance.rst"], **kwds)
Running doctests...
Doctesting 1 file.
... --warn-long 0.0 --random-seed=0 tolerance.rst
**********************************************************************
File "tolerance.rst", line ..., in sage.doctest.tests.tolerance
Failed example:
    print(":-(")    # abs tol 0.1
Expected:
    :-)
Got:
```

(continues on next page)

(continued from previous page)

```
    :-(
**********************************************************************
File "tolerance.rst", line ..., in sage.doctest.tests.tolerance
Failed example:
    print("1.0 2.0 3.0")  # abs tol 0.1
Expected:
    4.0 5.0
Got:
    1.0 2.0 3.0
**********************************************************************
File "tolerance.rst", line ..., in sage.doctest.tests.tolerance
Failed example:
    print("Hello")  # abs tol 0.1
Expected:
    1.0
Got:
    Hello
**********************************************************************
File "tolerance.rst", line ..., in sage.doctest.tests.tolerance
Failed example:
    print("1.0")  # abs tol 0.1
Expected:
    Hello
Got:
    1.0
**********************************************************************
File "tolerance.rst", line ..., in sage.doctest.tests.tolerance
Failed example:
    print("Hello 1.1")  # abs tol 0.1
Expected:
    Goodbye 1.0
Got:
    Hello 1.1
**********************************************************************
File "tolerance.rst", line ..., in sage.doctest.tests.tolerance
Failed example:
    print("Hello 1.0")  # rel tol 1e-6
Expected:
    Goodbye 0.999999
Got:
    Hello 1.0
Tolerance exceeded:
    0.999999 vs 1.0, tolerance 2e-6 > 1e-6
**********************************************************************
File "tolerance.rst", line ..., in sage.doctest.tests.tolerance
Failed example:
    print("Hello 1.0")  # rel tol 1e-6
Expected:
    Hello ...
Got:
    Hello 1.0
Note: combining tolerance (# tol) with ellipsis (...) is not supported
```

(continues on next page)

(continued from previous page)

```
**********************************************************************
...
1
```

Test the `--initial` option:

```
sage: subprocess.call(["python3", "-m", "sage.doctest", "--warn-long", "0",     # long␣
↪time
....:       "--random-seed=0", "--optional=sage", "-i", "initial.rst"], **kwds)
Running doctests...
Doctesting 1 file.
... --warn-long 0.0 --random-seed=0 initial.rst
**********************************************************************
File "initial.rst", line 4, in sage.doctest.tests.initial
Failed example:
    a = binomiak(10,5)  # random to test that we still get the exception
Exception raised:
    Traceback (most recent call last):
    ...
    NameError: name 'binomiak' is not defined
**********************************************************************
File "initial.rst", line 14, in sage.doctest.tests.initial
Failed example:
    binomial(10,5)
Expected:
    255
Got:
    252
**********************************************************************
...
----------------------------------------------------------------------
...  --warn-long 0.0 --random-seed=0 initial.rst  # 5 doctests failed
----------------------------------------------------------------------
...
1
```

```
>>> from sage.all import *
>>> subprocess.call(["python3", "-m", "sage.doctest", "--warn-long", "0",     # long␣
↪time
...       "--random-seed=0", "--optional=sage", "-i", "initial.rst"], **kwds)
Running doctests...
Doctesting 1 file.
... --warn-long 0.0 --random-seed=0 initial.rst
**********************************************************************
File "initial.rst", line 4, in sage.doctest.tests.initial
Failed example:
    a = binomiak(10,5)  # random to test that we still get the exception
Exception raised:
    Traceback (most recent call last):
    ...
    NameError: name 'binomiak' is not defined
**********************************************************************
```

(continues on next page)

(continued from previous page)

```
File "initial.rst", line 14, in sage.doctest.tests.initial
Failed example:
    binomial(10,5)
Expected:
    255
Got:
    252
**********************************************************************
...
----------------------------------------------------------------------
...  --warn-long 0.0 --random-seed=0 initial.rst  # 5 doctests failed
----------------------------------------------------------------------
...
1
```

Test the `--exitfirst` option:

```
sage: subprocess.call(["python3", "-m", "sage.doctest", "--warn-long", "0",     # long␣
↪time
....:       "--random-seed=0", "--optional=sage", "--exitfirst", "initial.rst"],␣
↪**kwds)
Running doctests...
Doctesting 1 file.
... --warn-long 0.0 --random-seed=0 initial.rst
**********************************************************************
File "initial.rst", line 4, in sage.doctest.tests.initial
Failed example:
    a = binomiak(10,5)  # random to test that we still get the exception
Exception raised:
    Traceback (most recent call last):
    ...
    NameError: name 'binomiak' is not defined
**********************************************************************
...
----------------------------------------------------------------------
...  --warn-long 0.0 --random-seed=0 initial.rst  # 1 doctest failed
----------------------------------------------------------------------
...
1
```

```
>>> from sage.all import *
>>> subprocess.call(["python3", "-m", "sage.doctest", "--warn-long", "0",     # long␣
↪time
...       "--random-seed=0", "--optional=sage", "--exitfirst", "initial.rst"], **kwds)
Running doctests...
Doctesting 1 file.
... --warn-long 0.0 --random-seed=0 initial.rst
**********************************************************************
File "initial.rst", line 4, in sage.doctest.tests.initial
Failed example:
    a = binomiak(10,5)  # random to test that we still get the exception
Exception raised:
```

(continues on next page)

(continued from previous page)

```
    Traceback (most recent call last):
    ...
    NameError: name 'binomiak' is not defined
**********************************************************************
...
----------------------------------------------------------------------
...  --warn-long 0.0 --random-seed=0 initial.rst  # 1 doctest failed
----------------------------------------------------------------------
...
1
```

Test a timeout using the `SAGE_TIMEOUT` environment variable. Also set `CYSIGNALS_CRASH_NDEBUG` to help ensure the test times out in a timely manner (Issue #26912):

```
sage: from copy import deepcopy
sage: kwds2 = deepcopy(kwds)
sage: kwds2['env'].update({'SAGE_TIMEOUT': '1', 'CYSIGNALS_CRASH_NDEBUG': '1'})
sage: subprocess.call(["python3", "-m", "sage.doctest", "--warn-long", "0",     # long␣
↪time
....:       "--random-seed=0", "--optional=sage", "99seconds.rst"], **kwds2)
Running doctests...
Doctesting 1 file.
... --warn-long 0.0 --random-seed=0 99seconds.rst
    Timed out
**********************************************************************
Tests run before process (pid=...) timed out:
...
----------------------------------------------------------------------
... --warn-long 0.0 --random-seed=0 99seconds.rst  # Timed out
----------------------------------------------------------------------
...
4
```

```
>>> from sage.all import *
>>> from copy import deepcopy
>>> kwds2 = deepcopy(kwds)
>>> kwds2['env'].update({'SAGE_TIMEOUT': '1', 'CYSIGNALS_CRASH_NDEBUG': '1'})
>>> subprocess.call(["python3", "-m", "sage.doctest", "--warn-long", "0",     # long␣
↪time
...       "--random-seed=0", "--optional=sage", "99seconds.rst"], **kwds2)
Running doctests...
Doctesting 1 file.
... --warn-long 0.0 --random-seed=0 99seconds.rst
    Timed out
**********************************************************************
Tests run before process (pid=...) timed out:
...
----------------------------------------------------------------------
... --warn-long 0.0 --random-seed=0 99seconds.rst  # Timed out
----------------------------------------------------------------------
...
4
```

Test handling of `KeyboardInterrupt` in doctests:

```
sage: subprocess.call(["python3", "-m", "sage.doctest", "--warn-long", "0",    # long␣
↪time
....:       "--random-seed=0", "--optional=sage", "keyboardinterrupt.rst"], **kwds)
Running doctests...
Doctesting 1 file.
... --warn-long 0.0 --random-seed=0 keyboardinterrupt.rst
**********************************************************************
File "keyboardinterrupt.rst", line 11, in sage.doctest.tests.keyboardinterrupt
Failed example:
    raise KeyboardInterrupt
Exception raised:
    Traceback (most recent call last):
    ...
    KeyboardInterrupt
**********************************************************************
...
----------------------------------------------------------------------
... --warn-long 0.0 --random-seed=0 keyboardinterrupt.rst  # 1 doctest failed
----------------------------------------------------------------------
...
1
```

```
>>> from sage.all import *
>>> subprocess.call(["python3", "-m", "sage.doctest", "--warn-long", "0",    # long␣
↪time
...       "--random-seed=0", "--optional=sage", "keyboardinterrupt.rst"], **kwds)
Running doctests...
Doctesting 1 file.
... --warn-long 0.0 --random-seed=0 keyboardinterrupt.rst
**********************************************************************
File "keyboardinterrupt.rst", line 11, in sage.doctest.tests.keyboardinterrupt
Failed example:
    raise KeyboardInterrupt
Exception raised:
    Traceback (most recent call last):
    ...
    KeyboardInterrupt
**********************************************************************
...
----------------------------------------------------------------------
... --warn-long 0.0 --random-seed=0 keyboardinterrupt.rst  # 1 doctest failed
----------------------------------------------------------------------
...
1
```

Interrupt the doctester:

```
sage: subprocess.call(["python3", "-m", "sage.doctest", "--warn-long", "0",    # long␣
↪time
....:       "--random-seed=0", "--optional=sage", "interrupt.rst"], **kwds)
Running doctests...
```

(continues on next page)

(continued from previous page)

```
Doctesting 1 file...
----------------------------------------------------------------------
Doctests interrupted: 0/1 files tested
----------------------------------------------------------------------
...
128
```

```
>>> from sage.all import *
>>> subprocess.call(["python3", "-m", "sage.doctest", "--warn-long", "0",     # long␣
↪time
...       "--random-seed=0", "--optional=sage", "interrupt.rst"], **kwds)
Running doctests...
Doctesting 1 file...
----------------------------------------------------------------------
Doctests interrupted: 0/1 files tested
----------------------------------------------------------------------
...
128
```

Interrupt the doctester (while parallel testing) when a doctest cannot be interrupted. We also test that passing a ridiculous number of threads doesn't hurt:

```
sage: F = tmp_filename()
sage: from copy import deepcopy
sage: kwds2 = deepcopy(kwds)
sage: kwds2['env']['DOCTEST_TEST_PID_FILE'] = F  # Doctester will write its PID in␣
↪this file
sage: subprocess.call(["python3", "-m", "sage.doctest", "-p", "1000000", "--
↪timeout=120",  # long time
....:       "--die_timeout=10", "--optional=sage",
....:       "--warn-long", "0", "99seconds.rst", "interrupt_diehard.rst"], **kwds2)
Running doctests...
Doctesting 2 files using 1000000 threads...
Killing test 99seconds.rst
Killing test interrupt_diehard.rst
----------------------------------------------------------------------
Doctests interrupted: 0/2 files tested
----------------------------------------------------------------------
...
128
```

```
>>> from sage.all import *
>>> F = tmp_filename()
>>> from copy import deepcopy
>>> kwds2 = deepcopy(kwds)
>>> kwds2['env']['DOCTEST_TEST_PID_FILE'] = F  # Doctester will write its PID in this␣
↪file
>>> subprocess.call(["python3", "-m", "sage.doctest", "-p", "1000000", "--timeout=120
↪",  # long time
...       "--die_timeout=10", "--optional=sage",
...       "--warn-long", "0", "99seconds.rst", "interrupt_diehard.rst"], **kwds2)
Running doctests...
```

(continues on next page)

(continued from previous page)

```
Doctesting 2 files using 1000000 threads...
Killing test 99seconds.rst
Killing test interrupt_diehard.rst
----------------------------------------------------------------------
Doctests interrupted: 0/2 files tested
----------------------------------------------------------------------
...
128
```

Even though the doctester master process has exited, the child process is still alive, but it should be killed automatically after the `die_timeout` given above (10 seconds):

```
sage: # long time
sage: pid = int(open(F).read())
sage: time.sleep(2)
sage: os.kill(pid, signal.SIGQUIT)  # 2 seconds passed => still alive
sage: time.sleep(8)
sage: os.kill(pid, signal.SIGQUIT)  # 10 seconds passed => dead  # random
Traceback (most recent call last):
...
ProcessLookupError: ...
```

```
>>> from sage.all import *
>>> # long time
>>> pid = int(open(F).read())
>>> time.sleep(Integer(2))
>>> os.kill(pid, signal.SIGQUIT)  # 2 seconds passed => still alive
>>> time.sleep(Integer(8))
>>> os.kill(pid, signal.SIGQUIT)  # 10 seconds passed => dead  # random
Traceback (most recent call last):
...
ProcessLookupError: ...
```

If the child process is dead and removed, the last output should be as above. However, the child process interrupted its parent process (see `'interrupt_diehard.rst'`), and became an orphan process. Depending on the system, an orphan process may eventually become a zombie process instead of being removed, and then the last output would just be a blank. Hence the `# random` tag.

Test a doctest failing with `abort()`:

```
sage: subprocess.call(["python3", "-m", "sage.doctest", "--warn-long", "0",    # long␣
↪time
....:      "--random-seed=0", "--optional=sage", "abort.rst"], **kwds)
Running doctests...
Doctesting 1 file.
... --warn-long 0.0 --random-seed=0 abort.rst
    Killed due to abort
**********************************************************************
Tests run before process (pid=...) failed:
...
----------------------------------------------------------------------
Unhandled SIGABRT: An abort() occurred.
This probably occurred because a *compiled* module has a bug
```

(continues on next page)

(continued from previous page)

```
in it and is not properly wrapped with sig_on(), sig_off().
Python will now terminate.
-----------------------------------------------------------------------
...
-----------------------------------------------------------------------
... --warn-long 0.0 --random-seed=0 abort.rst  # Killed due to abort
-----------------------------------------------------------------------
...
16
```

```
>>> from sage.all import *
>>> subprocess.call(["python3", "-m", "sage.doctest", "--warn-long", "0",     # long␣
↪time
...      "--random-seed=0", "--optional=sage", "abort.rst"], **kwds)
Running doctests...
Doctesting 1 file.
... --warn-long 0.0 --random-seed=0 abort.rst
    Killed due to abort
**********************************************************************
Tests run before process (pid=...) failed:
...
-----------------------------------------------------------------------
Unhandled SIGABRT: An abort() occurred.
This probably occurred because a *compiled* module has a bug
in it and is not properly wrapped with sig_on(), sig_off().
Python will now terminate.
-----------------------------------------------------------------------
...
-----------------------------------------------------------------------
... --warn-long 0.0 --random-seed=0 abort.rst  # Killed due to abort
-----------------------------------------------------------------------
...
16
```

A different kind of crash (also test printing of line continuation . . . :, represented by <DOTSCOLON> below):

```
sage: # long time
sage: proc = subprocess.run(["python3", "-m", "sage.doctest", "--warn-long", "0",
....:      "--random-seed=0", "--optional=sage", "fail_and_die.rst"], **kwds,
....:      stdout=subprocess.PIPE, text=True)
sage: # the replacements are needed to avoid the strings being interpreted
....: # specially by the doctesting framework
sage: print(proc.stdout.replace('sage:', 'sage<COLON>').replace('....:', '<DOTSCOLON>
↪'))
Running doctests...
Doctesting 1 file.
... --warn-long 0.0 --random-seed=0 fail_and_die.rst
**********************************************************************
File "fail_and_die.rst", line 8, in sage.doctest.tests.fail_and_die
Failed example:
    this_gives_a_NameError
Exception raised:
```

(continues on next page)

(continued from previous page)

```
    Traceback (most recent call last):
    ...
    NameError: name 'this_gives_a_NameError' is not defined
    Killed due to kill signal
**********************************************************************
Tests run before process (pid=...) failed:
sage<COLON> import time, signal ## line 4 ##
sage<COLON> print(1,
<DOTSCOLON>       2) ## line 5 ##
1 2
sage<COLON> this_gives_a_NameError ## line 8 ##
sage<COLON> os.kill(os.getpid(), signal.SIGKILL) ## line 9 ##
**********************************************************************
----------------------------------------------------------------------
... --warn-long 0.0 --random-seed=0 fail_and_die.rst  # Killed due to kill signal
----------------------------------------------------------------------
...
sage: proc.returncode
16
```

```
>>> from sage.all import *
>>> # long time
>>> proc = subprocess.run(["python3", "-m", "sage.doctest", "--warn-long", "0",
...       "--random-seed=0", "--optional=sage", "fail_and_die.rst"], **kwds,
...       stdout=subprocess.PIPE, text=True)
>>> # the replacements are needed to avoid the strings being interpreted
... # specially by the doctesting framework
>>> print(proc.stdout.replace('sage:', 'sage<COLON>').replace('....:', '<DOTSCOLON>'))
Running doctests...
Doctesting 1 file.
... --warn-long 0.0 --random-seed=0 fail_and_die.rst
**********************************************************************
File "fail_and_die.rst", line 8, in sage.doctest.tests.fail_and_die
Failed example:
    this_gives_a_NameError
Exception raised:
    Traceback (most recent call last):
    ...
    NameError: name 'this_gives_a_NameError' is not defined
    Killed due to kill signal
**********************************************************************
Tests run before process (pid=...) failed:
sage<COLON> import time, signal ## line 4 ##
sage<COLON> print(1,
<DOTSCOLON>       2) ## line 5 ##
1 2
sage<COLON> this_gives_a_NameError ## line 8 ##
sage<COLON> os.kill(os.getpid(), signal.SIGKILL) ## line 9 ##
**********************************************************************
----------------------------------------------------------------------
... --warn-long 0.0 --random-seed=0 fail_and_die.rst  # Killed due to kill signal
----------------------------------------------------------------------
```

(continues on next page)

(continued from previous page)

```
...
>>> proc.returncode
16
```

Test that `sig_on_count` is checked correctly:

```
sage: subprocess.call(["python3", "-m", "sage.doctest", "--warn-long", "0",     # long␣
↪time
....:       "--random-seed=0", "--optional=sage", "sig_on.rst"], **kwds)
Running doctests...
Doctesting 1 file.
... --warn-long 0.0 --random-seed=0 sig_on.rst
**********************************************************************
File "sig_on.rst", line 6, in sage.doctest.tests.sig_on
Failed example:
    sig_on_count() # check sig_on/off pairings (virtual doctest)
Expected:
    0
Got:
    1
**********************************************************************
1 item had failures:
   1 of   5 in sage.doctest.tests.sig_on
    [3 tests, 1 failure, ...]
----------------------------------------------------------------------
... --warn-long 0.0 --random-seed=0 sig_on.rst  # 1 doctest failed
----------------------------------------------------------------------
...
1
```

```
>>> from sage.all import *
>>> subprocess.call(["python3", "-m", "sage.doctest", "--warn-long", "0",     # long␣
↪time
...       "--random-seed=0", "--optional=sage", "sig_on.rst"], **kwds)
Running doctests...
Doctesting 1 file.
... --warn-long 0.0 --random-seed=0 sig_on.rst
**********************************************************************
File "sig_on.rst", line 6, in sage.doctest.tests.sig_on
Failed example:
    sig_on_count() # check sig_on/off pairings (virtual doctest)
Expected:
    0
Got:
    1
**********************************************************************
1 item had failures:
   1 of   5 in sage.doctest.tests.sig_on
    [3 tests, 1 failure, ...]
----------------------------------------------------------------------
... --warn-long 0.0 --random-seed=0 sig_on.rst  # 1 doctest failed
----------------------------------------------------------------------
```

(continues on next page)

(continued from previous page)

```
...
1
```

Test logfiles in serial and parallel mode (see Issue #19271):

```
sage: t = tmp_filename()
sage: subprocess.call(["python3", "-m", "sage.doctest", "--serial", "--warn-long", "0
↪",     # long time
....:       "--random-seed=0", "--optional=sage", "--logfile", t, "simple_failure.rst
↪"],
....:       stdout=open(os.devnull, "w"), **kwds)
1
sage: print(open(t).read())  # long time
Running doctests...
Doctesting 1 file.
... --warn-long 0.0 --random-seed=0 simple_failure.rst
**********************************************************************
File "simple_failure.rst", line 7, in sage.doctest.tests.simple_failure
Failed example:
    a * b
Expected:
    20
Got:
    15
**********************************************************************
1 item had failures:
   1 of   5 in sage.doctest.tests.simple_failure
    [4 tests, 1 failure, ...]
----------------------------------------------------------------------
... --warn-long 0.0 --random-seed=0 simple_failure.rst  # 1 doctest failed
----------------------------------------------------------------------
...

sage: subprocess.call(["python3", "-m", "sage.doctest", "--warn-long", "0",     # long␣
↪time
....:       "--random-seed=0", "--optional=sage", "--logfile", t, "simple_failure.rst
↪"],
....:       stdout=open(os.devnull, "w"), **kwds)
1
sage: print(open(t).read())  # long time
Running doctests...
Doctesting 1 file.
... --warn-long 0.0 --random-seed=0 simple_failure.rst
**********************************************************************
File "simple_failure.rst", line 7, in sage.doctest.tests.simple_failure
Failed example:
    a * b
Expected:
    20
Got:
    15
**********************************************************************
```

(continues on next page)

(continued from previous page)

```
1 item had failures:
   1 of   5 in sage.doctest.tests.simple_failure
    [4 tests, 1 failure, ...]
----------------------------------------------------------------------
... --warn-long 0.0 --random-seed=0 simple_failure.rst  # 1 doctest failed
----------------------------------------------------------------------
...
```

```
>>> from sage.all import *
>>> t = tmp_filename()
>>> subprocess.call(["python3", "-m", "sage.doctest", "--serial", "--warn-long", "0",␣
↪   # long time
...       "--random-seed=0", "--optional=sage", "--logfile", t, "simple_failure.rst"],
...       stdout=open(os.devnull, "w"), **kwds)
1
>>> print(open(t).read())  # long time
Running doctests...
Doctesting 1 file.
... --warn-long 0.0 --random-seed=0 simple_failure.rst
**********************************************************************
File "simple_failure.rst", line 7, in sage.doctest.tests.simple_failure
Failed example:
    a * b
Expected:
    20
Got:
    15
**********************************************************************
1 item had failures:
   1 of   5 in sage.doctest.tests.simple_failure
    [4 tests, 1 failure, ...]
----------------------------------------------------------------------
... --warn-long 0.0 --random-seed=0 simple_failure.rst  # 1 doctest failed
----------------------------------------------------------------------
...

>>> subprocess.call(["python3", "-m", "sage.doctest", "--warn-long", "0",    # long␣
↪time
...       "--random-seed=0", "--optional=sage", "--logfile", t, "simple_failure.rst"],
...       stdout=open(os.devnull, "w"), **kwds)
1
>>> print(open(t).read())  # long time
Running doctests...
Doctesting 1 file.
... --warn-long 0.0 --random-seed=0 simple_failure.rst
**********************************************************************
File "simple_failure.rst", line 7, in sage.doctest.tests.simple_failure
Failed example:
    a * b
Expected:
    20
Got:
```

(continues on next page)

(continued from previous page)

```
    15
**********************************************************************
1 item had failures:
   1 of   5 in sage.doctest.tests.simple_failure
    [4 tests, 1 failure, ...]
----------------------------------------------------------------------
... --warn-long 0.0 --random-seed=0 simple_failure.rst  # 1 doctest failed
----------------------------------------------------------------------
...
```

Test the `--debug` option:

```
sage: subprocess.call(["python3", "-m", "sage.doctest", "--warn-long", "0",    # long␣
↪time
....:      "--random-seed=0", "--optional=sage", "--debug", "simple_failure.rst"],
....:      stdin=open(os.devnull), **kwds)
Running doctests...
Doctesting 1 file.
... --warn-long 0.0 --random-seed=0 simple_failure.rst
**********************************************************************
File "simple_failure.rst", line 7, in sage.doctest.tests.simple_failure
Failed example:
    a * b
Expected:
    20
Got:
    15
**********************************************************************
Previously executed commands:
    s...: a = 3
    s...: b = 5
    s...: a + b
    8
sage:

Returning to doctests...
**********************************************************************
1 item had failures:
   1 of   5 in sage.doctest.tests.simple_failure
    [4 tests, 1 failure, ...]
----------------------------------------------------------------------
... --warn-long 0.0 --random-seed=0 simple_failure.rst  # 1 doctest failed
----------------------------------------------------------------------
...
1
```

```
>>> from sage.all import *
>>> subprocess.call(["python3", "-m", "sage.doctest", "--warn-long", "0",    # long␣
↪time
...      "--random-seed=0", "--optional=sage", "--debug", "simple_failure.rst"],
...      stdin=open(os.devnull), **kwds)
Running doctests...
```

(continues on next page)

(continued from previous page)

```
Doctesting 1 file.
... --warn-long 0.0 --random-seed=0 simple_failure.rst
**********************************************************************
File "simple_failure.rst", line 7, in sage.doctest.tests.simple_failure
Failed example:
    a * b
Expected:
    20
Got:
    15
**********************************************************************
Previously executed commands:
    s...: a = 3
    s...: b = 5
    s...: a + b
    8
sage:
<BLANKLINE>
Returning to doctests...
**********************************************************************
1 item had failures:
   1 of   5 in sage.doctest.tests.simple_failure
    [4 tests, 1 failure, ...]
----------------------------------------------------------------------
... --warn-long 0.0 --random-seed=0 simple_failure.rst  # 1 doctest failed
----------------------------------------------------------------------
...
1
```

Test running under gdb, without and with a timeout:

```
sage: subprocess.call(["python3", "-m", "sage.doctest",  "--warn-long", "0",    # long␣
↪time, optional: gdb
....:      "--random-seed=0", "--optional=sage", "--gdb", "1second.rst"],
....:      stdin=open(os.devnull), **kwds)
exec gdb ...
Running doctests...
Doctesting 1 file...
... 1second.rst...
    [2 tests, ...s wall]
----------------------------------------------------------------------
All tests passed!
----------------------------------------------------------------------
...
0
```

```
>>> from sage.all import *
>>> subprocess.call(["python3", "-m", "sage.doctest",  "--warn-long", "0",    # long␣
↪time, optional: gdb
...      "--random-seed=0", "--optional=sage", "--gdb", "1second.rst"],
...      stdin=open(os.devnull), **kwds)
exec gdb ...
```

(continues on next page)

(continued from previous page)

```
Running doctests...
Doctesting 1 file...
... 1second.rst...
    [2 tests, ...s wall]
----------------------------------------------------------------------
All tests passed!
----------------------------------------------------------------------
...
0
```

gdb might need a long time to start up, so we allow 30 seconds:

```
sage: subprocess.call(["python3", "-m", "sage.doctest",  "--warn-long", "0",    # long␣
↪time, optional: gdb
....:       "--random-seed=0", "--optional=sage", "--gdb", "-T30", "99seconds.rst"],
....:       stdin=open(os.devnull), **kwds)
exec gdb ...
Running doctests...
    Timed out
4
```

```
>>> from sage.all import *
>>> subprocess.call(["python3", "-m", "sage.doctest",  "--warn-long", "0",    # long␣
↪time, optional: gdb
...       "--random-seed=0", "--optional=sage", "--gdb", "-T30", "99seconds.rst"],
...       stdin=open(os.devnull), **kwds)
exec gdb ...
Running doctests...
    Timed out
4
```

Test the `--show-skipped` option:

```
sage: subprocess.call(["python3", "-m", "sage.doctest", "--warn-long", "0",     # long␣
↪time
....:       "--random-seed=0", "--optional=sage", "--show-skipped", "show_skipped.rst
↪"], **kwds)
Running doctests ...
Doctesting 1 file.
... --warn-long 0.0 --random-seed=0 show_skipped.rst
    2 tests not run due to known bugs
    1 gap test not run
    1 long test not run
    1 not tested test not run
    0 tests not run because we ran out of time
    [2 tests, ...s wall]
----------------------------------------------------------------------
All tests passed!
----------------------------------------------------------------------
...
0
```

```
>>> from sage.all import *
>>> subprocess.call(["python3", "-m", "sage.doctest", "--warn-long", "0",     # long␣
↪time
...       "--random-seed=0", "--optional=sage", "--show-skipped", "show_skipped.rst"],␣
↪**kwds)
Running doctests ...
Doctesting 1 file.
... --warn-long 0.0 --random-seed=0 show_skipped.rst
    2 tests not run due to known bugs
    1 gap test not run
    1 long test not run
    1 not tested test not run
    0 tests not run because we ran out of time
    [2 tests, ...s wall]
----------------------------------------------------------------------
All tests passed!
----------------------------------------------------------------------
...
0
```

Optional tests are run correctly:

```
sage: subprocess.call(["python3", "-m", "sage.doctest", "--warn-long", "0", "--long",␣
↪ # long time
....:       "--random-seed=0", "--show-skipped", "--optional=sage,gap", "show_skipped.
↪rst"], **kwds)
Running doctests ...
Doctesting 1 file.
... --long --warn-long 0.0 --random-seed=0 show_skipped.rst
    2 tests not run due to known bugs
    1 not tested test not run
    0 tests not run because we ran out of time
    [4 tests, ...s wall]
----------------------------------------------------------------------
All tests passed!
----------------------------------------------------------------------
...
0

sage: subprocess.call(["python3", "-m", "sage.doctest", "--warn-long", "0", "--long",␣
↪ # long time
....:       "--random-seed=0", "--show-skipped", "--optional=gAp", "show_skipped.rst"],
↪ **kwds)
Running doctests ...
Doctesting 1 file.
... --long --warn-long 0.0 --random-seed=0 show_skipped.rst
    2 tests not run due to known bugs
    1 not tested test not run
    2 sage tests not run
    0 tests not run because we ran out of time
    [2 tests, ...s wall]
----------------------------------------------------------------------
All tests passed!
```

(continues on next page)

(continued from previous page)

```
----------------------------------------------------------------------
...
0
```

```
>>> from sage.all import *
>>> subprocess.call(["python3", "-m", "sage.doctest", "--warn-long", "0", "--long",
↪# long time
...       "--random-seed=0", "--show-skipped", "--optional=sage,gap", "show_skipped.rst
↪"], **kwds)
Running doctests ...
Doctesting 1 file.
... --long --warn-long 0.0 --random-seed=0 show_skipped.rst
    2 tests not run due to known bugs
    1 not tested test not run
    0 tests not run because we ran out of time
    [4 tests, ...s wall]
----------------------------------------------------------------------
All tests passed!
----------------------------------------------------------------------
...
0

>>> subprocess.call(["python3", "-m", "sage.doctest", "--warn-long", "0", "--long",
↪# long time
...       "--random-seed=0", "--show-skipped", "--optional=gAp", "show_skipped.rst"],␣
↪**kwds)
Running doctests ...
Doctesting 1 file.
... --long --warn-long 0.0 --random-seed=0 show_skipped.rst
    2 tests not run due to known bugs
    1 not tested test not run
    2 sage tests not run
    0 tests not run because we ran out of time
    [2 tests, ...s wall]
----------------------------------------------------------------------
All tests passed!
----------------------------------------------------------------------
...
0
```

Test an invalid value for `--optional`:

```
sage: subprocess.call(["python3", "-m", "sage.doctest", "--warn-long", "0",
....:       "--random-seed=0", "--optional=bad-option", "show_skipped.rst"], **kwds)
Traceback (most recent call last):
...
ValueError: invalid optional tag 'bad-option'
1
```

```
>>> from sage.all import *
>>> subprocess.call(["python3", "-m", "sage.doctest", "--warn-long", "0",
...       "--random-seed=0", "--optional=bad-option", "show_skipped.rst"], **kwds)
```

(continues on next page)

(continued from previous page)

```
Traceback (most recent call last):
...
ValueError: invalid optional tag 'bad-option'
1
```

Test `atexit` support in the doctesting framework:

```
sage: F = tmp_filename()
sage: os.path.isfile(F)
True
sage: from copy import deepcopy
sage: kwds2 = deepcopy(kwds)
sage: kwds2['env']['DOCTEST_DELETE_FILE'] = F
sage: subprocess.call(["python3", "-m", "sage.doctest", "--warn-long", "0",     # long␣
↪time
....:       "--random-seed=0", "--optional=sage", "atexit.rst"], **kwds2)
Running doctests...
Doctesting 1 file.
... --warn-long 0.0 --random-seed=0 atexit.rst
    [3 tests, ...s wall]
----------------------------------------------------------------------
All tests passed!
----------------------------------------------------------------------
...
0
sage: os.path.isfile(F)  # long time
False
sage: try:
....:     os.unlink(F)
....: except OSError:
....:     pass
```

```
>>> from sage.all import *
>>> F = tmp_filename()
>>> os.path.isfile(F)
True
>>> from copy import deepcopy
>>> kwds2 = deepcopy(kwds)
>>> kwds2['env']['DOCTEST_DELETE_FILE'] = F
>>> subprocess.call(["python3", "-m", "sage.doctest", "--warn-long", "0",     # long␣
↪time
...       "--random-seed=0", "--optional=sage", "atexit.rst"], **kwds2)
Running doctests...
Doctesting 1 file.
... --warn-long 0.0 --random-seed=0 atexit.rst
    [3 tests, ...s wall]
----------------------------------------------------------------------
All tests passed!
----------------------------------------------------------------------
...
0
>>> os.path.isfile(F)  # long time
```

(continues on next page)

(continued from previous page)

```
False
>>> try:
...     os.unlink(F)
... except OSError:
...     pass
```

Test that random tests are reproducible:

```
sage: subprocess.call(["python3", "-m", "sage.doctest", "--warn-long", "0",     # long␣
↪time
....:       "--random-seed=0", "--optional=sage", "random_seed.rst"], **kwds)
Running doctests...
Doctesting 1 file.
... --warn-long 0.0 --random-seed=0 random_seed.rst
**********************************************************************
File "random_seed.rst", line 3, in sage.doctest.tests.random_seed
Failed example:
    randint(5, 10)
Expected:
    9
Got:
    5
**********************************************************************
1 item had failures:
   1 of   2 in sage.doctest.tests.random_seed
    [1 test, 1 failure, ...s wall]
----------------------------------------------------------------------
... --warn-long 0.0 --random-seed=0 random_seed.rst  # 1 doctest failed
----------------------------------------------------------------------
...
1
sage: subprocess.call(["python3", "-m", "sage.doctest", "--warn-long", "0",     # long␣
↪time
....:       "--random-seed=4", "--optional=sage", "random_seed.rst"], **kwds)
Running doctests...
Doctesting 1 file.
... --warn-long 0.0 --random-seed=4 random_seed.rst
    [1 test, ...s wall]
----------------------------------------------------------------------
All tests passed!
----------------------------------------------------------------------
...
0
```

```
>>> from sage.all import *
>>> subprocess.call(["python3", "-m", "sage.doctest", "--warn-long", "0",     # long␣
↪time
...       "--random-seed=0", "--optional=sage", "random_seed.rst"], **kwds)
Running doctests...
Doctesting 1 file.
... --warn-long 0.0 --random-seed=0 random_seed.rst
**********************************************************************
```

(continues on next page)

(continued from previous page)

```
File "random_seed.rst", line 3, in sage.doctest.tests.random_seed
Failed example:
    randint(5, 10)
Expected:
    9
Got:
    5
**********************************************************************
1 item had failures:
   1 of   2 in sage.doctest.tests.random_seed
    [1 test, 1 failure, ...s wall]
----------------------------------------------------------------------
... --warn-long 0.0 --random-seed=0 random_seed.rst  # 1 doctest failed
----------------------------------------------------------------------
...
1
>>> subprocess.call(["python3", "-m", "sage.doctest", "--warn-long", "0",     # long␣
↪time
...      "--random-seed=4", "--optional=sage", "random_seed.rst"], **kwds)
Running doctests...
Doctesting 1 file.
... --warn-long 0.0 --random-seed=4 random_seed.rst
    [1 test, ...s wall]
----------------------------------------------------------------------
All tests passed!
----------------------------------------------------------------------
...
0
```

CHAPTER

NINE

UTILITY FUNCTIONS

This module contains various utility functions and classes used in doctesting.

AUTHORS:

- David Roe (2012-03-27) – initial version, based on Robert Bradshaw's code.

class sage.doctest.util.**NestedName**(*base*)

Bases: object

Class used to construct fully qualified names based on indentation level.

EXAMPLES:

```
sage: from sage.doctest.util import NestedName
sage: qname = NestedName('sage.categories.algebras')
sage: qname[0] = 'Algebras'; qname
sage.categories.algebras.Algebras
sage: qname[4] = '__contains__'; qname
sage.categories.algebras.Algebras.__contains__
sage: qname[4] = 'ParentMethods'
sage: qname[8] = 'from_base_ring'; qname
sage.categories.algebras.Algebras.ParentMethods.from_base_ring
```

```
>>> from sage.all import *
>>> from sage.doctest.util import NestedName
>>> qname = NestedName('sage.categories.algebras')
>>> qname[Integer(0)] = 'Algebras'; qname
sage.categories.algebras.Algebras
>>> qname[Integer(4)] = '__contains__'; qname
sage.categories.algebras.Algebras.__contains__
>>> qname[Integer(4)] = 'ParentMethods'
>>> qname[Integer(8)] = 'from_base_ring'; qname
sage.categories.algebras.Algebras.ParentMethods.from_base_ring
```

class sage.doctest.util.**RecordingDict**(**args*, ***kwds*)

Bases: dict

This dictionary is used for tracking the dependencies of an example.

This feature allows examples in different doctests to be grouped for better timing data. It's obtained by recording whenever anything is set or retrieved from this dictionary.

EXAMPLES:

```
sage: from sage.doctest.util import RecordingDict
sage: D = RecordingDict(test=17)
sage: D.got
set()
sage: D['test']
17
sage: D.got
{'test'}
sage: D.set
set()
sage: D['a'] = 1
sage: D['a']
1
sage: D.set
{'a'}
sage: D.got
{'test'}
```

```
>>> from sage.all import *
>>> from sage.doctest.util import RecordingDict
>>> D = RecordingDict(test=Integer(17))
>>> D.got
set()
>>> D['test']
17
>>> D.got
{'test'}
>>> D.set
set()
>>> D['a'] = Integer(1)
>>> D['a']
1
>>> D.set
{'a'}
>>> D.got
{'test'}
```

copy()

Note that set and got are not copied.

EXAMPLES:

```
sage: from sage.doctest.util import RecordingDict
sage: D = RecordingDict(d = 42)
sage: D['a'] = 4
sage: D.set
{'a'}
sage: E = D.copy()
sage: E.set
set()
sage: sorted(E.keys())
['a', 'd']
```

```
>>> from sage.all import *
>>> from sage.doctest.util import RecordingDict
>>> D = RecordingDict(d = Integer(42))
>>> D['a'] = Integer(4)
>>> D.set
{'a'}
>>> E = D.copy()
>>> E.set
set()
>>> sorted(E.keys())
['a', 'd']
```

get(*name*, *default=None*)

EXAMPLES:

```
sage: from sage.doctest.util import RecordingDict
sage: D = RecordingDict(d = 42)
sage: D.get('d')
42
sage: D.got
{'d'}
sage: D.get('not_here')
sage: sorted(list(D.got))
['d', 'not_here']
```

```
>>> from sage.all import *
>>> from sage.doctest.util import RecordingDict
>>> D = RecordingDict(d = Integer(42))
>>> D.get('d')
42
>>> D.got
{'d'}
>>> D.get('not_here')
>>> sorted(list(D.got))
['d', 'not_here']
```

start()

We track which variables have been set or retrieved. This function initializes these lists to be empty.

EXAMPLES:

```
sage: from sage.doctest.util import RecordingDict
sage: D = RecordingDict(d = 42)
sage: D.set
set()
sage: D['a'] = 4
sage: D.set
{'a'}
sage: D.start(); D.set
set()
```

```
>>> from sage.all import *
>>> from sage.doctest.util import RecordingDict
```

(continues on next page)

(continued from previous page)

```
>>> D = RecordingDict(d = Integer(42))
>>> D.set
set()
>>> D['a'] = Integer(4)
>>> D.set
{'a'}
>>> D.start(); D.set
set()
```

class sage.doctest.util.**Timer**

Bases: object

A simple timer.

EXAMPLES:

```
sage: from sage.doctest.util import Timer
sage: Timer()
{}
sage: TestSuite(Timer()).run()
```

```
>>> from sage.all import *
>>> from sage.doctest.util import Timer
>>> Timer()
{}
>>> TestSuite(Timer()).run()
```

annotate(*object*)

Annotates the given object with the cputime and walltime stored in this timer.

EXAMPLES:

```
sage: from sage.doctest.util import Timer
sage: Timer().start().annotate(EllipticCurve)
sage: EllipticCurve.cputime # random
2.817255
sage: EllipticCurve.walltime # random
1332649288.410404
```

```
>>> from sage.all import *
>>> from sage.doctest.util import Timer
>>> Timer().start().annotate(EllipticCurve)
>>> EllipticCurve.cputime # random
2.817255
>>> EllipticCurve.walltime # random
1332649288.410404
```

start()

Start the timer.

Can be called multiple times to reset the timer.

EXAMPLES:

```
sage: from sage.doctest.util import Timer
sage: Timer().start()
{'cputime': ..., 'walltime': ...}
```

```
>>> from sage.all import *
>>> from sage.doctest.util import Timer
>>> Timer().start()
{'cputime': ..., 'walltime': ...}
```

stop()

Stops the timer, recording the time that has passed since it was started.

EXAMPLES:

```
sage: from sage.doctest.util import Timer
sage: import time
sage: timer = Timer().start()
sage: time.sleep(float(0.5))
sage: timer.stop()
{'cputime': ..., 'walltime': ...}
```

```
>>> from sage.all import *
>>> from sage.doctest.util import Timer
>>> import time
>>> timer = Timer().start()
>>> time.sleep(float(RealNumber('0.5')))
>>> timer.stop()
{'cputime': ..., 'walltime': ...}
```

sage.doctest.util.**count_noun**(*number*, *noun*, *plural=None*, *pad_number=False*, *pad_noun=False*)

EXAMPLES:

```
sage: from sage.doctest.util import count_noun
sage: count_noun(1, "apple")
'1 apple'
sage: count_noun(1, "apple", pad_noun=True)
'1 apple '
sage: count_noun(1, "apple", pad_number=3)
'  1 apple'
sage: count_noun(2, "orange")
'2 oranges'
sage: count_noun(3, "peach", "peaches")
'3 peaches'
sage: count_noun(1, "peach", plural='peaches', pad_noun=True)
'1 peach  '
```

```
>>> from sage.all import *
>>> from sage.doctest.util import count_noun
>>> count_noun(Integer(1), "apple")
'1 apple'
>>> count_noun(Integer(1), "apple", pad_noun=True)
'1 apple '
```

(continues on next page)

(continued from previous page)

```
>>> count_noun(Integer(1), "apple", pad_number=Integer(3))
'  1 apple'
>>> count_noun(Integer(2), "orange")
'2 oranges'
>>> count_noun(Integer(3), "peach", "peaches")
'3 peaches'
>>> count_noun(Integer(1), "peach", plural='peaches', pad_noun=True)
'1 peach  '
```

sage.doctest.util.**dict_difference**(*self*, *other*)

Return a dict with all key-value pairs occurring in `self` but not in `other`.

EXAMPLES:

```
sage: from sage.doctest.util import dict_difference
sage: d1 = {1: 'a', 2: 'b', 3: 'c'}
sage: d2 = {1: 'a', 2: 'x', 4: 'c'}
sage: dict_difference(d2, d1)
{2: 'x', 4: 'c'}
```

```
>>> from sage.all import *
>>> from sage.doctest.util import dict_difference
>>> d1 = {Integer(1): 'a', Integer(2): 'b', Integer(3): 'c'}
>>> d2 = {Integer(1): 'a', Integer(2): 'x', Integer(4): 'c'}
>>> dict_difference(d2, d1)
{2: 'x', 4: 'c'}
```

```
sage: from sage.doctest.control import DocTestDefaults
sage: D1 = DocTestDefaults()
sage: D2 = DocTestDefaults(foobar='hello', timeout=100)
sage: dict_difference(D2.__dict__, D1.__dict__)
{'foobar': 'hello', 'timeout': 100}
```

```
>>> from sage.all import *
>>> from sage.doctest.control import DocTestDefaults
>>> D1 = DocTestDefaults()
>>> D2 = DocTestDefaults(foobar='hello', timeout=Integer(100))
>>> dict_difference(D2.__dict__, D1.__dict__)
{'foobar': 'hello', 'timeout': 100}
```

sage.doctest.util.**ensure_interruptible_after**(**args*, ***kwds*)

Helper function for doctesting to ensure that the code is interruptible after a certain amount of time. This should only be used for internal doctesting purposes.

EXAMPLES:

```
sage: from sage.doctest.util import ensure_interruptible_after
sage: with ensure_interruptible_after(1) as data: sleep(3)
```

```
>>> from sage.all import *
>>> from sage.doctest.util import ensure_interruptible_after
>>> with ensure_interruptible_after(Integer(1)) as data: sleep(Integer(3))
```

`as data` is optional, but if it is used, it will contain a few useful values:

```
sage: data  # abs tol 1
{'alarm_raised': True, 'elapsed': 1.0}
```

```
>>> from sage.all import *
>>> data  # abs tol 1
{'alarm_raised': True, 'elapsed': 1.0}
```

`max_wait_after_interrupt` can be passed if the function may take longer than usual to be interrupted:

```
sage: # needs sage.misc.cython
sage: cython(r'''
....: from posix.time cimport clock_gettime, CLOCK_REALTIME, timespec, time_t
....: from cysignals.signals cimport sig_check
....:
....: cpdef void uninterruptible_sleep(double seconds):
....:     cdef timespec start_time, target_time
....:     clock_gettime(CLOCK_REALTIME, &start_time)
....:
....:     cdef time_t floor_seconds = <time_t>seconds
....:     target_time.tv_sec = start_time.tv_sec + floor_seconds
....:     target_time.tv_nsec = start_time.tv_nsec + <long>((seconds - floor_
↪seconds) * 1e9)
....:     if target_time.tv_nsec >= 1000000000:
....:         target_time.tv_nsec -= 1000000000
....:         target_time.tv_sec += 1
....:
....:     while True:
....:         clock_gettime(CLOCK_REALTIME, &start_time)
....:         if start_time.tv_sec > target_time.tv_sec or (start_time.tv_sec ==␣
↪target_time.tv_sec and start_time.tv_nsec >= target_time.tv_nsec):
....:             break
....:
....: cpdef void check_interrupt_only_occasionally():
....:     for i in range(10):
....:         uninterruptible_sleep(0.8)
....:         sig_check()
....: ''')
sage: with ensure_interruptible_after(1):  # not passing max_wait_after_interrupt␣
↪will raise an error
....:     check_interrupt_only_occasionally()
Traceback (most recent call last):
...
RuntimeError: Function is not interruptible within 1.0000 seconds, only after 1.6.
↪.. seconds
sage: with ensure_interruptible_after(1, max_wait_after_interrupt=0.9):
....:     check_interrupt_only_occasionally()
```

```
>>> from sage.all import *
>>> # needs sage.misc.cython
>>> cython(r'''
... from posix.time cimport clock_gettime, CLOCK_REALTIME, timespec, time_t
```

(continues on next page)

(continued from previous page)

```
... from cysignals.signals cimport sig_check
....:
>>> cpdef void uninterruptible_sleep(double seconds):
...     cdef timespec start_time, target_time
...     clock_gettime(CLOCK_REALTIME, &start_time)
....:
>>>     cdef time_t floor_seconds = <time_t>seconds
...     target_time.tv_sec = start_time.tv_sec + floor_seconds
...     target_time.tv_nsec = start_time.tv_nsec + <long>((seconds - floor_
↪seconds) * RealNumber('1e9'))
...     if target_time.tv_nsec >= Integer(1000000000):
...         target_time.tv_nsec -= Integer(1000000000)
...         target_time.tv_sec += Integer(1)
....:
>>>     while True:
...         clock_gettime(CLOCK_REALTIME, &start_time)
...         if start_time.tv_sec > target_time.tv_sec or (start_time.tv_sec ==␣
↪target_time.tv_sec and start_time.tv_nsec >= target_time.tv_nsec):
...             break
....:
>>> cpdef void check_interrupt_only_occasionally():
...     for i in range(Integer(10)):
...         uninterruptible_sleep(RealNumber('0.8'))
...         sig_check()
... ''')
>>> with ensure_interruptible_after(Integer(1)):  # not passing max_wait_after_
↪interrupt will raise an error
...     check_interrupt_only_occasionally()
Traceback (most recent call last):
...
RuntimeError: Function is not interruptible within 1.0000 seconds, only after 1.6.
↪.. seconds
>>> with ensure_interruptible_after(Integer(1), max_wait_after_
↪interrupt=RealNumber('0.9')):
...     check_interrupt_only_occasionally()
```

sage.doctest.util.**make_recording_dict**(*D*, *st*, *gt*)

Auxiliary function for pickling.

EXAMPLES:

```
sage: from sage.doctest.util import make_recording_dict
sage: D = make_recording_dict({'a':4,'d':42},set(),set(['not_here']))
sage: sorted(D.items())
[('a', 4), ('d', 42)]
sage: D.got
{'not_here'}
```

```
>>> from sage.all import *
>>> from sage.doctest.util import make_recording_dict
>>> D = make_recording_dict({'a':Integer(4),'d':Integer(42)},set(),set(['not_here
↪']))
```

(continues on next page)

(continued from previous page)

```
>>> sorted(D.items())
[('a', 4), ('d', 42)]
>>> D.got
{'not_here'}
```

CHAPTER

TEN

FIXTURES TO HELP TESTING FUNCTIONALITY

Utilities which modify or replace code to help with doctesting functionality. Wrappers, proxies and mockups are typical examples of fixtures.

AUTHORS:

- Martin von Gagern (2014-12-15): AttributeAccessTracerProxy and trace_method
- Martin von Gagern (2015-01-02): Factor out TracerHelper and reproducible_repr

EXAMPLES:

You can use *trace_method()* to see how a method communicates with its surroundings:

```
sage: class Foo():
....:     def f(self):
....:         self.y = self.g(self.x)
....:     def g(self, arg):
....:         return arg + 1
....:
sage: foo = Foo()
sage: foo.x = 3
sage: from sage.doctest.fixtures import trace_method
sage: trace_method(foo, "f")
sage: foo.f()
enter f()
  read x = 3
  call g(3) -> 4
  write y = 4
exit f -> None
```

```
>>> from sage.all import *
>>> class Foo():
...     def f(self):
...         self.y = self.g(self.x)
...     def g(self, arg):
...         return arg + Integer(1)
....:
>>> foo = Foo()
>>> foo.x = Integer(3)
>>> from sage.doctest.fixtures import trace_method
>>> trace_method(foo, "f")
>>> foo.f()
enter f()
```

(continues on next page)

(continued from previous page)

```
  read x = 3
  call g(3) -> 4
  write y = 4
exit f -> None
```

class sage.doctest.fixtures.**AttributeAccessTracerHelper**(*delegate*, *prefix=' '*, *reads=True*)

Bases: object

Helper to print proxied access to attributes.

This class does the actual printing of access traces for objects proxied by *AttributeAccessTracerProxy*. The fact that it's not a proxy at the same time helps avoiding complicated attribute access syntax.

INPUT:

- delegate – the actual object to be proxied
- prefix – (default: " ") string to prepend to each printed output
- reads – (default: True) whether to trace read access as well

EXAMPLES:

```
sage: class Foo():
....:     def f(self, *args):
....:         return self.x*self.x
....:
sage: foo = Foo()
sage: from sage.doctest.fixtures import AttributeAccessTracerHelper
sage: pat = AttributeAccessTracerHelper(foo)
sage: pat.set("x", 2)
  write x = 2
sage: pat.get("x")
  read x = 2
2
sage: pat.get("f")(3)
  call f(3) -> 4
4
```

```
>>> from sage.all import *
>>> class Foo():
...     def f(self, *args):
...         return self.x*self.x
....:
>>> foo = Foo()
>>> from sage.doctest.fixtures import AttributeAccessTracerHelper
>>> pat = AttributeAccessTracerHelper(foo)
>>> pat.set("x", Integer(2))
  write x = 2
>>> pat.get("x")
  read x = 2
2
>>> pat.get("f")(Integer(3))
  call f(3) -> 4
4
```

get(*name*)

Read an attribute from the wrapped delegate object.

If that value is a method (i.e. a callable object which is not contained in the dictionary of the object itself but instead inherited from some class) then it is replaced by a wrapper function to report arguments and return value. Otherwise an attribute read access is reported.

EXAMPLES:

```
sage: class Foo():
....:     def f(self, *args):
....:         return self.x*self.x
....:
sage: foo = Foo()
sage: foo.x = 2
sage: from sage.doctest.fixtures import AttributeAccessTracerHelper
sage: pat = AttributeAccessTracerHelper(foo)
sage: pat.get("x")
  read x = 2
2
sage: pat.get("f")(3)
  call f(3) -> 4
4
```

```
>>> from sage.all import *
>>> class Foo():
...     def f(self, *args):
...         return self.x*self.x
....:
>>> foo = Foo()
>>> foo.x = Integer(2)
>>> from sage.doctest.fixtures import AttributeAccessTracerHelper
>>> pat = AttributeAccessTracerHelper(foo)
>>> pat.get("x")
  read x = 2
2
>>> pat.get("f")(Integer(3))
  call f(3) -> 4
4
```

set(*name*, *val*)

Write an attribute to the wrapped delegate object.

The name and new value are also reported in the output.

EXAMPLES:

```
sage: class Foo():
....:     pass
....:
sage: foo = Foo()
sage: from sage.doctest.fixtures import AttributeAccessTracerHelper
sage: pat = AttributeAccessTracerHelper(foo)
sage: pat.set("x", 2)
  write x = 2
```

(continues on next page)

(continued from previous page)

```
sage: foo.x
2
```

```
>>> from sage.all import *
>>> class Foo():
...     pass
....:
>>> foo = Foo()
>>> from sage.doctest.fixtures import AttributeAccessTracerHelper
>>> pat = AttributeAccessTracerHelper(foo)
>>> pat.set("x", Integer(2))
  write x = 2
>>> foo.x
2
```

class sage.doctest.fixtures.**AttributeAccessTracerProxy**(*delegate*, ***kwds*)

Bases: object

Proxy object which prints all attribute and method access to an object.

The implementation is kept lean since all access to attributes of the proxy itself requires complicated syntax. For this reason, the actual handling of attribute access is delegated to a *AttributeAccessTracerHelper*.

INPUT:

- delegate – the actual object to be proxied
- prefix – (default: " ") string to prepend to each printed output
- reads – (default: True) whether to trace read access as well

EXAMPLES:

```
sage: class Foo():
....:     def f(self, *args):
....:         return self.x*self.x
....:
sage: foo = Foo()
sage: from sage.doctest.fixtures import AttributeAccessTracerProxy
sage: pat = AttributeAccessTracerProxy(foo)
sage: pat.x = 2
  write x = 2
sage: pat.x
  read x = 2
2
sage: pat.f(3)
  call f(3) -> 4
4
```

```
>>> from sage.all import *
>>> class Foo():
...     def f(self, *args):
...         return self.x*self.x
....:
>>> foo = Foo()
```

(continues on next page)

(continued from previous page)

```
>>> from sage.doctest.fixtures import AttributeAccessTracerProxy
>>> pat = AttributeAccessTracerProxy(foo)
>>> pat.x = Integer(2)
  write x = 2
>>> pat.x
  read x = 2
2
>>> pat.f(Integer(3))
  call f(3) -> 4
4
```

__getattribute__(*name*)

Read an attribute from the wrapped delegate object.

If that value is a method (i.e. a callable object which is not contained in the dictionary of the object itself but instead inherited from some class) then it is replaced by a wrapper function to report arguments and return value. Otherwise an attribute read access is reported.

EXAMPLES:

```
sage: class Foo():
....:     def f(self, *args):
....:         return self.x*self.x
....:
sage: foo = Foo()
sage: foo.x = 2
sage: from sage.doctest.fixtures import AttributeAccessTracerProxy
sage: pat = AttributeAccessTracerProxy(foo)
sage: pat.x
  read x = 2
2
sage: pat.f(3)
  call f(3) -> 4
4
```

```
>>> from sage.all import *
>>> class Foo():
...     def f(self, *args):
...         return self.x*self.x
....:
>>> foo = Foo()
>>> foo.x = Integer(2)
>>> from sage.doctest.fixtures import AttributeAccessTracerProxy
>>> pat = AttributeAccessTracerProxy(foo)
>>> pat.x
  read x = 2
2
>>> pat.f(Integer(3))
  call f(3) -> 4
4
```

__setattr__(*name*, *val*)

Write an attribute to the wrapped delegate object.

The name and new value are also reported in the output.

EXAMPLES:

```
sage: class Foo():
....:     pass
....:
sage: foo = Foo()
sage: from sage.doctest.fixtures import AttributeAccessTracerProxy
sage: pat = AttributeAccessTracerProxy(foo)
sage: pat.x = 2
  write x = 2
sage: foo.x
2
```

```
>>> from sage.all import *
>>> class Foo():
...     pass
....:
>>> foo = Foo()
>>> from sage.doctest.fixtures import AttributeAccessTracerProxy
>>> pat = AttributeAccessTracerProxy(foo)
>>> pat.x = Integer(2)
  write x = 2
>>> foo.x
2
```

sage.doctest.fixtures.**reproducible_repr**(*val*)

String representation of an object in a reproducible way.

Note

This function is deprecated, in most cases it suffices to use the automatic sorting of dictionary keys and set items by a displayhook. See *sage.doctest.forker.init_sage()*. If used in a format string, use IPython.lib.pretty.pretty(). In the rare cases where the ordering of the elements is not reliable or transitive, sorted with a sane key can be used instead.

This tries to ensure that the returned string does not depend on factors outside the control of the doctest. One example is the order of elements in a hash-based structure. For most objects, this is simply the repr of the object.

All types for which special handling had been implemented are covered by the examples below. If a doctest requires special handling for additional types, this function may be extended appropriately. It is an error if an argument to this function has a non-reproducible repr implementation and is not explicitly mentioned in an example case below.

INPUT:

- val – an object to be represented

OUTPUT:

A string representation of that object, similar to what repr returns but for certain cases with more guarantees to ensure exactly the same result for semantically equivalent objects.

EXAMPLES:

```
sage: # not tested (test fails because of deprecation warning)
sage: from sage.doctest.fixtures import reproducible_repr
sage: print(reproducible_repr(set(["a", "c", "b", "d"])))
set(['a', 'b', 'c', 'd'])
sage: print(reproducible_repr(frozenset(["a", "c", "b", "d"])))
frozenset(['a', 'b', 'c', 'd'])
sage: print(reproducible_repr([1, frozenset("cab"), set("bar"), 0]))
[1, frozenset(['a', 'b', 'c']), set(['a', 'b', 'r']), 0]
sage: print(reproducible_repr({3.0: "three", "2": "two", 1: "one"}))            #␣
↪needs sage.rings.real_mpfr
{'2': 'two', 1: 'one', 3.00000000000000: 'three'}
sage: print(reproducible_repr("foo\nbar"))  # demonstrate default case
'foo\nbar'
```

```
>>> from sage.all import *
>>> # not tested (test fails because of deprecation warning)
>>> from sage.doctest.fixtures import reproducible_repr
>>> print(reproducible_repr(set(["a", "c", "b", "d"])))
set(['a', 'b', 'c', 'd'])
>>> print(reproducible_repr(frozenset(["a", "c", "b", "d"])))
frozenset(['a', 'b', 'c', 'd'])
>>> print(reproducible_repr([Integer(1), frozenset("cab"), set("bar"),␣
↪Integer(0)]))
[1, frozenset(['a', 'b', 'c']), set(['a', 'b', 'r']), 0]
>>> print(reproducible_repr({RealNumber('3.0'): "three", "2": "two", Integer(1):
↪"one"}))            # needs sage.rings.real_mpfr
{'2': 'two', 1: 'one', 3.00000000000000: 'three'}
>>> print(reproducible_repr("foo\nbar"))  # demonstrate default case
'foo\nbar'
```

sage.doctest.fixtures.**trace_method**(*obj*, *meth*, ***kwds*)

Trace the doings of a given method. It prints method entry with arguments, access to members and other methods during method execution as well as method exit with return value.

INPUT:

- `obj` – the object containing the method
- `meth` – the name of the method to be traced
- `prefix` – (default: `" "`) string to prepend to each printed output
- `reads` – (default: `True`) whether to trace read access as well

EXAMPLES:

```
sage: class Foo():
....:     def f(self, arg=None):
....:         self.y = self.g(self.x)
....:         if arg: return arg*arg
....:     def g(self, arg):
....:         return arg + 1
....:
sage: foo = Foo()
sage: foo.x = 3
```

(continues on next page)

(continued from previous page)

```
sage: from sage.doctest.fixtures import trace_method
sage: trace_method(foo, "f")
sage: foo.f()
enter f()
  read x = 3
  call g(3) -> 4
  write y = 4
exit f -> None
sage: foo.f(3)
enter f(3)
  read x = 3
  call g(3) -> 4
  write y = 4
exit f -> 9
9
```

```
>>> from sage.all import *
>>> class Foo():
...     def f(self, arg=None):
...         self.y = self.g(self.x)
...         if arg: return arg*arg
...     def g(self, arg):
...         return arg + Integer(1)
....:
>>> foo = Foo()
>>> foo.x = Integer(3)
>>> from sage.doctest.fixtures import trace_method
>>> trace_method(foo, "f")
>>> foo.f()
enter f()
  read x = 3
  call g(3) -> 4
  write y = 4
exit f -> None
>>> foo.f(Integer(3))
enter f(3)
  read x = 3
  call g(3) -> 4
  write y = 4
exit f -> 9
9
```

CHAPTER

ELEVEN

INDICES AND TABLES

- Index
- Module Index
- Search Page

PYTHON MODULE INDEX

d

sage.doctest.control, 1
sage.doctest.external, 119
sage.doctest.fixtures, 165
sage.doctest.forker, 43
sage.doctest.marked_output, 103
sage.doctest.parsing, 75
sage.doctest.reporting, 105
sage.doctest.sources, 23
sage.doctest.test, 129
sage.doctest.util, 155

INDEX

Non-alphabetical

__getattribute__() (*sage.doctest.fixtures.AttributeAccessTracerProxy method*), 169
__setattr__() (*sage.doctest.fixtures.AttributeAccessTracerProxy method*), 169
_process_doc() (*sage.doctest.sources.DocTestSource method*), 23

A

abs_tol (*sage.doctest.marked_output.MarkedOutput attribute*), 103
add_files() (*sage.doctest.control.DocTestController method*), 1
annotate() (*sage.doctest.util.Timer method*), 158
attempted (*sage.doctest.forker.TestResults attribute*), 71
AttributeAccessTracerHelper (*class in sage.doctest.fixtures*), 166
AttributeAccessTracerProxy (*class in sage.doctest.fixtures*), 168
AvailableSoftware (*class in sage.doctest.external*), 119

B

basename() (*sage.doctest.sources.FileDocTestSource method*), 25

C

check_output() (*sage.doctest.parsing.SageOutputChecker method*), 82
cleanup() (*sage.doctest.control.DocTestController method*), 2
compile_and_execute() (*sage.doctest.forker.SageDocTestRunner method*), 53
copy() (*sage.doctest.util.RecordingDict method*), 156
count_noun() (*in module sage.doctest.util*), 159
create_doctests() (*sage.doctest.sources.FileDocTestSource method*), 26
create_doctests() (*sage.doctest.sources.StringDocTestSource method*), 37
create_run_id() (*sage.doctest.control.DocTestController method*), 3

D

detectable() (*sage.doctest.external.AvailableSoftware method*), 120
dict_difference() (*in module sage.doctest.util*), 160
DictAsObject (*class in sage.doctest.sources*), 23
dispatch() (*sage.doctest.forker.DocTestDispatcher method*), 43
do_fixup() (*sage.doctest.parsing.SageOutputChecker method*), 87
DocTestController (*class in sage.doctest.control*), 1
DocTestDefaults (*class in sage.doctest.control*), 18
DocTestDispatcher (*class in sage.doctest.forker*), 43
DocTestReporter (*class in sage.doctest.reporting*), 105
DocTestSource (*class in sage.doctest.sources*), 23
DocTestTask (*class in sage.doctest.forker*), 47
DocTestWorker (*class in sage.doctest.forker*), 48
dummy_handler() (*in module sage.doctest.forker*), 71

E

ending_docstring() (*sage.doctest.sources.PythonSource method*), 29
ending_docstring() (*sage.doctest.sources.RestSource method*), 32
ending_docstring() (*sage.doctest.sources.TexSource method*), 38
ensure_interruptible_after() (*in module sage.doctest.util*), 160
environment variable
    TERM, 129
expand_files_into_sources() (*sage.doctest.control.DocTestController method*), 3
external_features() (*in module sage.doctest.external*), 121

F

failed (*sage.doctest.forker.TestResults attribute*), 71
file_optional_tags (*sage.doctest.parsing.SageDocTestParser attribute*), 76
file_optional_tags() (*sage.doctest.sources.DocTestSource method*), 24
file_optional_tags() (*sage.doctest.sources.FileDocTestSource method*), 26

FileDocTestSource (*class in sage.doctest.sources*), 25
filter_sources() (*sage.doctest.control.DocTestController method*), 5
finalize() (*sage.doctest.reporting.DocTestReporter method*), 105
flush() (*sage.doctest.control.Logger method*), 19

G

get() (*sage.doctest.fixtures.AttributeAccessTracerHelper method*), 166
get() (*sage.doctest.util.RecordingDict method*), 157
get_basename() (*in module sage.doctest.sources*), 41
get_source() (*in module sage.doctest.parsing*), 91
getvalue() (*sage.doctest.forker.SageSpoofInOut method*), 68

H

has_4ti2() (*in module sage.doctest.external*), 122
has_cplex() (*in module sage.doctest.external*), 122
has_ffmpeg() (*in module sage.doctest.external*), 122
has_graphviz() (*in module sage.doctest.external*), 122
has_gurobi() (*in module sage.doctest.external*), 123
has_imagemagick() (*in module sage.doctest.external*), 123
has_internet() (*in module sage.doctest.external*), 123
has_latex() (*in module sage.doctest.external*), 123
has_lualatex() (*in module sage.doctest.external*), 124
has_macaulay2() (*in module sage.doctest.external*), 124
has_magma() (*in module sage.doctest.external*), 124
has_maple() (*in module sage.doctest.external*), 124
has_mathematica() (*in module sage.doctest.external*), 125
has_matlab() (*in module sage.doctest.external*), 125
has_octave() (*in module sage.doctest.external*), 125
has_pandoc() (*in module sage.doctest.external*), 125
has_pdflatex() (*in module sage.doctest.external*), 126
has_rubiks() (*in module sage.doctest.external*), 126
has_scilab() (*in module sage.doctest.external*), 126
has_xelatex() (*in module sage.doctest.external*), 126
hidden() (*sage.doctest.external.AvailableSoftware method*), 120
human_readable_escape_sequences() (*sage.doctest.parsing.SageOutputChecker method*), 89

I

in_lib() (*sage.doctest.sources.FileDocTestSource method*), 27
init_sage() (*in module sage.doctest.forker*), 71
issuperset() (*sage.doctest.external.AvailableSoftware method*), 121

K

kill() (*sage.doctest.forker.DocTestWorker method*), 49

L

load_baseline_stats() (*sage.doctest.control.DocTestController method*), 6
load_environment() (*sage.doctest.control.DocTestController method*), 6
load_stats() (*sage.doctest.control.DocTestController method*), 7
log() (*sage.doctest.control.DocTestController method*), 8
Logger (*class in sage.doctest.control*), 19
long (*sage.doctest.parsing.SageDocTestParser attribute*), 76

M

make_marked_output() (*in module sage.doctest.marked_output*), 104
make_recording_dict() (*in module sage.doctest.util*), 162
MarkedOutput (*class in sage.doctest.marked_output*), 103
module
    sage.doctest.control, 1
    sage.doctest.external, 119
    sage.doctest.fixtures, 165
    sage.doctest.forker, 43
    sage.doctest.marked_output, 103
    sage.doctest.parsing, 75
    sage.doctest.reporting, 105
    sage.doctest.sources, 23
    sage.doctest.test, 129
    sage.doctest.util, 155

N

NestedName (*class in sage.doctest.util*), 155

O

optional_only (*sage.doctest.parsing.SageDocTestParser attribute*), 76
optional_tags (*sage.doctest.parsing.SageDocTestParser attribute*), 76
optionals (*sage.doctest.parsing.SageDocTestParser attribute*), 76
OriginalSource (*class in sage.doctest.parsing*), 75
output_difference() (*sage.doctest.parsing.SageOutputChecker method*), 89

P

parallel_dispatch() (*sage.doctest.forker.DocTestDispatcher method*), 44
parse() (*sage.doctest.parsing.SageDocTestParser method*), 76
parse_docstring() (*sage.doctest.sources.RestSource method*), 32
parse_docstring() (*sage.doctest.sources.SourceLanguage method*), 35

parse_file_optional_tags() (*in module sage.doctest.parsing*), 92
parse_optional_tags() (*in module sage.doctest.parsing*), 93
parse_tolerance() (*in module sage.doctest.parsing*), 95
pre_hash() (*in module sage.doctest.parsing*), 96
printpath() (*sage.doctest.sources.FileDocTestSource method*), 28
probed_tags (*sage.doctest.parsing.SageDocTestParser attribute*), 81
PythonSource (*class in sage.doctest.sources*), 29

R

random (*sage.doctest.marked_output.MarkedOutput attribute*), 103
read_messages() (*sage.doctest.forker.DocTestWorker method*), 51
RecordingDict (*class in sage.doctest.util*), 155
reduce_hex() (*in module sage.doctest.parsing*), 96
rel_tol (*sage.doctest.marked_output.MarkedOutput attribute*), 103
report() (*sage.doctest.reporting.DocTestReporter method*), 108
report_failure() (*sage.doctest.forker.SageDocTestRunner method*), 56
report_head() (*sage.doctest.reporting.DocTestReporter method*), 115
report_overtime() (*sage.doctest.forker.SageDocTestRunner method*), 58
report_start() (*sage.doctest.forker.SageDocTestRunner method*), 60
report_success() (*sage.doctest.forker.SageDocTestRunner method*), 60
report_unexpected_exception() (*sage.doctest.forker.SageDocTestRunner method*), 62
reproducible_repr() (*in module sage.doctest.fixtures*), 170
RestSource (*class in sage.doctest.sources*), 31
run() (*sage.doctest.control.DocTestController method*), 9
run() (*sage.doctest.forker.DocTestWorker method*), 51
run() (*sage.doctest.forker.SageDocTestRunner method*), 63
run_doctests() (*in module sage.doctest.control*), 19
run_doctests() (*sage.doctest.control.DocTestController method*), 14
run_val_gdb() (*sage.doctest.control.DocTestController method*), 14

S

sage.doctest.control
    module, 1
sage.doctest.external
    module, 119
sage.doctest.fixtures
    module, 165
sage.doctest.forker
    module, 43
sage.doctest.marked_output
    module, 103
SageDocTestParser (*class in sage.doctest.parsing*), 76
sage.doctest.parsing
    module, 75
sage.doctest.reporting
    module, 105
SageDocTestRunner (*class in sage.doctest.forker*), 52
sage.doctest.sources
    module, 23
sage.doctest.test
    module, 129
sage.doctest.util
    module, 155
SageOutputChecker (*class in sage.doctest.parsing*), 81
SageSpoofInOut (*class in sage.doctest.forker*), 67
save_result_output() (*sage.doctest.forker.DocTestWorker method*), 51
save_stats() (*sage.doctest.control.DocTestController method*), 15
seen() (*sage.doctest.external.AvailableSoftware method*), 121
serial_dispatch() (*sage.doctest.forker.DocTestDispatcher method*), 46
set() (*sage.doctest.fixtures.AttributeAccessTracerHelper method*), 167
showwarning_with_traceback() (*in module sage.doctest.forker*), 73
signal_name() (*in module sage.doctest.reporting*), 116
skipdir() (*in module sage.doctest.control*), 20
skipfile() (*in module sage.doctest.control*), 20
sort_sources() (*sage.doctest.control.DocTestController method*), 16
source_baseline() (*sage.doctest.control.DocTestController method*), 17
SourceLanguage (*class in sage.doctest.sources*), 34
start() (*sage.doctest.forker.DocTestWorker method*), 52
start() (*sage.doctest.util.RecordingDict method*), 157
start() (*sage.doctest.util.Timer method*), 158
start_finish_can_overlap (*sage.doctest.sources.PythonSource attribute*), 30
start_finish_can_overlap (*sage.doctest.sources.RestSource attribute*), 33
start_finish_can_overlap (*sage.doctest.sources.TexSource attribute*), 39
start_spoofing() (*sage.doctest.forker.SageSpoofInOut method*), 69
starting_docstring() (*sage.doctest.sources.PythonSource method*), 30

starting_docstring() (*sage.doctest.sources.RestSource method*), 34
starting_docstring() (*sage.doctest.sources.TexSource method*), 39
stop() (*sage.doctest.util.Timer method*), 159
stop_spoofing() (*sage.doctest.forker.SageSpoofInOut method*), 70
StringDocTestSource (*class in sage.doctest.sources*), 35
summarize() (*sage.doctest.forker.SageDocTestRunner method*), 64

T

TERM, 129
TestResults (*class in sage.doctest.forker*), 71
TexSource (*class in sage.doctest.sources*), 37
Timer (*class in sage.doctest.util*), 158
tol (*sage.doctest.marked_output.MarkedOutput attribute*), 103
trace_method() (*in module sage.doctest.fixtures*), 171

U

unparse_optional_tags() (*in module sage.doctest.parsing*), 97
update() (*sage.doctest.marked_output.MarkedOutput method*), 103
update_digests() (*sage.doctest.forker.SageDocTestRunner method*), 65
update_optional_tags() (*in module sage.doctest.parsing*), 97
update_results() (*sage.doctest.forker.SageDocTestRunner method*), 66

W

were_doctests_with_optional_tag_run() (*sage.doctest.reporting.DocTestReporter method*), 115
write() (*sage.doctest.control.Logger method*), 19